

NSI

Terminale

Édition de travail 2026-2027

ÉDITION PROFESSEUR

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

NSI

Terminale

ÉDITION PROFESSEUR

Édition 2026-2027



NEXUS ♦ RÉUSSITE

Sommaire

1	1	Algorithmes sur les arbres et les graphes	1
	1.1	Taille, hauteur et parcours d'un arbre binaire	1
	1.2	Arbre binaire de recherche	2
	1.3	Parcours d'un graphe	3
	1.4	Cycle et chemin dans un graphe	4
	1.5	Diviser pour régner	5
	1.6	Programmation dynamique	6
	1.7	Recherche d'un motif : l'algorithme de Boyer-Moore	7
2	2	Architectures matérielles, systèmes d'exploitation et réseaux	35
	2.1	Le système sur puce	35
	2.2	Processus : création, ordonnancement, interblocage	36
	2.3	Protocoles de routage	37
	2.4	Chiffrement symétrique et asymétrique	38
3	3	Bases de données relationnelles	55
	3.1	Le modèle relationnel	55
	3.2	Le système de gestion de bases de données	56
	3.3	Interroger une base : SELECT, FROM, WHERE	57
	3.4	Croiser les données : JOIN et ORDER BY	57
	3.5	Modifier une base : INSERT, UPDATE, DELETE	58
4	4	Histoire de l'informatique	79
	4.1	Événements clés de l'histoire de l'informatique	79
	4.2	Évolution des rôles du logiciel et du matériel	79
5	5	Langages, récursivité et paradigmes de programmation	89
	5.1	Programme, donnée et calculabilité	89
	5.2	Écrire et analyser un programme récursif	90
	5.3	API, bibliothèques et modules	91
	5.4	Paradigmes de programmation	92
	5.5	Mise au point : causes typiques de bugs	93
6	6	Structures de données	113
	6.1	Interface et implémentation d'une structure de données	113
	6.2	Définir et utiliser une classe en Python	114

			♦
6.3	Piles, files, et choix de structure adaptée	115	
6.4	Arbres binaires	116	
6.5	Graphes : modélisation et représentations	117	
7	7 Conduire un projet informatique	145	
7.1	Conduire un projet informatique en Terminale	145	

1 Algorithmes sur les arbres et les graphes

1.1 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0 .

Exemple.

```
1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))
```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```
1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)
```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10            file.append(noeud.gauche)
11            if noeud.droite is not None:
12                file.append(noeud.droite)
13    return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.2 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7 def rechercher(arbre, cle):
8     if arbre is None:
9         return False
10    if cle == arbre.valeur:
11        return True
12    if cle < arbre.valeur:
13        return rechercher(arbre.gauche, cle)
14    return rechercher(arbre.droite, cle)

```


◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```
1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre
```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.3 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
```

```
10 return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4         visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])` : au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.4 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7     elif voisin != parent:
```

```

8     return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets A et B revient à effectuer un parcours (BFS ou DFS) depuis A en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis B pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si B n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                predecesseur[voisin] = sommet
11                file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
14     chemin_trouve = []
15     sommet = arrivee
16     while sommet is not None:
17         chemin_trouve.append(sommet)
18         sommet = predecesseur[sommet]
19     chemin_trouve.reverse()
20     return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.5 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):

```



```
5     if gauche[i] <= droite[j]:
6         resultat.append(gauche[i])
7         i += 1
8     else:
9         resultat.append(droite[j])
10        j += 1
11    return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)
```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.6 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémore** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```
1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
```

```

8         if meilleur is None or candidat < meilleur:
9             meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p <= somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémoïsation** Sans mémoïsation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémoïsation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.7 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1 def recherche_naive(texte, motif):
2     positions = []
3     for i in range(len(texte) - len(motif) + 1):
4         if texte[i:i + len(motif)] == motif:
5             positions.append(i)
6     return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1 def table_decalage(motif):
2     table = {}
3     for i, c in enumerate(motif):
4         table[c] = i
5     return table
6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i <= n - m:
13        j = m - 1
14        while j >= 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions

```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

④ MÉTHODE M1 Dérouler à la main les parcours d'un arbre binaire

Quand l'utiliser ? Quand l'énoncé donne un arbre **dessiné** et demande la liste des valeurs dans l'ordre préfixe, infixe, suffixe, ou en largeur. Le code du cours ne t'aide pas ici : c'est toi la machine, et c'est l'ordre de tes gestes qui décide du résultat.

Pas à pas — les trois parcours en profondeur :

1. **Choisis ta place autour du nœud.** Préfixe : on écrit la racine **avant** de descendre. Infixe : on écrit la racine **entre** les deux descentes. Suffixe : on écrit la racine **après** les deux descentes.
2. **Descends toujours à gauche d'abord**, jusqu'à un sous-arbre vide. Un sous-arbre vide ne produit rien : on remonte immédiatement.
3. **Remonte et applique la règle de l'étape 1** au nœud d'où tu viens.
4. **Descends ensuite à droite**, avec la même règle, jusqu'à épuisement.
5. **Compte tes valeurs.** La liste finale doit contenir **exactement** autant de valeurs que l'arbre a de nœuds : c'est le premier contrôle à faire.

Pas à pas — le parcours en largeur : tiens un tableau à deux colonnes, l'état de la file et la valeur qu'on sort.

1. **Enfile la racine.**
2. **Défile le nœud de tête**, écris sa valeur dans le résultat.
3. **Enfile ses enfants non vides**, gauche puis droite, à la **fin** de la file.
4. **Recommence** tant que la file n'est pas vide.

Exemple : l'arbre de racine 1, de fils gauche 2 (lui-même de fils 4 et 5) et de fils droit 3.

Préfixe : on écrit 1, on descend à gauche, on écrit 2, on descend à gauche, on écrit 4 (pas d'enfants), on remonte, on descend à droite, on écrit 5, on remonte deux fois, on descend à droite, on écrit 3. Résultat : 1, 2, 4, 5, 3.

Infixe : la même descente, mais la racine s'écrit entre les deux : 4, 2, 5, 1, 3.

Suffixe : la racine après les deux : 4, 5, 2, 3, 1.

Largeur : file [1] ; on sort 1, on enfile 2 et 3 ; file [2, 3] ; on sort 2, on enfile 4 et 5 ; file [3, 4, 5] ; on sort 3, 4, 5. Résultat : 1, 2, 3, 4, 5.

Pièges :

- **Confondre préfixe et largeur.** Sur un arbre en peigne, les deux donnent la même liste ; dès que l'arbre se ramifie, elles diffèrent. Ne conclus jamais d'un exemple dégénéré.
- **Écrire la racine trop tôt en infixe.** L'infixe n'écrit la racine qu'une fois le sous-arbre gauche entièrement terminé, pas au moment où on l'atteint.
- **Enfiler les enfants au début de la file.** Une file se remplit par la fin : les enfileur au début en ferait une pile, et tu obtiendrais un parcours en profondeur.
- **Oublier un sous-arbre vide.** Un nœud à un seul enfant a bien un côté vide, qui ne produit rien mais ne dispense pas de remonter.

Vérifier : compte les valeurs — il en faut autant que de nœuds, sans répétition. Et si l'arbre est un arbre binaire de recherche, l'infixe doit sortir **croissant** : c'est le contrôle le plus rapide qui existe.

S'entraîner : exercices C3 et C4 du chapitre.

④ MÉTHODE M2 Dérouler à la main un parcours de graphe

Quand l'utiliser ? Quand l'énoncé donne un graphe et demande l'ordre de visite des sommets en largeur ou en profondeur. Contrairement à un arbre, un graphe a des cycles : sans mémoire des sommets déjà vus, tu tournes en rond. Deux états sont donc à tenir, pas un.

Pas à pas :

1. **Fixe l'ordre des voisins** et écris-le. C'est presque toujours l'ordre du dictionnaire donné par l'énoncé. Sans cette convention, deux réponses différentes sont également défendables, et la correction ne peut pas trancher.

2. **Prépare deux colonnes** : la structure d'attente, et la liste des visités. Largeur : une **file**, on prend au début, on ajoute à la fin. Profondeur : une **pile**, on prend et on ajoute du même côté — ou la récursion, qui est la même pile.
3. **Marque le sommet de départ comme visité** au moment où tu l'ajoutes, pas au moment où tu le sors. Marquer trop tard laisse un sommet entrer deux fois.
4. **Sors un sommet, écris-le dans le résultat**, puis passe ses voisins en revue **dans l'ordre fixé** : tout voisin non visité est marqué et ajouté.
5. **Recommence** tant que la structure n'est pas vide.
6. **Si des sommets restent non visités**, c'est que le graphe n'est pas connexe : l'énoncé demande alors de relancer depuis un sommet non visité, ou de conclure.

Exemple : $G = \{0 : [1, 2], 1 : [0, 3], 2 : [0, 3], 3 : [1, 2]\}$, départ 0.

En largeur : visités $\{0\}$, file $[0]$. On sort 0, voisins 1 puis 2 : les deux sont neufs, on les marque et on les enfle. File $[1, 2]$. On sort 1, voisins 0 (déjà vu) et 3 (neuf, marqué, enfilé). File $[2, 3]$. On sort 2 : 0 et 3 sont déjà vus. On sort 3. Résultat : 0, 1, 2, 3.

En profondeur (récursive) : on visite 0, on part sur 1, de 1 on part sur 3, de 3 on part sur 2 — car 1 est déjà visité — et de 2 tout est vu. Résultat : 0, 1, 3, 2.

Les deux listes contiennent les mêmes sommets et **pas** dans le même ordre : c'est normal, et c'est même tout l'intérêt de la question.

Pièges :

- ▶ **Ne pas fixer l'ordre des voisins**. L'ordre de visite en dépend entièrement : en inversant les listes du même graphe, le parcours en largeur donne 0, 2, 1, 3 au lieu de 0, 1, 2, 3.
- ▶ **Marquer un sommet au moment où on le sort**. Il a alors pu être ajouté plusieurs fois entre-temps. L'ordre final peut rester bon, mais la structure gonfle, et sur un graphe dense la copie manuscrite devient fausse.
- ▶ **Oublier la mémoire des visités**. Le cycle $0 - 1 - 3 - 2 - 0$ suffit à faire boucler l'algorithme indéfiniment.
- ▶ **Croire qu'un parcours visite tout**. Il ne visite que la composante connexe du départ. Les sommets restants doivent être mentionnés.

Vérifier : chaque sommet apparaît **une seule fois** dans le résultat, et le premier est le sommet de départ. En largeur, contrôle en plus que les sommets sortent par distance croissante au départ : d'abord les voisins directs, puis leurs voisins.

S'entraîner : exercices C7 et C8 du chapitre.

🔗 MÉTHODE M3 Structurer un algorithme diviser pour régner

Quand l'utiliser ? Quand on te demande d'écrire un algorithme « en utilisant la méthode diviser pour régner » sur un problème que le cours n'a pas traité. Le tri fusion ne te servira pas de modèle à recopier : c'est sa **structure** qu'il faut retrouver sur ton problème.

Pas à pas :

1. **Écris d'abord le cas de base**, avant tout le reste. Quelle est la plus petite instance que tu sais résoudre sans rien découper ? Une liste vide, une liste à un élément, un intervalle réduit à un point. Sans ce cas, la récursion ne s'arrête jamais.
2. **Dis comment tu divises**, et vérifie que les sous-instances sont **strictement plus petites**. Couper en deux moitiés, retirer un élément, diviser l'exposant par deux : peu importe, pourvu que la taille décroisse à chaque appel.
3. **Formule l'appel récursif** en supposant qu'il fonctionne. C'est l'étape qui demande de la confiance : tu n'as pas à dérouler la récursion, tu as à écrire ce que tu ferais si la réponse aux sous-problèmes t'était donnée.
4. **Écris la combinaison**. Comment reconstitue-t-on la réponse du problème entier à partir des réponses partielles ? C'est là que se trouve le vrai travail — fusionner deux listes triées, additionner deux comptes, prendre un maximum.

5. **Vérifie la couverture des cas.** Le cas de base et le cas général doivent couvrir toutes les tailles possibles, sans trou ni recouvrement.
6. **Estime le coût :** combien de sous-problèmes par appel, de quelle taille, et combien coûte la combinaison. Deux moitiés plus une combinaison linéaire donnent $n \log_2 n$.

Exemple : compter les occurrences d'une valeur dans une liste, par diviser pour régner.

Cas de base : une liste vide contient 0 occurrence ; une liste à un élément en contient 1 si c'est la valeur cherchée, 0 sinon.

Division : on coupe en deux moitiés au milieu ; chacune est strictement plus courte dès que la liste a au moins deux éléments.

Récursion : on suppose savoir compter dans chaque moitié.

Combinaison : le compte total est la **somme** des deux comptes — aucune occurrence n'est à cheval, puisqu'une occurrence est un élément et qu'un élément est dans une moitié ou dans l'autre.

Coût : deux sous-problèmes de taille $n/2$ et une combinaison en temps constant, donc un coût proportionnel à n — le même qu'un parcours simple. Diviser pour régner ne rend pas tout plus rapide, et le dire fait partie de la réponse.

Pièges :

- ▶ **Écrire la division avant le cas de base.** C'est l'ordre qui fait oublier le cas de base, et une récursion sans cas de base ne termine pas : couper une liste vide en deux donne deux listes vides, indéfiniment.
- ▶ **Une division qui ne fait pas décroître la taille.** Couper « la liste et elle-même », ou traiter n puis n , boucle. La décroissance stricte est ce qui garantit d'atteindre le cas de base.
- ▶ **Vouloir dérouler la récursion dans sa tête.** On écrit un algorithme récursif en supposant les appels corrects, pas en simulant trois niveaux.
- ▶ **Croire que diviser pour régner accélère toujours.** Ici le coût reste linéaire. Le gain du tri fusion vient de ce que la combinaison remplace un travail quadratique, pas du découpage lui-même.

Vérifier : teste ton algorithme sur les trois tailles qui cassent tout — 0, 1, et 2 éléments — puis compare-le à une méthode directe sur une dizaine de cas au hasard. Si les deux divergent une seule fois, c'est la combinaison qu'il faut relire.

S'entraîner : exercices C11 du chapitre.

🔗 MÉTHODE M4 Passer d'une récursion naïve à une version mémorisée

Quand l'utiliser ? Quand l'énoncé demande de « résoudre par programmation dynamique » ou signale qu'une fonction récursive est « trop lente ». La programmation dynamique n'est pas un nouvel algorithme à inventer : c'est une transformation mécanique d'un algorithme récursif que tu as déjà.

Pas à pas :

1. **Écris d'abord la version naïve**, même si tu sais qu'elle sera lente. Sans elle, tu n'as rien à transformer, et c'est elle qui porte la logique du problème.
2. **Vérifie que les sous-problèmes se RECOUPENT.** Déroule deux niveaux d'appels : si le même argument revient dans deux branches différentes, la mémorisation servira. S'il ne revient jamais — comme dans le tri fusion — elle ne gagnerait rien.
3. **Identifie la clé :** quel argument détermine entièrement le résultat ? C'est lui qui indexera le dictionnaire. Si deux arguments varient, la clé est le couple.
4. **Ajoute le dictionnaire en paramètre**, avec la valeur par défaut None, et initialise-le à l'intérieur de la fonction. Jamais `memo={}` dans la signature.
5. **Encadre le corps de deux lignes :** au début, « si la clé est connue, renvoie la valeur mémorisée » ; à la fin, « enregistre le résultat avant de le renvoyer ».
6. **Contrôle que le résultat n'a pas changé** sur plusieurs valeurs. La mémorisation accélère ; elle ne doit rien calculer d'autre.

Exemple : la suite de Fibonacci.

Version naïve : $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, avec $\text{fib}(0)=0$ et $\text{fib}(1)=1$.

Recoupement : $\text{fib}(5)$ appelle $\text{fib}(4)$ et $\text{fib}(3)$; mais $\text{fib}(4)$ rappelle $\text{fib}(3)$. Le même sous-problème est calculé deux fois, et ce doublement se répète à chaque niveau.

Clé : l'entier n suffit.

Après transformation, $\text{fib}(30)$ demande 31 calculs distincts au lieu de plus de deux millions d'appels.

Pièges :

- ▶ **Mémoïser un algorithme dont les sous-problèmes ne se recoupent pas.** Le tri fusion ne rencontre jamais deux fois la même sous-liste : le dictionnaire ne ferait qu'occuper de la mémoire. Vérifie le recoupement avant de transformer.
- ▶ **Mettre le dictionnaire par défaut dans la signature.** `memo={}` est créé une seule fois, à la définition, et **partagé** entre tous les appels : deux calculs indépendants se contaminent. Toujours `memo=None`.
- ▶ **Oublier d'enregistrer avant de renvoyer.** La fonction reste correcte et reste lente : c'est le pire cas, parce que rien ne le signale.
- ▶ **Choisir une clé incomplète.** Si deux arguments influencent le résultat et que la clé n'en retient qu'un, la mémoïsation renvoie la valeur d'un autre sous-problème : l'algorithme devient faux, pas seulement lent.

Vérifier : compare naïve et mémoïsée sur toutes les valeurs jusqu'à 20 — elles doivent coïncider exactement — puis compte les entrées du dictionnaire : il doit y en avoir autant que de sous-problèmes distincts, pas davantage.

S'entraîner : exercices C12 du chapitre.

Exercice 1

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 2

En utilisant les fonctions `inserer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 3

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 4

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 5 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

ALGORITHMES SUR LES ARBRES ET LES GRAPHS — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] La taille d'un arbre binaire se calcule récursivement par :
 - A. $1 + \text{taille}(\text{gauche}) + \text{taille}(\text{droite})$, l'arbre vide ayant pour taille 0.
 - B. $1 + \max(\text{taille}(\text{gauche}), \text{taille}(\text{droite}))$.
 - C. le nombre de feuilles seulement.
 - D. $\text{taille}(\text{gauche}) \times \text{taille}(\text{droite})$.

Capacité C2

2. [Q2] Selon la convention du cours, la hauteur d'un arbre réduit à un seul nœud vaut :
 - A. 0.
 - B. -1.
 - C. 1.
 - D. indéfinie.

Capacité C3

3. [Q3] Le parcours INFIXE d'un arbre binaire visite :
 - A. la racine, puis le sous-arbre gauche, puis le sous-arbre droit.
 - B. le sous-arbre gauche, puis la racine, puis le sous-arbre droit.
 - C. le sous-arbre gauche, puis le sous-arbre droit, puis la racine.
 - D. les nœuds niveau par niveau, de haut en bas.

Capacité C4

4. [Q4] Le parcours en largeur d'un arbre s'appuie sur :
 - A. une pile.
 - B. un dictionnaire, et rien d'autre.
 - C. une file.
 - D. aucune structure auxiliaire.

Capacité C5

5. [Q5] Dans un arbre binaire de recherche, la recherche d'une clé plus petite que celle de la racine se poursuit :
 - A. en repartant de la racine.
 - B. dans les deux sous-arbres, pour être sûr.
 - C. dans le sous-arbre droit uniquement.
 - D. dans le sous-arbre gauche uniquement.

Capacité C6

6. [Q6] Insérer successivement les clés 1, 2, 3, 4 dans un arbre binaire de recherche vide produit :
 - A. un arbre dégénéré, de hauteur 3, semblable à une liste chaînée.
 - B. un arbre parfaitement équilibré de hauteur 1.

- C. un arbre où 4 devient la racine.
- D. une erreur, les clés devant être insérées dans le désordre.

Capacité C7

- 7. [Q7] Le parcours en largeur d'un graphe, à partir d'un sommet de départ, visite les sommets :
 - A. dans l'ordre croissant de leur numéro.
 - B. par distance croissante au sommet de départ, comptée en nombre d'arêtes.
 - C. en descendant d'abord le plus loin possible le long d'un chemin.
 - D. dans un ordre qui dépend du hasard.

Capacité C8

- 8. [Q8] Le parcours en profondeur d'un graphe s'implémente naturellement :
 - A. après un tri préalable des sommets.
 - B. avec une file, et elle seule.
 - C. par récursion, ou au moyen d'une pile explicite.
 - D. par une recherche dichotomique.

Capacité C9

- 9. [Q9] Lors du parcours d'un graphe non orienté, on détecte un cycle quand on rencontre un sommet déjà visité qui :
 - A. est le sommet de départ.
 - B. possède le plus grand nombre de voisins.
 - C. n'a aucun voisin.
 - D. n'est pas le parent direct dans le parcours.

Capacité C10

- 10. [Q10] Pour reconstituer le chemin trouvé entre deux sommets par un parcours, il faut :
 - A. mémoriser, pour chaque sommet visité, celui depuis lequel on l'a atteint.
 - B. relancer le parcours depuis l'arrivée.
 - C. trier les sommets par distance.
 - D. conserver uniquement la liste des sommets visités, dans l'ordre.

Capacité C11

- 11. [Q11] La méthode « diviser pour régner » consiste à :
 - A. traiter les données une par une, du début à la fin.
 - B. découper le problème en sous-problèmes de même nature, les résoudre, puis combiner leurs solutions.
 - C. essayer toutes les solutions possibles avant de choisir.
 - D. mémoriser les résultats déjà calculés pour ne pas les recalculer.

Capacité C12

- 12. [Q12] La programmation dynamique est particulièrement utile lorsque :
 - A. les sous-problèmes ne se recoupent jamais.
 - B. le problème n'admet aucune formulation récursive.
 - C. les mêmes sous-problèmes réapparaissent plusieurs fois.
 - D. il n'existe qu'un seul sous-problème.

Capacité C13

- 13. [Q13] À la différence de la recherche naïve, l'algorithme de Boyer-Moore compare le motif au texte :
 - A. dans un ordre tiré au hasard.
 - B. en ignorant complètement le texte.
 - C. sur le seul premier caractère.
 - D. de droite à gauche, en s'appuyant sur un prétraitement du motif.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	A
Q3	C3	B
Q4	C4	C
Q5	C5	D
Q6	C6	A
Q7	C7	B
Q8	C8	C
Q9	C9	D
Q10	C10	A
Q11	C11	B
Q12	C12	C
Q13	C13	D

Diagnostic des réponses erronées

► Q1

- **B** — Cette formule est celle de la HAUTEUR : le maximum ne retient qu'une branche, alors que la taille doit additionner tous les nœuds. *Renvoi : C1, taille d'un arbre.*
- **C** — Les nœuds internes appartiennent aussi à l'arbre ; les omettre sous-estime la taille. *Renvoi : C1, taille d'un arbre.*
- **D** — Un produit s'annulerait dès qu'un sous-arbre est vide, et donnerait 0 pour un arbre pourtant non vide. *Renvoi : C1, taille d'un arbre.*

► Q2

- **B** — -1 est la hauteur de l'arbre VIDE. Un arbre à un nœud n'est pas vide. *Renvoi : C2, hauteur d'un arbre.*
- **C** — L'élève compte les niveaux au lieu des arêtes. La hauteur est la longueur du plus long chemin de la racine à une feuille : ici, ce chemin est vide. *Renvoi : C2, hauteur d'un arbre.*
- **D** — La définition récursive couvre ce cas sans ambiguïté, à partir de la hauteur -1 de l'arbre vide. *Renvoi : C2, hauteur d'un arbre.*

► Q3

- **A** — C'est le parcours PRÉFIXE : la racine y est visitée avant ses deux sous-arbres. *Renvoi : C3, parcours infixe, préfixe, suffixe.*
- **C** — C'est le parcours SUFFIXE : la racine y est visitée en dernier. *Renvoi : C3, parcours infixe, préfixe, suffixe.*
- **D** — C'est le parcours en LARGEUR, qui n'appartient pas à la famille des trois parcours en profondeur. *Renvoi : C3, parcours en profondeur et en largeur.*

► Q4

- **A** — La pile produit un parcours en PROFONDEUR : elle ressort d'abord le dernier nœud rencontré, donc le plus profond. *Renvoi : C4, parcours en largeur.*
- **B** — Un dictionnaire peut mémoriser les nœuds vus, mais il n'impose aucun ordre de traitement, qui est ici l'essentiel. *Renvoi : C4, parcours en largeur.*
- **D** — Sans structure d'attente, il est impossible de mémoriser les nœuds d'un niveau pendant qu'on le traite. *Renvoi : C4, parcours en largeur.*

► Q5

- **A** — Recommencer à la racine boucle indéfiniment : la recherche doit descendre d'un niveau à chaque comparaison. *Renvoi : C5, recherche dans un ABR.*
- **B** — Explorer les deux côtés reviendrait à parcourir tout l'arbre : c'est précisément ce que la propriété de l'ABR permet d'éviter. *Renvoi : C5, recherche dans un ABR.*
- **C** — L'élève inverse la comparaison. Le sous-arbre droit ne contient que des clés supérieures à la racine. *Renvoi : C5, recherche dans un ABR.*

► Q6

- B — L'insertion ne rééquilibre rien : chaque clé étant supérieure à la précédente, elle part toujours à droite. *Renvoi : C6, insertion dans un ABR.*
- C — La racine est fixée par la PREMIÈRE clé insérée, ici 1, et ne change plus. *Renvoi : C6, insertion dans un ABR.*
- D — L'insertion en ordre trié est parfaitement licite ; elle est seulement le pire cas pour la hauteur obtenue. *Renvoi : C6, insertion dans un ABR.*

► Q7

- A — La numérotation des sommets n'a aucune influence : seule la structure des arêtes gouverne l'ordre de visite. *Renvoi : C7, parcours en largeur d'un graphe.*
- C — C'est le parcours en PROFONDEUR. Le parcours en largeur épuise un niveau entier avant de passer au suivant. *Renvoi : C7, largeur et profondeur.*
- D — Le parcours est déterministe une fois fixé l'ordre des voisins dans la structure de données. *Renvoi : C7, parcours en largeur d'un graphe.*

► Q8

- A — Aucun tri n'est nécessaire ni même défini : un graphe n'est pas une structure ordonnée. *Renvoi : C8, parcours en profondeur.*
- B — La file donne le parcours en LARGEUR : c'est le choix de la structure d'attente qui distingue les deux parcours. *Renvoi : C8, parcours en profondeur.*
- D — La dichotomie suppose un ordre total sur les éléments, dont un graphe est dépourvu. *Renvoi : C8, parcours en profondeur.*

► Q9

- A — Un cycle peut se refermer sur n'importe quel sommet, pas seulement sur celui d'où l'on est parti. *Renvoi : C9, détection de cycle.*
- B — Le degré d'un sommet ne dit rien sur l'existence d'un cycle : un sommet de degré élevé peut appartenir à un arbre. *Renvoi : C9, détection de cycle.*
- C — Un sommet isolé ne peut appartenir à aucun cycle, faute d'arête pour y entrer. *Renvoi : C9, détection de cycle.*

► Q10

- B — Un second parcours redonnerait l'existence d'un chemin, mais pas celui qui vient d'être emprunté. *Renvoi : C10, reconstitution d'un chemin.*
- C — Connaître les distances ne suffit pas : il manque le lien qui relie chaque sommet à son prédécesseur. *Renvoi : C10, reconstitution d'un chemin.*
- D — L'ordre de visite mêle toutes les branches explorées ; il ne distingue pas celles qui mènent au but de celles qui ont été abandonnées. *Renvoi : C10, reconstitution d'un chemin.*

► Q11

- A — L'élève décrit un parcours séquentiel, sans aucun découpage récursif. *Renvoi : C11, diviser pour régner.*
- C — C'est la recherche exhaustive, que la méthode cherche justement à éviter. *Renvoi : C11, diviser pour régner.*
- D — La mémorisation caractérise la PROGRAMMATION DYNAMIQUE. Elle s'ajoute parfois au découpage, mais ne le définit pas. *Renvoi : C11, diviser pour régner et programmation dynamique.*

► Q12

- A — Sans recouvrement, il n'y a rien à réutiliser : un simple « diviser pour régner » suffit. *Renvoi : C12, programmation dynamique.*
- B — C'est l'inverse : la méthode part d'une relation de récurrence, qu'elle évalue en mémorisant les résultats. *Renvoi : C12, programmation dynamique.*
- D — Avec un unique sous-problème, il n'y a aucun recalcul à éviter et donc aucun gain. *Renvoi : C12, programmation dynamique.*

► Q13

- A — L'ordre est au contraire fixé et essentiel : c'est la comparaison depuis la fin du motif qui autorise les grands décalages. *Renvoi : C13, algorithme de Boyer-Moore.*

- **B** — Le texte est évidemment lu ; ce que l'algorithme économise, ce sont des comparaisons, grâce aux décalages qu'il peut anticiper. *Renvoi : C13, algorithme de Boyer-Moore.*
- **C** — Une comparaison sur un seul caractère ne prouverait rien : c'est le motif entier qui doit être reconnu. *Renvoi : C13, algorithme de Boyer-Moore.*

Corrigé

Q1. L'arbre a 5 nœuds (10, 5, 15, 2, 7) : taille = 5. Le plus long chemin racine-feuille est $10 \rightarrow 5 \rightarrow 2$ (ou $10 \rightarrow 5 \rightarrow 7$), soit 2 arêtes : hauteur = 2.

Q2.

```
1 def est_feuille(arbre):
2     return arbre is not None and arbre.gauche is None and arbre.droite is None
```

Corrigé

Q1. L'ABR obtenu a pour racine 10, sous-arbre gauche enraciné en 5 (avec 3 à gauche et 7 à droite), et sous-arbre droit enraciné en 15 (avec 12 à gauche).

Q2.

```
1 def minimum(arbre):
2     while arbre.gauche is not None:
3         arbre = arbre.gauche
4     return arbre.valeur
```

Dans un ABR, chaque descente à gauche mène vers des valeurs plus petites ; la plus petite valeur est donc atteinte lorsqu'il n'y a plus de sous-arbre gauche.

Évaluation A — Arbres binaires

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (taille, hauteur) ; Ex. 2 : 12 pts (arbre de recherche)

Exercice 1 — Taille et hauteur

(8 points)

On donne la classe Noeud du cours (attributs valeur, gauche, droite) et l'arbre :

```
1 arbre = Noeud(10, Noeud(5, Noeud(2), Noeud(7)), Noeud(15))
```

- (4 pts) Sans exécuter de code, donner la taille et la hauteur de cet arbre en justifiant (convention : hauteur d'une feuille = 0).
- (4 pts) Écrire une fonction `est_feuille(arbre)` qui renvoie True si et seulement si arbre est un nœud sans aucun enfant.

Exercice 2 — Arbre binaire de recherche

(12 points)

- (6 pts) En utilisant `insérer` du cours, construire un ABR à partir des valeurs 10, 5, 15, 3, 7, 12 insérées dans cet ordre.
- (6 pts) Écrire une fonction `minimum(arbre)` qui renvoie la plus petite valeur d'un ABR non vide (indication : la plus petite valeur d'un ABR se trouve toujours tout en bas à gauche).

Corrigé

Q1. Depuis "P" : on enfile "P", on le défile et on enfile ses voisins "Q" puis "R" ; on défile "Q", son voisin "S" n'est pas encore visité, on l'enfile ; on défile "R", son voisin "S" est déjà visité. Ordre de visite : P, Q, R, S.

Q2. Oui : $P \rightarrow Q \rightarrow S \rightarrow R \rightarrow P$ est un cycle (chaque arête utilisée existe bien dans le graphe, et on revient au sommet de départ sans repasser par une même arête).

Corrigé

Q1. Diviser (découper le problème en sous-instances plus petites de même nature), régner (résoudre récursivement chaque sous-instance), combiner (assembler les solutions partielles en la solution globale).

Q2.

```
1 def somme_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     return somme_dpr(liste[:milieu]) + somme_dpr(liste[milieu:])
```

Évaluation B — Graphes et diviser pour régner

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (parcours et cycle) ; Ex. 2 : 10 pts (diviser pour régner)

Exercice 1 — Parcours et cycle

(10 points)

On donne le graphe non orienté :

```
1 graphe = {
2     "P": ["Q", "R"],
3     "Q": ["P", "S"],
4     "R": ["P", "S"],
5     "S": ["Q", "R"],
6 }
```

- (5 pts) Donner l'ordre de visite des sommets par un parcours en largeur (BFS) depuis "P", en supposant que les voisins sont explorés dans l'ordre où ils apparaissent dans la liste.
- (5 pts) Ce graphe contient-il un cycle ? Justifier en citant un cycle si oui.

Exercice 2 — Diviser pour régner

(10 points)

- (4 pts) Rappeler les trois étapes de la méthode diviser pour régner.
- (6 pts) Écrire une fonction récursive `somme_dpr(liste)` qui calcule la somme des éléments d'une liste non vide par diviser pour régner (diviser en deux moitiés, additionner récursivement, puis combiner par addition).

Sujet S1 — Exercice 2 : parcours en largeur d'un réseau de capteurs

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Sept capteurs communiquent entre eux. Deux capteurs reliés peuvent échanger directement ; sinon, le message transite par des capteurs intermédiaires. Le réseau est décrit par un dictionnaire de listes de voisins :

```

1 reseau = {
2     "S1": ["S2", "S3"],
3     "S2": ["S1", "S4"],
4     "S3": ["S1", "S4", "S5"],
5     "S4": ["S2", "S3", "S6"],
6     "S5": ["S3"],
7     "S6": ["S4"],
8     "S7": [],
9 }
```

1. Écrire une fonction `est_non_oriente(graphe)` qui renvoie `True` si, pour toute arête, les deux extrémités se citent mutuellement. Vérifier que `reseau` satisfait cette propriété.
2. Écrire `parcours_largeur(graphe, depart)` qui renvoie la liste des capteurs **dans leur ordre de découverte**, en utilisant une file. Donner le résultat pour un départ en "S1". Les voisins sont examinés dans l'ordre où ils apparaissent dans la liste.
3. Modifier le `parcours` pour qu'il renvoie, au lieu de l'ordre de découverte, un dictionnaire associant à chaque capteur atteint sa **distance** en nombre de liaisons depuis le départ. Donner le dictionnaire obtenu depuis "S1".
4. Le capteur "S7" n'apparaît dans aucune des deux sorties précédentes. Expliquer pourquoi, et dire ce que cela signifie concrètement pour le réseau.
5. On dispose aussi d'un `parcours en profondeur`, qui depuis "S1" découvre les capteurs dans l'ordre [S1, S2, S4, S3, S5, S6]. Comparer avec la question 2. Lequel des deux parcours permet de conclure que "S6" est à trois liaisons de "S1" ? Justifier.
6. On veut que "S6" soit joignable en une seule liaison depuis "S1". Quelle arête faut-il ajouter ? Vérifier votre réponse en donnant la nouvelle distance. Ajouter l'arête entre "S5" et "S6" suffirait-il ? Justifier.

Corrigé

Question 1. (3 points)

```

1 def est_non_oriente(graphe):
2     for sommet, voisins in graphe.items():
3         for voisin in voisins:
4             if sommet not in graphe[voisin]:
5                 return False
6     return True
```

`reseau` satisfait la propriété : chaque arête y figure deux fois.

Question 2. (5 points)

```

1 from collections import deque
2
3 def parcours_largeur(graphe, depart):
4     vus = {depart}
5     ordre = [depart]
6     file = deque([depart])
7     while file:
8         courant = file.popleft()
9         for voisin in graphe[courant]:
10             if voisin not in vus:
```

```

11     vus.add(voisin)
12     ordre.append(voisin)
13     file.append(voisin)
14     return ordre

```

Résultat : [S1, S2, S3, S4, S5, S6].

Barème : 2 points pour la file, 2 pour le marquage à l'**enfilement** et non au défilement, 1 pour le résultat.

Question 3. (4 points)

```

1 def distances(graphe, depart):
2     d = {depart: 0}
3     file = deque([depart])
4     while file:
5         courant = file.popleft()
6         for voisin in graphe[courant]:
7             if voisin not in d:
8                 d[voisin] = d[courant] + 1
9                 file.append(voisin)
10    return d

```

Résultat : {"S1": 0, "S2": 1, "S3": 1, "S4": 2, "S5": 2, "S6": 3}. Le dictionnaire remplace ici le tableau des sommets vus : y figurer, c'est avoir été découvert.

Question 4. (3 points)

"S7" n'a aucun voisin : il forme une **composante connexe** à lui seul. Aucun message venant de "S1" ne peut l'atteindre, et aucun message qu'il émettrait ne sortirait. Le réseau n'est pas connexe.

Question 5. (3 points)

Les deux parcours visitent les mêmes six capteurs, mais dans des ordres différents. Seul le parcours en **largeur** permet de conclure sur la distance : il découvre les sommets par couches, donc le premier chemin trouvé vers un sommet est le plus court. Le parcours en profondeur atteint "S6" après "S4", en quatrième position, ce qui ne dit rien de la distance.

Question 6. (2 points)

Il faut ajouter l'arête "S1"–"S6" : la distance passe alors à 1. Ajouter "S5"–"S6" ne suffit **pas** : "S5" est déjà à distance 2, donc "S6" passerait de 3 à 3 — le chemin par "S5" n'est pas plus court que celui par "S4".

Critique

- ▶ **Marquer au défilement.** C'est l'erreur la plus fréquente : un sommet marqué seulement quand on le sort de la file peut être enfilé plusieurs fois. Sur ce réseau, "S4" serait enfilé deux fois, par "S2" et par "S3", et apparaîtrait deux fois dans l'ordre.
- ▶ **Confondre file et pile.** pop() au lieu de popleft() transforme silencieusement le parcours en profondeur, et les distances de la question 3 deviennent fausses sans qu'aucune erreur ne soit levée.
- ▶ **La question 6 piège l'intuition.** Relier "S5" et "S6" semble « rapprocher » S6 ; le calcul montre que non. Un candidat qui répond sans recalculer la distance perd les deux points.
- ▶ **« S7 est une erreur du sujet ».** Non : son isolement est le contenu de la question 4. Un graphe non connexe est un cas normal, pas une faute de frappe.

Portée pédagogique

L'exercice sépare nettement ce que le parcours en largeur *fait* — visiter tout ce qui est atteignable — de ce qu'il *garantit* — la minimalité des distances. Cette distinction est ce que la question 5 vient chercher, et elle n'est accessible qu'après avoir écrit les deux versions.

La question 1 est un échauffement volontaire : elle rapporte trois points sans récursivité ni file, et donne au candidat une lecture attentive de la structure de données avant de la parcourir. La question 6 est la seule qui demande d'anticiper un résultat plutôt que de le produire.

Sujet S3 — Exercice 1 : détecter un cycle dans un graphe de dépendances

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Une chaîne de production logicielle enchaîne des tâches. Une flèche $a \rightarrow b$ signifie « a doit être terminée avant b ».

```
1 taches = {
2     "compiler": ["tester"],
3     "tester":  ["publier"],
4     "publier": [],
5     "documenter": ["publier"],
6     "relire":  ["documenter"],
7 }
```

1. Écrire `profondeur(graphe, depart)`, parcours en profondeur renvoyant l'ordre de découverte. Donner le résultat depuis "compiler" puis depuis "relire". Aucune des deux listes ne contient les cinq tâches : expliquer pourquoi.
2. On veut savoir si la chaîne contient une dépendance circulaire. Un élève propose : « si le parcours revient sur un sommet déjà vu, il y a un cycle ». Montrer sur taches que ce critère est **faux**, en nommant le sommet concerné et les deux chemins qui y mènent.
3. Écrire `contient_cycle(graphe)` avec un marquage à trois états : **blanc** (jamais visité), **gris** (en cours de visite, donc dans le chemin courant), **noir** (terminé). Expliquer pourquoi seul un sommet **gris** rencontré signale un cycle. Vérifier que la fonction répond `False` sur taches, et `True` après ajout de la dépendance "publier" → "compiler".
4. Écrire `tri_topologique(graphe)` qui renvoie un ordre d'exécution respectant toutes les dépendances. On empilera chaque sommet **après** avoir visité tous ses successeurs, puis on inversera la liste. Donner un ordre valide pour taches, et énoncer la propriété que doit vérifier tout ordre correct.
5. Appliquer votre tri topologique au graphe cyclique de la question 3. Montrer qu'au moins une dépendance est violée, quel que soit l'ordre renvoyé. Qu'en conclure sur ce qu'un ordonnanceur doit vérifier avant de lancer la chaîne ?

Corrigé

Question 1. (3 points)

```
1 def profondeur(graphe, depart, vus=None):
2     if vus is None:
3         vus = []
4         vus.append(depart)
```

```

5     for suivant in graphe[depart]:
6         if suivant not in vus:
7             profondeur(graphe, suivant, vus)
8     return vus

```

Depuis "compiler" : [compiler, tester, publier]. Depuis "relire" : [relire, documenter, publier].

Les deux listes ont ici la même longueur mais pas le même contenu : un parcours ne découvre que ce qui est **atteignable depuis son départ**. Aucun des deux ne voit les cinq tâches.

Question 2. (4 points)

"publier" est atteint depuis "tester" **et** depuis "documenter". Un parcours global le rencontre donc deux fois, sans qu'aucun cycle n'existe : le graphe reste acyclique. « Déjà vu » ne distingue pas un sommet **revisité** d'un sommet **réatteint**.

Question 3. (6 points)

```

1 def contient_cycle(graphe):
2     BLANC, GRIS, NOIR = 0, 1, 2
3     couleur = {sommet: BLANC for sommet in graphe}
4
5     def visiter(sommet):
6         couleur[sommet] = GRIS
7         for suivant in graphe[sommet]:
8             if couleur[suivant] == GRIS:
9                 return True
10            if couleur[suivant] == BLANC and visiter(suivant):
11                return True
12        couleur[sommet] = NOIR
13        return False
14
15    return any(visiter(s) for s in graphe if couleur[s] == BLANC)

```

Un sommet **gris** est en cours de visite : il est donc sur le chemin courant, entre la racine de l'exploration et le sommet actuel. Y revenir ferme une boucle. Un sommet **noir** est terminé, il n'est plus sur ce chemin : le réatteindre ne prouve rien.

Sur tâches : False. Avec "publier" → "compiler" : True.

Question 4. (5 points)

```

1 def tri_topologique(graphe):
2     vus, ordre = set(), []
3
4     def visiter(sommet):
5         vus.add(sommet)
6         for suivant in graphe[sommet]:
7             if suivant not in vus:
8                 visiter(suivant)
9         ordre.append(sommet)
10
11    for sommet in sorted(graphe):
12        if sommet not in vus:
13            visiter(sommet)
14    return ordre[::-1]

```

Un ordre valide : relire, documenter, compiler, tester, publier. La propriété exigée : pour toute flèche $a \rightarrow b$, a apparaît **avant** b .

Question 5. (2 points)

Sur le graphe cyclique, la fonction renvoie bien une liste de cinq tâches, mais au moins une flèche est violée — la vérification de la question 4 échoue. C'est inévitable : un ordre total sur un cycle placerait chaque tâche avant celle qui la précède.

Un ordonnanceur doit donc **détecter le cycle avant** de produire un ordre. Un tri topologique appliqué sans vérification préalable renvoie une réponse d'apparence normale, et fausse.

Critique

- ▶ **Le cœur de l'exercice est la question 2.** Elle démolit un critère intuitif avant de demander le bon. Un candidat qui la traite trop vite écrit ensuite un marquage à deux états et ne voit pas son erreur : sa fonction déclare un cycle là où il n'y en a pas.
- ▶ **Le gris et le noir.** Beaucoup de copies confondent « visité » et « terminé ». La couleur noire n'existe que pour cette distinction.
- ▶ **La question 5 n'attend pas un code.** Elle attend le constat qu'un résultat plausible peut être faux, et la conséquence : vérifier avant de trier, pas après.
- ▶ **L'ordre renvoyé n'est pas unique.** Plusieurs ordres valides existent ; le corrigé en donne un et énonce la propriété, qui est le vrai critère de correction.

Portée pédagogique

L'exercice enseigne une seule idée, sous trois angles : atteindre un sommet n'est pas y revenir. Elle est fausse en question 2, corrigée en question 3, exploitée en question 4, et sa violation observée en question 5.

Il s'adresse à un candidat à l'aise avec les parcours et cherche à évaluer sa rigueur plutôt que sa vitesse. Les trois premières questions valent treize points sur vingt : un candidat qui s'arrête après la détection de cycle obtient une note honorable.

Sujet S4 — Exercice 2 : diviser pour régner sur un tableau trié

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Un capteur enregistre des écarts de température, rangés par ordre croissant. On veut trouver le **point de bascule** : le plus petit indice à partir duquel les valeurs cessent d'être négatives.

Exemple : sur $[-9, -4, -1, 0, 3, 8, 12]$, la bascule est à l'indice 3.

1. Écrire `bascule(tableau)` par dichotomie. La fonction renvoie `len(tableau)` si toutes les valeurs sont négatives, et 0 si aucune ne l'est. Vérifier sur $[-9, -4, -1, 0, 3, 8, 12]$, $[-3, -2, -1]$, $[0, 1, 2]$ et le tableau vide.
2. Compter le nombre de comparaisons effectuées sur le tableau de sept valeurs, puis sur un tableau de mille valeurs allant de -500 à 499 . Donner ensuite le nombre de comparaisons dans le **pire cas** sur mille valeurs, et relier cette borne à une puissance de 2.
3. À chaque tour de boucle, deux propriétés restent vraies : toutes les valeurs strictement avant gauche sont négatives, et toutes celles à partir de droite sont positives ou nulles. Écrire ces deux propriétés sous forme d'assertions dans la boucle. Expliquer ce qu'elles garantissent au moment où la boucle s'arrête.
4. Écrire `fusionner(a, b)`, qui fusionne deux tableaux triés en un seul, puis `tri_fusion(tableau)` par la méthode diviser pour régner. Vérifier sur $[5, 2, 9, 1, 7]$. Un élève écrit `if a[i] < b[j]` au lieu de `<=` : le tri reste-t-il correct ? Que perd-on ?

5. Appliquer bascule à $[3, -1, 5, -2]$. Le résultat est-il le bon ? La fonction signale-t-elle un problème ? Que faut-il vérifier avant d'appeler une recherche dichotomique, et pourquoi cette vérification n'est généralement pas faite dans la fonction elle-même ?

Corrigé

Question 1. (5 points)

```

1 def bascule(tableau):
2     gauche, droite = 0, len(tableau)
3     while gauche < droite:
4         milieu = (gauche + droite) // 2
5         if tableau[milieu] < 0:
6             gauche = milieu + 1
7         else:
8             droite = milieu
9     return gauche

```

Résultats : 3, 3, 0, 0.

Barème : 2 points pour l'encadrement `[gauche, droite]`, 2 pour les deux mises à jour, 1 pour les quatre cas.

Question 2. (4 points)

Sept valeurs : 3 comparaisons. Mille valeurs, bascule au milieu : 9.

Attention à ne pas confondre ce compte avec la borne. L'intervalle est **divisé par deux** à chaque tour, donc le nombre de tours est au plus le plus petit k tel que $2^k \geq n + 1$. Pour $n = 1000$, cela donne $k = 10$, puisque $2^9 = 512 < 1001 \leq 1024 = 2^{10}$. Selon la position de la bascule, on observe 9 ou 10 comparaisons : 9 ici, 10 dans le pire cas.

Question 3. (4 points)

```

1 while gauche < droite:
2     assert all(v < 0 for v in tableau[:gauche])
3     assert all(v >= 0 for v in tableau[droite:])
4     ...

```

Quand la boucle s'arrête, `gauche == droite`. Les deux propriétés disent alors : tout ce qui est avant cet indice est négatif, tout ce qui est à partir de cet indice ne l'est pas. C'est exactement la définition du point de bascule — l'invariant **prouve** le résultat, il ne se contente pas de le suggérer.

Question 4. (5 points)

```

1 def fusionner(a, b):
2     resultat, i, j = [], 0, 0
3     while i < len(a) and j < len(b):
4         if a[i] <= b[j]:
5             resultat.append(a[i]); i += 1
6         else:
7             resultat.append(b[j]); j += 1
8     return resultat + a[i:] + b[j:]
9
10 def tri_fusion(tableau):
11     if len(tableau) <= 1:
12         return list(tableau)
13     milieu = len(tableau) // 2
14     return fusionner(tri_fusion(tableau[:milieu]),
15                     tri_fusion(tableau[milieu:]))

```

[5, 2, 9, 1, 7] donne [1, 2, 5, 7, 9].

Avec $<$ strict, le tri reste **correct** : les valeurs sont toujours triées. On perd la **stabilité** — à valeur égale, l'élément venu de la moitié gauche ne passe plus en premier. Sur des couples (a, 1), (b, 2) et (c, 1), on obtient c, a, b au lieu de a, c, b.

Question 5. (2 points)

Sur [3, -1, 5, -2], la fonction renvoie une valeur, et cette valeur n'a aucun sens : le tableau n'est pas trié, donc le « point de bascule » n'existe pas. Aucun problème n'est signalé.

Il faut vérifier que le tableau est trié. Cette vérification n'est pas faite dans la fonction parce qu'elle coûterait un parcours complet — soit exactement ce que la dichotomie évite. Elle relève de la **précondition** : c'est à l'appelant de la garantir.

Critique

- ▶ **L'encadrement.** Écrire `droite = len(tableau) - 1` change tout : le cas « aucune valeur positive » n'est plus atteignable, et la fonction renvoie un indice au lieu de `len(tableau)`.
- ▶ **droite = milieu - 1.** Erreur classique et silencieuse : elle saute la valeur du milieu, qui est justement candidate. Les quatre cas de la question 1 la démasquent.
- ▶ **Confondre correction et stabilité.** La question 4 attend les deux mots. Un tri instable n'est pas faux ; il est inutilisable dès qu'on trie en deux passes.
- ▶ **La question 5 récompense la retenue.** Ajouter la vérification dans la fonction est tentant et ruinerait son intérêt. L'attendu est de nommer la précondition, pas de la coder.

Portée pédagogique

L'exercice réunit les deux usages de la division par deux : la recherche, qui divise l'espace du problème, et le tri, qui divise les données. Les mêmes trois lignes — couper, traiter, recoller — servent aux deux.

La question 3 est celle qui distingue un candidat qui code d'un candidat qui prouve : l'invariant n'est pas une vérification supplémentaire, c'est la raison pour laquelle l'algorithme est correct. La question 5 ferme l'exercice sur la limite du procédé, et sur qui en porte la responsabilité.

Sujet S5 — Exercice 2 : insertion et recherche dans un arbre binaire de recherche

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

On insère successivement les clés 50, 30, 70, 20, 40, 60, 80 dans un arbre binaire de recherche initialement vide.

```
1 class Noeud:
2     def __init__(self, cle, gauche=None, droite=None):
3         self.cle = cle
4         self.gauche = gauche
5         self.droite = droite
```

1. Dessiner l'arbre obtenu. Écrire `rechercher(arbre, cle)`, récursive, qui renvoie un booléen. Donner sa valeur pour 40 et pour 35.

- Écrire `chemin_recherche(arbre, cle)` qui renvoie la liste des clés examinées. Donner le chemin pour 40 puis pour 35. Les deux chemins sont identiques : expliquer ce qui distingue malgré tout les deux recherches.
- On veut vérifier qu'un arbre est bien un arbre binaire de recherche. Un élève propose de contrôler, pour chaque nœud, que son fils gauche lui est inférieur et son fils droit supérieur. Montrer que ce contrôle est **insuffisant** en construisant un contre-exemple à trois ou quatre nœuds. Écrire ensuite `est_abr` par encadrement, en propageant un minimum et un maximum.
- Écrire `supprimer(arbre, cle)`. Traiter d'abord les deux cas simples : la clé est portée par une feuille, ou par un nœud à un seul fils. Supprimer 20 et donner le parcours infixe obtenu.
- Traiter le cas d'un nœud à **deux** fils : on remplace sa clé par la plus petite clé de son sous-arbre droit, puis on supprime celle-ci. Supprimer 50, donner le parcours infixe et la nouvelle racine. Vérifier que l'arbre reste un arbre binaire de recherche.

Corrigé

Question 1. (3 points)

Arbre parfaitement équilibré : racine 50, fils 30 et 70, puis 20, 40, 60, 80 en feuilles.

```

1 def rechercher(arbre, cle):
2     if arbre is None:
3         return False
4     if cle == arbre.cle:
5         return True
6     if cle < arbre.cle:
7         return rechercher(arbre.gauche, cle)
8     return rechercher(arbre.droite, cle)

```

40 → True, 35 → False.

Question 2. (3 points)

Les deux chemins valent [50, 30, 40]. La différence est le **point d'arrêt** : pour 40, la descente s'arrête sur une égalité ; pour 35, elle continue vers le fils gauche de 40, qui est None. Le chemin parcouru ne dit donc rien du résultat — seule la raison de l'arrêt le dit.

Question 3. (6 points)

Contre-exemple : racine 50, fils gauche 30 dont le fils **droit** vaut 60, fils droit 70. Le contrôle local passe — $30 < 50$, $60 > 30$, $70 > 50$ — alors que 60 se trouve dans le sous-arbre gauche de 50 tout en lui étant supérieur.

```

1 def est_abr(arbre, mini=None, maxi=None):
2     if arbre is None:
3         return True
4     if mini is not None and arbre.cle <= mini:
5         return False
6     if maxi is not None and arbre.cle >= maxi:
7         return False
8     return (est_abr(arbre.gauche, mini, arbre.cle)
9             and est_abr(arbre.droite, arbre.cle, maxi))

```

La propriété d'un arbre binaire de recherche porte sur **tout** le sous-arbre, pas sur les fils immédiats : l'encadrement transporte cette exigence vers le bas.

Question 4. (4 points)

```

1 def supprimer(arbre, cle):
2     if arbre is None:
3         return None
4     if cle < arbre.cle:

```



```

5     arbre.gauche = supprimer(arbre.gauche, cle)
6     elif cle > arbre.cle:
7         arbre.droite = supprimer(arbre.droite, cle)
8     else:
9         if arbre.gauche is None:
10            return arbre.droite
11        if arbre.droite is None:
12            return arbre.gauche
13        suivant = arbre.droite
14        while suivant.gauche is not None:
15            suivant = suivant.gauche
16        arbre.cle = suivant.cle
17        arbre.droite = supprimer(arbre.droite, suivant.cle)
18    return arbre

```

Après suppression de 20 : [30,40,50,60,70,80].

Question 5. (4 points)

50 a deux fils. Sa clé est remplacée par 60, plus petite clé du sous-arbre droit, puis 60 est supprimé de ce sous-arbre. Parcours infixe : [30,40,60,70,80], nouvelle racine **60**, et `est_abr` renvoie `True`.

Le successeur infixe est le seul choix qui préserve l'ordre : toute clé du sous-arbre gauche lui reste inférieure, toute clé restante du sous-arbre droit lui reste supérieure.

Critique

- ▶ **Le contrôle local.** C'est l'erreur classique sur les arbres binaires de recherche, et la question 3 exige un contre-exemple explicite plutôt qu'une objection de principe.
- ▶ **Le chemin identique.** Beaucoup de candidats croient qu'une recherche infructueuse « part ailleurs ». Elle suit exactement le même chemin ; c'est la fin qui diffère.
- ▶ **Remplacer par le fils gauche.** Dans le cas à deux fils, substituer la clé du fils gauche casse l'ordre. Seuls le maximum du sous-arbre gauche et le minimum du sous-arbre droit conviennent.
- ▶ **Oublier de supprimer le successeur.** Après avoir recopié sa clé, il faut le retirer de son sous-arbre, sans quoi la clé apparaît deux fois. Le parcours infixe de la question 5 le révèle immédiatement.

Portée pédagogique

L'exercice traite le même invariant sous trois angles : on l'utilise pour chercher, on découvre qu'un contrôle naïf ne le vérifie pas, on doit le préserver en supprimant. La question 3 est le pivot — elle transforme une propriété apprise en propriété comprise.

Les questions 4 et 5 séparent volontairement les cas simples du cas difficile. Un candidat qui traite la question 4 et bute sur la 5 a montré l'essentiel de la manipulation d'arbre, et conserve seize points sur vingt.

Situation pratique P2 — parcours en profondeur d'un graphe fourni

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier `parcours.py` contient le graphe, la spécification, et le jeu de tests. Ce dernier ne doit pas être modifié.

```

1  graphe = {
2      "A": ["B", "C"],

```

```

3     "B": ["D"],
4     "C": ["D", "E"],
5     "D": [],
6     "E": ["A"],
7     "F": [],
8 }
9
10 def profondeur(graphe, depart):
11     """Parcours en profondeur iteratif, ordre de decouverte.
12
13     Precondition : depart est un sommet du graphe.
14     Postcondition : renvoie la liste des sommets atteignables depuis
15                     depart, chacun une seule fois, depart en tete.
16
17     """
18     ... # a completer
19
20 def atteignables(graphe, depart):
21     ... # a completer

```

Travail demandé

1. Compléter `profondeur` de façon **itérative**, avec une pile. Aucun appel récursif.
2. Rendre la précondition exécutable et vérifier qu'un sommet inconnu est refusé.
3. Compléter `atteignables`, qui renvoie l'**ensemble** des sommets atteignables.
4. Exécuter le jeu de tests fourni. Il doit passer entièrement.
5. Répondre par écrit, en commentaire : le graphe contient une flèche $E \rightarrow A$ qui referme un cycle. Pourquoi votre parcours ne boucle-t-il pas indéfiniment ? Quelle ligne exactement l'en empêche ?

Ce qui est évalué

Critère	Points
Le jeu de tests fourni passe entièrement	8
Le parcours est bien itératif, avec une pile explicite	3
La précondition est vérifiée	3
La réponse écrite désigne la bonne ligne	6

Réponse attendue à la question 5

Le parcours ne boucle pas grâce au test d'appartenance à l'ensemble des sommets déjà vus, avant l'empilement. Quand le parcours atteint E puis considère A , A est déjà dans vus : il n'est ni réempilé ni recompté.

Critique

- **Marquer au dépilement.** Un sommet marqué seulement quand on le sort de la pile peut y être empilé plusieurs fois. Ici, D est atteignable par B et par C : il apparaîtrait deux fois dans l'ordre, et le test d'unicité échoue.
- **Écrire une version récursive.** Elle passerait le jeu de tests, mais la consigne demande explicitement l'itératif — la pile doit être visible dans le code.
- **Oublier le sommet isolé.** F n'atteint personne et n'est atteint par personne. Un candidat qui parcourt tout le graphe au lieu de partir du sommet donné le verra apparaître à tort.

Portée pédagogique

La situation demande de transformer un algorithme connu sous forme récursive en sa forme itérative, ce qui oblige à rendre explicite la pile que la récursion cachait.

La question 5 vaut six points sur vingt : elle ne demande pas de code, mais de désigner *une ligne* et de dire ce qu'elle empêche. C'est la différence entre un parcours qui marche et un parcours qu'on sait correct.

Situation pratique P3 — taille et hauteur d'un arbre binaire

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Un arbre est représenté par None ou par un triplet (valeur, gauche, droite). Le fichier arbres.py contient les spécifications et le jeu de tests, à ne pas modifier.

```

1 def taille(arbre):
2     """Nombre de noeuds. Postcondition : entier >= 0, nul si vide."""
3     ...
4
5 def hauteur(arbre):
6     """Hauteur. Convention : -1 pour l'arbre vide, 0 pour une feuille."""
7     ...
8
9 def feuilles(arbre):
10    """Nombre de noeuds sans aucun fils."""
11    ...

```

Quatre arbres sont fournis : l'arbre vide, une feuille seule, un arbre à trois nœuds, et un « peigne » de quatre nœuds descendant tous à gauche.

Travail demandé

1. Compléter les trois fonctions, récursivement.
2. Exécuter le jeu de tests fourni. Il doit passer entièrement, y compris sur l'arbre vide.
3. Construire, dans le fichier, un arbre **parfait** à sept nœuds : tous les niveaux sont complets. Donner sa taille, sa hauteur et son nombre de feuilles.
4. Vérifier par le code, sur les cinq arbres, que $\text{taille} \leq 2^{\text{hauteur}+1} - 1$. Sur lequel de vos arbres cette inégalité est-elle une égalité ?
5. Répondre par écrit, en commentaire : pour une taille donnée, quelle forme d'arbre rend la hauteur **maximale**, et laquelle la rend **minimale** ? Illustrer avec deux de vos arbres.

Ce qui est évalué

Critère	Points
Le jeu de tests fourni passe entièrement	8
L'arbre parfait est correctement construit	3
L'inégalité est vérifiée par le code, pas affirmée	4
La réponse écrite nomme les deux formes extrêmes	5

Réponse attendue à la question 5

La hauteur est **maximale** pour un peigne, où chaque nœud n'a qu'un seul fils : la hauteur vaut alors $\text{taille} - 1$. Elle est **minimale** pour un arbre parfait, où tous les niveaux sont remplis : la taille vaut alors $2^{\text{hauteur}+1} - 1$, soit le maximum de nœuds pour cette hauteur.

Critique

- ▶ **La convention de hauteur.** Elle est imposée par la spécification, et le test sur l'arbre vide en dépend. Un candidat qui prend 0 pour l'arbre vide échoue partout d'un cran.
- ▶ **Oublier le cas de base dans feuilles.** Un nœud sans fils doit renvoyer 1 ; sans ce cas, la fonction renvoie toujours 0 et passe pourtant le test de l'arbre vide.
- ▶ **Vérifier l'inégalité à la main.** La question 4 demande explicitement une vérification **par le code**, sur les cinq arbres.
- ▶ **Confondre hauteur et nombre de niveaux.** Ils diffèrent de 1, et c'est ce décalage que la convention fixe une fois pour toutes.

Portée pédagogique

Trois fonctions récursives de trois lignes chacune, et une inégalité qui les relie : la situation évalue moins l'écriture que la cohérence entre une convention, des cas de base et une propriété.

La question 5 fait constater que taille et hauteur sont deux mesures indépendantes, dont l'écart caractérise la forme de l'arbre. C'est le préalable à toute discussion sur l'équilibrage.

Situation pratique P7 — un algorithme glouton et son contre-exemple

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier monnaie.py contient les deux spécifications et le jeu de tests.

```

1 def rendu_glouton(montant, pieces):
2     """Rend `montant` en prenant toujours la plus grande piece possible.
3
4     Precondition : montant >= 0, pieces non vide, valeurs > 0.
5     Postcondition : la somme des pieces rendues vaut montant,
6                     ou None si la methode n'aboutit pas.
7
8     """
9     ...
10 def rendu_optimal(montant, pieces):
11     """Nombre minimal de pieces, par programmation dynamique."""
12     ...

```

Travail demandé

1. Compléter `rendu_glouton`. Rendre la précondition exécutable.
2. Compléter `rendu_optimal` par programmation dynamique : on construit un tableau où la case m contient le nombre minimal de pièces pour rendre m .
3. Vérifier, pour tous les montants de 0 à 99 avec le système [1, 2, 5, 10, 20, 50], que le glouton rend le montant exact **et** qu'il utilise le nombre minimal de pièces.
4. Avec le système [1, 3, 4], chercher **par le programme** le plus petit montant pour lequel le glouton n'est pas optimal. Donner ce montant, le rendu glouton, et le nombre minimal de pièces.
5. Avec le système [3, 5], donner ce que renvoient les deux fonctions pour les montants 4, 7 et 8. Répondre par écrit : quand le glouton renvoie None, est-ce toujours parce qu'aucune solution n'existe ?

Ce qui est évalué

Critère	Points
Le glouton est correct et sa précondition vérifiée	4
La programmation dynamique est correcte	5
La vérification sur les cent montants passe	3
Le contre-exemple est trouvé par le programme	4
La réponse écrite de la question 5 est juste	4

Réponses attendues

Question 4. Le plus petit contre-exemple est 6. Le glouton prend $4 + 1 + 1$, soit **trois** pièces ; l'optimum est $3 + 3$, soit **deux**.

Question 5. Avec $[3, 5]$: pour 4 et 7, les deux fonctions renvoient None ; pour 8, le glouton rend $[5, 3]$ et l'optimum vaut 2.

Non, le None du glouton ne prouve **pas** l'absence de solution en général — il signifie seulement que sa stratégie n'aboutit pas. Ici, pour 4 et 7, aucune solution n'existe effectivement, et c'est rendu_optimal qui l'établit. Seule une méthode exhaustive permet de conclure à l'impossibilité.

Critique

- ▶ **Confondre correct et optimal.** Sur $[1, 3, 4]$ et un montant de 6, le glouton rend bien 6 : il est correct. Il n'est simplement pas minimal. La question 4 sépare les deux notions par la mesure.
- ▶ **Chercher le contre-exemple à la main.** La consigne exige de le trouver **par le programme**. Un candidat qui écrit 6 sans balayage n'a rien établi.
- ▶ **Interpréter None comme « impossible ».** C'est l'erreur visée par la question 5, et elle est fréquente parce que sur les systèmes usuels — qui contiennent la pièce 1 — le glouton aboutit toujours.
- ▶ **Oublier `table[0] = 0`.** Sans ce cas de base, la programmation dynamique ne démarre pas et tout vaut l'infini.

Portée pédagogique

La situation met face à face deux stratégies sur le même problème et fait mesurer leur écart au lieu de l'énoncer. Le glouton est écrit en premier parce qu'il est plus simple ; sa limite n'apparaît qu'une fois l'optimum disponible pour comparaison.

La question 5 est la plus exigeante : elle demande de ne pas confondre l'échec d'une méthode avec l'inexistence d'une solution. C'est une distinction qui dépasse largement le rendu de monnaie.

Exercice 6

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire,

par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Version aménagée — Extrait Algorithmique

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Compter les feuilles d'un arbre (C1, C3)

Une **feuille** est un nœud sans aucun fils.

On représente un arbre par (valeur, gauche, droite), et None pour un arbre vide.

```
1 arbre = (1,
2         (2, None, None),
3         (3, (4, None, None), None))
```

Étape 1. Classe chaque nœud. Le premier est donné.

Nœud	Fils gauche	Fils droit	Feuille ?
2	aucun	aucun	oui
4			
3			
1			

Étape 2. Combien de feuilles au total ?

.....

Étape 3. Complète le cas d'arrêt de la fonction récursive.

```
1 def compter_feuilles(arbre):
2     if arbre is ____:
3         return 0
4     valeur, gauche, droite = arbre
5     if gauche is None and droite is None:
6         return ____
7     return compter_feuilles(gauche) + compter_feuilles(droite)
```

Pour le second trou :

Exercice 2 — Un trajet de métro (C7, C10)

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

Étape 1. Complète les voisins de chaque station.

Station	Voisines
A	B
B	
C	

Étape 2. On part de A et on cherche E, en visitant d'abord toutes les stations à une station de distance, puis à deux, etc.

Complète la découverte, niveau par niveau.

Distance depuis A	Stations
0	A
1	
2	
3	

Étape 3. Écris le trajet trouvé de A à E :

A → → → E

Étape 4. Combien de stations ce trajet compte-t-il, départ et arrivée compris ?

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend 7 → 3 → 5 (trouvé) ; celle de 4 descend 7 → 3 → 5 → sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

2 Architectures matérielles, systèmes d'exploitation et réseaux

2.1 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

✦ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.2 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution
```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```
1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}
```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.3 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2   "A": {"B": 1, "C": 5},
3   "B": {"A": 1, "D": 1},
4   "C": {"A": 5, "D": 1},
5   "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.4 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer et déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def déchiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

♦ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

+ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

④ MÉTHODE M1 Dérouler un ordonnancement de processus

Quand l'utiliser ? Quand l'énoncé donne des processus avec leurs durées, une politique d'ordonnancement et parfois un quantum, puis demande le chronogramme, l'ordre d'exécution, ou un temps d'attente. Le code du cours ne se déroule pas dans la tête : il faut un tableau.

Pas à pas :

1. **Relève les données** : chaque processus avec sa durée totale, sa date d'arrivée si l'énoncé en donne une, et le quantum s'il y en a un. Une date d'arrivée non précisée vaut 0.
2. **Trace un tableau** à quatre colonnes : instant, processus élu, durée consommée, temps restant de chacun. Une ligne par tranche, jamais par processus.
3. **Applique la politique pour élire**. Tourniquet : le premier de la file, et le processus qui vient de s'exécuter repart **à la fin** s'il lui reste du travail. Plus court d'abord : celui dont le temps restant est le plus petit parmi les arrivés.
4. **Consomme le minimum entre le quantum et le temps restant**. Un processus à qui il reste 1 ms avec un quantum de 3 ms ne consomme que 1 ms : il libère le processeur en avance.
5. **Remets à jour la file** avant la ligne suivante — les arrivées d'abord, le processus préempté ensuite. Cet ordre change le résultat.
6. **Recommence** jusqu'à ce que tous les temps restants soient nuls, puis réponds à la question posée : ordre d'exécution, instant de fin de chacun, ou temps d'attente.

Exemple : trois processus arrivés en même temps, tourniquet de quantum 2. P_1 dure 5, P_2 dure 3, P_3 dure 1.

File $[P_1, P_2, P_3]$. P_1 prend 2 (reste 3) et repart derrière : $[P_2, P_3, P_1]$. P_2 prend 2 (reste 1) : $[P_3, P_1, P_2]$. P_3 prend 1 et **termine** — il ne consomme pas les deux : $[P_1, P_2]$. P_1 prend 2 (reste 1) : $[P_2, P_1]$. P_2 prend son dernier 1 et termine. P_1 prend son dernier 1 et termine.

Chronogramme : $P_1(2), P_2(2), P_3(1), P_1(2), P_2(1), P_1(1)$. Total 9, ce qui est bien $5 + 3 + 1$: aucun temps n'est créé ni perdu.

P_3 , le plus court, termine à l'instant 5 ; P_1 , le plus long, à l'instant 9.

Pièges :

- **Faire consommer le quantum entier à un processus presque fini**. P_3 ne consomme que 1 ms, pas 2 : le total du chronogramme doit valoir exactement la somme des durées, ni plus ni moins.
- **Oublier de remettre le processus préempté à la fin de la file**. Le tourniquet devient un « premier arrivé premier servi », et le processus le plus court attend la fin de tous les autres.
- **Placer le processus préempté avant les nouvelles arrivées**. Quand l'énoncé donne des dates d'arrivée, l'ordre d'insertion change le chronogramme : les arrivants d'abord.
- **Croire qu'un grand quantum est toujours meilleur**. Avec un quantum supérieur à la plus longue durée, il n'y a plus de rotation du tout, et les processus courts attendent le maximum.

Vérifier : additionne les tranches de ton chronogramme — le total doit être exactement la somme des durées. Puis vérifie que chaque processus a reçu sa durée complète, ni plus ni moins. Ces deux contrôles attrapent presque toutes les erreurs de recopie.

S'entraîner : exercices C3 du chapitre.

④ MÉTHODE M2 Déterminer la route empruntée selon la métrique du protocole

Quand l'utiliser ? Quand l'énoncé donne un réseau de routeurs, éventuellement avec des coûts de liaison, et demande quelle route un paquet emprunte « avec RIP » ou « avec OSPF ». La question n'est pas « quel est le chemin le plus court » : c'est « que minimise ce protocole-là ».

Pas à pas :

1. **Écris la métrique du protocole, en toutes lettres.** RIP minimise le **nombre de sauts**, c'est-à-dire le nombre de liaisons traversées, sans regarder les coûts. OSPF minimise la **somme des coûts** des liaisons traversées, sans regarder le nombre de sauts.
2. **Énumère les routes possibles** du départ à l'arrivée, sans repasser deux fois par le même routeur. Sur les réseaux des exercices, elles sont peu nombreuses : liste-les toutes plutôt que d'en deviner une.
3. **Calcule la métrique de chaque route**, dans une colonne par protocole demandé. Pour RIP, compte les liaisons — pas les routeurs. Pour OSPF, additionne les coûts.
4. **Choisis le minimum**, protocole par protocole. Si deux routes sont à égalité, dis-le : l'énoncé attend alors le critère de départage qu'il aura précisé, ou la mention des deux routes.
5. **Compare les deux réponses.** Quand elles diffèrent, explique pourquoi : c'est exactement ce que la question cherche à faire dire.

Exemple : réseau A, B, C, D . Liaisons et coûts : $A-B$ de coût 1, $B-D$ de coût 1, $A-C$ de coût 5, $C-D$ de coût 1, et une liaison directe $A-D$ de coût 3. Quelle route de A vers D ?

Routes possibles : $A-D$ (1 saut, coût 3), $A-B-D$ (2 sauts, coût 2), $A-C-D$ (2 sauts, coût 6). RIP minimise les sauts : il choisit $A-D$, un seul saut, alors que cette liaison coûte 3.

OSPF minimise le coût : il choisit $A-B-D$, coût 2, bien qu'elle traverse un routeur de plus.

Les deux protocoles donnent des routes **différentes** sur le même réseau, et aucune n'est « fausse » : chacune est optimale pour sa métrique.

Pièges :

- ▶ **Chercher « le chemin le plus court » sans dire au sens de quoi.** La question n'a pas de réponse tant que la métrique n'est pas fixée : RIP et OSPF choisissent ici deux routes différentes sur le même réseau.
- ▶ **Compter les routeurs au lieu des liaisons.** $A-B-D$ traverse 3 routeurs mais fait 2 sauts. Le décalage d'une unité passe souvent inaperçu au classement et fausse la réponse chiffrée.
- ▶ **Croire qu'une route à moins de sauts coûte forcément moins cher.** C'est justement le contre-exemple de l'énoncé : la liaison directe est la plus courte en sauts et la plus chère en coût.
- ▶ **Généraliser la divergence.** Sans la liaison directe, les deux protocoles choisissent la même route. La divergence vient du réseau donné, pas d'une opposition de principe entre les protocoles.

Vérifier : contrôle que tu as bien listé **toutes** les routes sans répétition de routeur, puis que la route retenue a la plus petite valeur dans sa colonne — et seulement dans la sienne. Si les deux protocoles donnent la même route, dis-le : c'est une réponse, pas un échec.

S'entraîner : exercices C5 du chapitre.

Exercice 1

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 2

Trois processus se partagent trois ressources : $P1$ retient $R1$ et attend $R2$; $P2$ retient $R2$ et attend $R3$; $P3$ retient $R3$ et attend $R1$.

1. Représenter cette situation par les deux dictionnaires attente et retenue du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 3

On donne le réseau de coûts suivant :

```

1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }

```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant bfs_sauts du cours, combien de sauts compte-t-elle ?
2. En utilisant plus_court_cout du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 4

En utilisant chiffrer_xor du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

ARCHITECTURES MATÉRIELLES, SYSTÈMES D'EXPLOITATION ET RÉSEAUX — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Par rapport à des composants séparés, le principal inconvénient d'un système sur puce est :
 - A. la perte de modularité : un composant défaillant ne peut pas être remplacé seul.
 - B. une consommation d'énergie plus élevée.
 - C. un encombrement plus important.
 - D. un coût de fabrication systématiquement supérieur.

Capacité C2

2. [Q2] Créer un processus consiste, pour le système d'exploitation, à :
 - A. compiler le code source du programme.
 - B. lui allouer de la mémoire et l'inscrire parmi les processus à exécuter.
 - C. arrêter tous les autres processus.
 - D. lui attribuer une adresse IP.

Capacité C3

3. [Q3] L'ordonnanceur du système d'exploitation a pour rôle de :
 - A. trier les fichiers sur le disque.
 - B. compiler les programmes avant leur lancement.
 - C. décider quel processus prêt obtient le processeur, et pour combien de temps.
 - D. supprimer les processus terminés de la mémoire, et rien d'autre.

Capacité C4

4. [Q4] Dans le graphe d'attente des processus, un interblocage se manifeste par :
 - A. un cycle.
 - B. un arbre.
 - C. un graphe sans aucune arête.
 - D. un sommet isolé.

Capacité C5

5. [Q5] Le protocole de routage RIP choisit la route qui minimise :
 - A. le coût cumulé des liaisons traversées.
 - B. le nombre de sauts.
 - C. la latence mesurée en temps réel.

D. le nombre d'utilisateurs connectés.

Capacité C6

6. [Q6] Le chiffrement asymétrique repose sur :

- A. une unique clé secrète, partagée par les deux correspondants.
- B. l'absence de toute clé.
- C. une paire de clés, l'une publique et l'autre privée.
- D. une clé différente pour chaque caractère du message.

Capacité C7

7. [Q7] Au cours d'une connexion HTTPS, le chiffrement asymétrique sert principalement à :

- A. chiffrer la totalité des données échangées pendant toute la session.
- B. compresser les données avant leur envoi.
- C. afficher le cadenas dans le navigateur.
- D. convenir en sécurité d'une clé symétrique de session.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	A
Q5	C5	B
Q6	C6	C
Q7	C7	D

Diagnostic des réponses erronées

► Q1

- **B** — L'intégration réduit au contraire les distances parcourues par les signaux, donc la consommation. C'est un de ses avantages. *Renvoi : C1, système sur puce.*
- **C** — Regrouper les composants sur une même puce diminue la taille : c'est ce qui rend possibles les appareils mobiles. *Renvoi : C1, système sur puce.*
- **D** — En grande série, l'intégration abaisse le coût unitaire. Le défaut porte sur la réparabilité, non sur le prix. *Renvoi : C1, système sur puce.*

► Q2

- **A** — La compilation précède l'exécution et relève d'un autre outil. Le système lance un programme déjà exécutable. *Renvoi : C2, création d'un processus.*
- **C** — C'est le contraire de la multiprogrammation : de nombreux processus coexistent, et l'ordonnanceur les fait alterner. *Renvoi : C2, création d'un processus.*
- **D** — L'adresse IP identifie une machine sur un réseau, non un processus sur une machine. *Renvoi : C2, création d'un processus.*

► Q3

- **A** — L'élève confond ordonnancement des processus et organisation du système de fichiers, qui sont deux fonctions distinctes. *Renvoi : C3, ordonnancement des processus.*
- **B** — La compilation est antérieure et extérieure au système ; l'ordonnanceur ne manipule que des processus déjà créés. *Renvoi : C3, ordonnancement des processus.*
- **D** — La libération de mémoire suit la fin d'un processus. L'ordonnanceur, lui, décide de l'ATTRIBUTION du processeur aux processus prêts. *Renvoi : C3, ordonnancement des processus.*

► Q4

- **B** — Un arbre est par définition sans cycle : les attentes y remontent vers une racine et finissent donc par se résoudre. *Renvoi : C4, interblocage.*
- **C** — Sans arête, aucun processus n'attend de ressource détenue par un autre. *Renvoi : C4, interblocage.*
- **D** — Un sommet isolé désigne un processus qui n'attend rien : c'est exactement le contraire d'un blocage. *Renvoi : C4, interblocage.*

► Q5

- **A** — Le coût cumulé, qui tient compte du débit des liaisons, est le critère d'OSPF. RIP ignore la qualité des liens. *Renvoi : C5, protocoles de routage.*
- **C** — RIP ne mesure rien en temps réel : sa métrique est fixe et se contente de dénombrer les routeurs traversés. *Renvoi : C5, protocoles de routage.*
- **D** — La charge des machines n'intervient ni dans la métrique de RIP ni dans celle d'OSPF. *Renvoi : C5, protocoles de routage.*

► Q6

- **A** — C'est la définition du chiffrement SYMÉTRIQUE, dont la difficulté est précisément de transmettre cette clé unique. *Renvoi : C6, chiffrement symétrique et asymétrique.*
- **B** — Sans clé, il n'y a plus de secret : n'importe qui pourrait déchiffrer le message. *Renvoi : C6, chiffrement symétrique et asymétrique.*

- **D** — L'élève décrit un chiffrement par flot avec masque jetable, qui reste symétrique et ne correspond pas à la question. *Renvoi : C6, chiffrement symétrique et asymétrique.*

► Q7

- **A** — Le chiffrement asymétrique est bien trop lent pour un flux entier. Il n'intervient qu'à l'établissement de la connexion. *Renvoi : C7, échange de clé et HTTPS.*
- **B** — Chiffrer et compresser sont deux opérations distinctes ; le protocole ne confond pas les deux. *Renvoi : C7, échange de clé et HTTPS.*
- **C** — Le cadenas n'est qu'un indicateur affiché par le navigateur : il rend compte de la sécurité obtenue, il ne la produit pas. *Renvoi : C7, échange de clé et HTTPS.*

Corrigé

Q1. Avantages : compacité, consommation énergétique réduite, coût de fabrication réduit en grande série. Inconvénient : perte de modularité (un composant défaillant oblige à remplacer toute la puce).
Q2. Le système d'exploitation alloue de la mémoire au nouveau processus, y charge le code du programme, et l'inscrit dans la liste des processus à exécuter.

Corrigé

Q1.

```
1 resultat = tourniquet(["P1", "P2", "P3"], 2, {"P1": 2, "P2": 4, "P3": 3})
2 print(resultat) # [('P1', 2), ('P2', 2), ('P3', 2), ('P2', 2), ('P3', 1)]
```

P1 termine en une seule tranche (durée 2), P2 et P3 nécessitent chacun deux passages.

Q2. Non, il n'y a **pas** d'interblocage : P3 retient R3 sans rien attendre, donc la chaîne d'attente P1 → P2 (via R2 puis R1) ne boucle pas – P2 finira par obtenir R1 dès que P1 le libère, sans dépendre de personne d'autre.

Évaluation A — Système sur puce et processus

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (SoC, création de processus) ; Ex. 2 : 12 pts (ordonnancement, interblocage)

Exercice 1 — Système sur puce et processus

(8 points)

- (4 pts) Citer trois avantages de l'intégration de plusieurs composants sur un système sur puce, et son principal inconvénient.
- (4 pts) Décrire, en une phrase, ce que fait le système d'exploitation lors de la création d'un processus.

Exercice 2 — Ordonnancement et interblocage

(12 points)

- (6 pts) En utilisant tourniquet du cours, déterminer l'ordre d'exécution pour trois processus P1, P2, P3 de durées respectives 2, 4, 3, avec une tranche de temps de 2.
- (6 pts) Trois processus P1, P2, P3 retiennent chacun une ressource R1, R2, R3, et P1 attend R2, P2 attend R1 (pas de troisième attente). Y a-t-il un interblocage ? Justifier avec `a_un_interblocage` du cours.

Corrigé

Q1. RIP choisirait la route directe A-D (un seul saut), quel que soit son coût élevé. `bfs_sauts(reseau, "A", "D")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "D")` renvoie 3 : la route la moins coûteuse est A-B-C-D (coût $1 + 1 + 1 = 3$), bien moins chère que la route directe (coût 8). Elle **ne coïncide pas** avec la route RIP : RIP privilégie le nombre de sauts, OSPF le coût cumulé.

Corrigé

Q1. Le chiffrement symétrique utilise une **unique clé secrète** partagée pour chiffrer et déchiffrer ; le chiffrement asymétrique utilise une **paire de clés** (publique/privée) : ce qui est chiffré avec la clé publique ne se déchiffre qu'avec la clé privée correspondante.

Q2. Le chiffrement asymétrique est beaucoup plus lent à calculer que le chiffrement symétrique. L'utiliser pour **tout** l'échange (pages web, formulaires, fichiers) ralentirait considérablement la connexion. HTTPS l'utilise donc seulement pour échanger, en toute sécurité, une clé symétrique de session au début de la connexion ; le reste des données est ensuite chiffré avec cette clé symétrique, bien plus rapide.

Évaluation B — Routage et cryptographie

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (routage) ; Ex. 2 : 10 pts (chiffrement, HTTPS)

Exercice 1 — Routage

(10 points)

```
1 reseau = {
2     "A": {"B": 1, "D": 8},
3     "B": {"A": 1, "C": 1},
4     "C": {"B": 1, "D": 1},
5     "D": {"A": 8, "C": 1},
6 }
```

- (5 pts) En utilisant `bfs_sauts` du cours, quelle route RIP choisirait-il de A vers D, et en combien de sauts ?
- (5 pts) En utilisant `plus_court_cout` du cours, quel est le coût de la route la moins coûteuse de A vers D ? Coïncide-t-elle avec la route RIP ?

Exercice 2 — Chiffrement et HTTPS

(10 points)

- (4 pts) Quelle est la différence essentielle entre chiffrement symétrique et asymétrique ?
- (6 pts) Expliquer pourquoi une connexion HTTPS combine les deux, plutôt que d'utiliser uniquement le chiffrement asymétrique pour tout l'échange.

Sujet S2 — Exercice 3 : ordonnancement de trois processus

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Trois processus arrivent en même temps sur un processeur unique. Ils ont besoin de 5, 2 et 3 tranches de temps :

Processus	Tranches nécessaires
P1	5
P2	2
P3	3

L'ordonnanceur utilise un **tourniquet** : il donne à chaque processus au plus *quantum* tranches consécutives, puis le remet en fin de file s'il n'a pas terminé. La file initiale est P1, P2, P3.

1. Avec un quantum de 2, donner la suite des processus exécutés, tranche par tranche, ainsi que l'instant de fin de chacun. On compte les instants à partir de 1.
2. Refaire la question 1 avec un quantum de 4. Le dernier processus à terminer est-il le même ? Quel processus voit sa fin retardée, et pourquoi ?
3. Le **temps d'attente** d'un processus est la différence entre son instant de fin et le nombre de tranches dont il avait besoin. Calculer le temps d'attente moyen pour les deux quanta. Lequel est préférable ici ? Cette conclusion vaut-elle pour tout jeu de processus ?
4. Les trois processus se partagent maintenant trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1. Écrire une fonction qui, partant d'un processus, suit la chaîne « attend une ressource retenue par ». Que renvoie-t-elle depuis P1 ? Comment reconnaît-on un interblocage ?
5. Le système décide que P3 abandonne sa demande sur R1. Montrer qu'il n'y a plus d'interblocage. Le tourniquet de la question 1 aurait-il pu, à lui seul, éviter la situation de la question 4 ? Justifier.

Corrigé

Question 1. (5 points)

Instant	1	2	3	4	5	6	7	8	9	10
Processus	P1	P1	P2	P2	P3	P3	P1	P1	P3	P1

Fins : P2 à 4, P3 à 9, P1 à 10.

Question 2. (4 points)

Instant	1	2	3	4	5	6	7	8	9	10
Processus	P1	P1	P1	P1	P2	P2	P3	P3	P3	P1

Fins : P2 à 6, P3 à 9, P1 à 10. Le dernier à terminer reste P1 — le total de travail n'a pas changé. C'est P2, le plus court, dont la fin est retardée de 4 à 6 : un quantum plus grand fait attendre les processus courts derrière les longs.

Question 3. (5 points)

Quantum 2 : $(10 - 5) + (4 - 2) + (9 - 3) = 5 + 2 + 6 = 13$, soit $13/3 \approx 4,33$.

Quantum 4 : $(10 - 5) + (6 - 2) + (9 - 3) = 5 + 4 + 6 = 15$, soit 5.

Le quantum 2 est préférable **ici**. La conclusion ne se généralise pas : un quantum plus petit multiplie les changements de contexte, dont le coût est ignoré dans ce modèle. Avec un coût de commutation non nul, un quantum trop petit dégrade tout.

Question 4. (4 points)

```

1 def cycle_attente(attente, retenue_par, depart):
2     vus, courant = [], depart
3     while courant is not None and courant not in vus:
4         vus.append(courant)
5         ressource = attente.get(courant)
6         courant = retenue_par.get(ressource) if ressource else None
7     return vus if courant is not None else None

```

Depuis P1 : [P1, P2, P3], puis on revient sur P1. La chaîne **boucle** : c'est exactement la définition d'un interblocage. Aucun des trois ne pourra jamais avancer, car chacun attend une ressource que le suivant ne libérera pas.

Question 5. (2 points)

Sans la demande de P3 sur R1, la chaîne partant de P1 s'arrête sur P3, qui n'attend plus rien : la fonction renvoie None. P3 termine, libère R3, P2 avance, et ainsi de suite.

Le tourniquet n'aurait **rien** changé : il répartit le *processeur*, pas les *ressources*. Un processus qui attend une ressource ne consomme aucune tranche ; lui en donner davantage ne le débloque pas.

Critique

- ▶ **Compter les instants à partir de 0.** L'énoncé le fixe pour éviter deux corrigés également défendables sur les instants de fin.
- ▶ **Croire qu'un quantum plus grand est toujours pire.** La question 3 demande explicitement si la conclusion se généralise ; le modèle ignore le coût de commutation, et c'est ce coût qui renverse la comparaison.
- ▶ **Confondre famine et interblocage.** Un processus qui n'obtient jamais le processeur est affamé mais pourrait avancer ; un processus interbloqué ne le pourrait pas, même seul sur la machine.
- ▶ **La question 5 est un piège de niveau.** Ordonnancement et ressources sont deux mécanismes distincts, et l'exercice les met côte à côte pour que le candidat ne les confonde pas.

Portée pédagogique

L'exercice sépare deux choses qu'un système d'exploitation fait toutes les deux, et qu'on résume souvent en un seul mot « gestion » : partager le temps, et arbitrer l'accès aux ressources. Les trois premières questions travaillent la première, les deux dernières la seconde, et la question 5 exige de dire pourquoi l'une ne résout pas l'autre.

Le déroulé tranche par tranche des questions 1 et 2 est volontairement mécanique : c'est le tableau, pas le raisonnement, qui fait apparaître le retard de P2.

Sujet S4 — Exercice 3 : routage, coût d'un chemin et confidentialité

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Quatre routeurs sont reliés par des liaisons dont le coût est indiqué :

```

1 reseau = {
2     "A": {"B": 2, "C": 9},
3     "B": {"A": 2, "C": 3, "D": 7},
4     "C": {"A": 9, "B": 3, "D": 1},
5     "D": {"B": 7, "C": 1},
6 }
```

1. Écrire `cout_chemin(chemin)`, qui somme les coûts des liaisons successives. Calculer le coût de $A \rightarrow C$ puis de $A \rightarrow B \rightarrow C$. Qu'en conclure sur la différence entre « le moins de sauts » et « le moins coûteux » ?
2. Écrire `plus_courts(graphe, depart)` qui renvoie, pour chaque routeur, le coût minimal depuis le départ, en explorant toujours le sommet non réglé de plus petit coût connu. Donner les quatre coûts depuis A.

3. Faire renvoyer à votre fonction, en plus des coûts, le prédécesseur de chaque routeur sur le meilleur chemin. Reconstruire le chemin de A à D et vérifier que son coût vaut bien le coût minimal trouvé. Comparer avec le coût de $A \rightarrow B \rightarrow D$.
4. On remplace le coût de la liaison $A-C$ par -5 . Calculer le coût de l'aller-retour $A \rightarrow C \rightarrow A$. Exécuter votre fonction et lire la valeur obtenue pour A lui-même. En quoi cette valeur montre-t-elle que la question posée n'a plus de réponse ? Que faudrait-il vérifier avant d'appliquer l'algorithme ?
5. Les routeurs chiffrent leurs messages par un XOR octet à octet avec une clé répétée. Écrire `xor` (message, cle), et vérifier qu'appliquée deux fois elle redonne le message. Citer ensuite **deux** propriétés que ce chiffrement ne garantit pas, en donnant pour chacune une observation qui le montre.

Corrigé

Question 1. (3 points)

```
1 def cout_chemin(chemin):
2     return sum(reseau[chemin[i]][chemin[i + 1]]
3               for i in range(len(chemin) - 1))
```

$A \rightarrow C$ coûte 9 en un saut ; $A \rightarrow B \rightarrow C$ coûte $2 + 3 = 5$ en deux sauts. Le chemin le plus court en nombre de liaisons n'est **pas** le moins coûteux : compter les sauts revient à supposer tous les coûts égaux.

Question 2. (6 points)

```
1 import heapq
2
3 def plus_courts(graphe, depart):
4     distances, precedent = {depart: 0}, {}
5     tas, regles = [(0, depart)], set()
6     while tas:
7         cout, courant = heapq.heappop(tas)
8         if courant in regles:
9             continue
10        regles.add(courant)
11        for voisin, poids in graphe[courant].items():
12            candidat = cout + poids
13            if voisin not in distances or candidat < distances[voisin]:
14                distances[voisin] = candidat
15                precedent[voisin] = courant
16                heapq.heappush(tas, (candidat, voisin))
17    return distances, precedent
```

Depuis A : $A \rightarrow 0, B \rightarrow 2, C \rightarrow 5, D \rightarrow 6$.

Question 3. (4 points)

```
1 def chemin(precedent, depart, arrivee):
2     route = [arrivee]
3     while route[-1] != depart:
4         route.append(precedent[route[-1]])
5     return route[::-1]
```

$A \rightarrow B \rightarrow C \rightarrow D$, de coût $2 + 3 + 1 = 6$, égal au coût minimal. Le chemin $A \rightarrow B \rightarrow D$, plus court en sauts, coûte $2 + 7 = 9$.

Question 4. (4 points)

La liaison étant bidirectionnelle, l'aller-retour $A \rightarrow C \rightarrow A$ coûte $-5 + (-5) = -10$. On peut le parcourir autant de fois qu'on veut, en diminuant le coût à chaque tour : c'est un **cycle de coût négatif**.

La fonction renvoie $A \rightarrow -10$: le point de départ serait à distance négative de lui-même. Or la distance d'un sommet à lui-même vaut 0 par définition — le chemin vide. Cette valeur ne signale pas une erreur de programmation : elle montre que la **question** n'a plus de réponse. Dès qu'un cycle négatif existe, il n'y a pas de coût minimal, seulement des coûts qui décroissent sans fin.

Il faut donc vérifier, avant d'appliquer l'algorithme, qu'aucun coût n'est négatif. C'est une hypothèse sur les données, pas une propriété du code.

Question 5. (3 points)

```
1 def xor(message, cle):
2     return bytes(octet ^ cle[i % len(cle)]
3                 for i, octet in enumerate(message))
```

$\text{xor}(\text{xor}(m, k), k) == m$: l'opération est son propre inverse.

Deux propriétés non garanties :

- ▶ **L'indistinguabilité.** Deux messages identiques chiffrés avec la même clé donnent le même résultat. Un observateur qui voit deux fois la même suite d'octets sait que le même message a été envoyé, sans le déchiffrer.
- ▶ **L'intégrité.** Modifier un octet du message chiffré produit un déchiffrement différent, de même longueur, que rien ne signale. Le destinataire n'a aucun moyen de savoir que le message a été altéré.

Critique

- ▶ **Confondre distance et coût.** La question 1 est là pour cela, et la question 3 y revient : le chemin en deux sauts bat le chemin en un saut, puis le chemin en trois sauts bat celui en deux.
- ▶ **Régler un sommet deux fois.** Sans l'ensemble *regles*, un sommet extrait plusieurs fois du tas relance des explorations inutiles. Le résultat reste juste, le coût non.
- ▶ **« Dijkstra ne marche pas avec des coûts négatifs ».** La formule est juste et l'explication attendue va plus loin : sur une liaison bidirectionnelle, un coût négatif ne rend pas seulement l'algorithme faux, il rend la question sans objet. Un candidat qui cherche « la bonne réponse » à la question 4 cherche quelque chose qui n'existe pas.
- ▶ **Croire que chiffrer suffit.** La question 5 sépare confidentialité, indistinguabilité et intégrité. Un candidat qui répond « le message est protégé » n'a nommé aucune des trois.

Portée pédagogique

L'exercice suit une même idée dans deux domaines : une garantie n'existe que sous ses hypothèses. Le plus court chemin suppose des coûts positifs ; le chiffrement XOR assure la confidentialité du contenu et rien d'autre. Dans les deux cas, la question demande de nommer l'hypothèse, pas seulement de constater l'échec.

La question 4 va un cran plus loin que « l'algorithme se trompe » : elle fait lire au candidat une valeur — le départ à distance -10 de lui-même — dont l'absurdité prouve que l'énoncé du problème s'est effondré avec l'hypothèse.

Les questions 1 à 3 sont progressives et rapportent treize points : un candidat qui maîtrise le routage sans aborder la cryptographie obtient une note convenable. La question 5 est indépendante des quatre précédentes, ce qui la rend accessible même après un échec sur l'algorithme.

Situation pratique P1 — simuler un ordonnancement par tourniquet

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier `tourniquet.py` contient une fonction incomplète et le jeu de tests qui servira à l'évaluation. Le jeu de tests ne doit pas être modifié.

```

1 from collections import deque
2
3 def tourniquet(besoins, quantum):
4     """Ordonnancement par tourniquet.
5
6     Precondition : besoins est un dict nom -> tranches, toutes > 0 ;
7                   quantum > 0.
8     Postcondition : renvoie (trace, fins). `trace` liste les processus
9                     exécutes tranche par tranche ; `fins[nom]` est
10                    l'instant de fin, les instants commençant à 1.
11
12                    """
13
14    ... # à compléter
15
16 def attente_moyenne(besoins, fins):
17     ... # à compléter

```

Travail demandé

1. Compléter `tourniquet`. Les processus entrent dans la file par ordre alphabétique de leur nom. Un processus qui n'a pas terminé après son quantum repart en fin de file.
2. Rendre les deux préconditions exécutables. Vérifier qu'un quantum nul et qu'un besoin nul sont refusés.
3. Compléter `attente_moyenne`, définie comme la moyenne, sur les processus, de la différence entre l'instant de fin et le besoin.
4. Exécuter le jeu de tests fourni. Il doit passer entièrement.
5. Répondre par écrit, dans un commentaire en tête du fichier : que devient l'ordonnancement quand le quantum dépasse le plus grand des besoins ? Vérifier votre réponse en exécutant votre code.

Ce qui est évalué

Critère	Points
Le jeu de tests fourni passe entièrement	8
Les deux préconditions sont vérifiées, pas seulement écrites	4
<code>attente_moyenne</code> est correcte sur les deux quanta	4
La réponse écrite de la question 5 est juste et vérifiée	4

Réponse attendue à la question 5

Quand le quantum dépasse le plus grand besoin, aucun processus n'est jamais remis en file : chacun s'exécute d'un seul tenant, dans l'ordre d'entrée. Le tourniquet dégénère en « premier arrivé, premier servi ».

Critique

- ▶ **Remettre en file un processus terminé.** La file ne se vide jamais et la boucle ne s'arrête pas. C'est la panne la plus fréquente, et elle se manifeste par un blocage, pas par un résultat faux.
- ▶ **Compter les instants à partir de 0.** Tous les instants de fin sont alors décalés de 1, et le jeu de tests échoue en bloc — ce qui est préférable à un échec partiel.
- ▶ **Écrire les préconditions en commentaire.** La question 2 exige qu'elles soient exécutables : un commentaire ne refuse aucun appel.

Portée pédagogique

La situation tient en une heure parce que la structure est fournie et que le jeu de tests dit exactement ce qui est attendu. Le candidat n'a pas à deviner le format de sortie : il a à faire fonctionner une mécanique et à en décrire un cas limite.

La question 5 est la seule qui demande une phrase. Elle sépare le candidat qui a fait passer les tests de celui qui a compris ce que le code fait.

Exercice 5

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Version aménagée — Extrait Architectures matérielles et systèmes

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — L'ordonnement par tourniquet (C3)

Deux processus se partagent le processeur. Chacun reçoit une tranche de temps, à tour de rôle.

X a besoin de 3 tranches. Y a besoin de 2 tranches.

Étape 1. Complète le tour de rôle. Le premier est donné.

Tranche	Processus	Reste à X	Reste à Y
1	X	2	2
2			
3			
4			
5			

Étape 2. Quel processus se termine le premier ?

☐ X

☐ Y

Étape 3. Combien de tranches au total ?

.....

Exercice 2 — Un interblocage (C4)

Trois processus, trois ressources :

- ▶ P1 retient R1 et attend R2 ;
- ▶ P2 retient R2 et attend R3 ;
- ▶ P3 retient R3 et attend R1.

Étape 1. Complète les deux tableaux. La première ligne de chacun est faite.

Processus	Attend	Ressource	Retenue par
P1	R2	R1	P1
P2	...	R2	
P3	...	R3	

Étape 2. Suis la chaîne d'attente. Complète.

P1 attend R2, retenue par, qui attend R3, retenue par, qui attend R1, retenue par

Étape 3. Cette chaîne revient-elle à son point de départ ?

Étape 4. Y a-t-il interblocage ?

Justifie en une phrase :

Aide : un cycle d'attente signifie qu'aucun ne pourra jamais avancer.

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente P1 → P2 → P3 → P1 signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

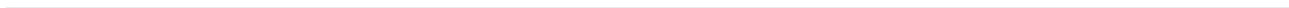
Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).



3 Bases de données relationnelles

3.1 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.2 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.3 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, année INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, <=, >=, <> pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE année > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.4 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe **JOIN ... ON ...** précise la condition de rapprochement.

Exemple. Reprenons Livre et une table Emprunt qui référence Livre.id :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
```

```

2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;

```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table Emprunt.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause ORDER BY trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (ASC, par défaut) ou décroissant (DESC).

Exemple. Livres triés du plus récent au plus ancien :

```

1 SELECT titre, année FROM Livre ORDER BY année DESC;

```

⚠ ERREUR FRÉQUENTE

Oublier que ORDER BY se place après WHERE. L'ordre des clauses est fixe : SELECT ... FROM ... WHERE ... ORDER BY Écrire ORDER BY avant WHERE est une erreur de syntaxe.

3.5 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec VALUES.

Exemple.

```

1 INSERT INTO Livre(titre, auteur, année)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);

```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause UPDATE ... SET ... WHERE ... modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```

1 UPDATE Livre SET année = 1954 WHERE titre = 'Fahrenheit 451';

```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```

1 DELETE FROM Livre WHERE année < 1950;

```


⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

🕒 MÉTHODE M1 Repérer les anomalies d'un schéma relationnel

Quand l'utiliser ? Quand l'énoncé donne un schéma — une ou plusieurs relations avec leurs attributs — et demande de « repérer les anomalies » ou de « critiquer la conception ». Ce n'est pas une question d'opinion : chaque anomalie se montre en exhibant une opération qui tourne mal.

Pas à pas :

1. **Cherche l'information stockée deux fois.** Passe chaque attribut en revue et demande-toi : cette valeur peut-elle apparaître à l'identique sur plusieurs lignes ? Le titre d'un livre répété à chaque emprunt est le cas d'école.
2. **Nomme l'anomalie que la redondance provoque**, en trois temps. *Mise à jour* : corriger une faute dans le titre oblige à modifier toutes les lignes, et en oublier une rend la base incohérente. *Insertion* : on ne peut pas enregistrer un livre tant que personne ne l'a emprunté. *Suppression* : supprimer le dernier emprunt efface aussi le livre.
3. **Vérifie que chaque relation a une clé primaire**, et qu'elle identifie bien **un seul** tuple. Une relation sans clé primaire autorise les doublons parfaits, qu'aucune requête ne peut ensuite distinguer.
4. **Vérifie que chaque référence est une clé étrangère** pointant vers une clé primaire existante. Un attribut qui contient un numéro « par convention », sans contrainte déclarée, laisse entrer des références vers rien.
5. **Traque les attributs à valeurs multiples** : un attribut auteurs qui contiendrait « Orwell, Huxley » n'est pas interrogeable — aucune requête ne peut chercher proprement un auteur là-dedans.
6. **Propose la correction** : sortir l'information répétée dans sa propre relation, lui donner une clé primaire, et la référencer par une clé étrangère.

Exemple : `Emprunt(id, titre_livre, auteur, nom_usager, date)`.

Redondance : les colonnes `titre_livre` et `auteur` se répètent à chaque emprunt du même livre, et la colonne `nom_usager` se répète à chaque emprunt du même lecteur. La même information est donc stockée autant de fois qu'il y a d'emprunts.

Mise à jour : corriger « Orwel » en « Orwell » demande de modifier toutes les lignes de ce livre. En oublier une laisse deux orthographes dans la base, et une requête sur l'auteur en manquera une partie.

Insertion : on ne peut pas enregistrer un livre acquis mais jamais emprunté — il faudrait inventer un emprunt.

Suppression : supprimer le seul emprunt d'un livre efface toute trace de ce livre.

Correction : trois relations liées par deux clés étrangères.

- ▶ `Livre(id, titre, auteur)`
- ▶ `Usager(id, nom)`
- ▶ `Emprunt(id, id_livre, id_usager, date)`

Pièges :

- ▶ **Confondre redondance et répétition légitime.** Une date qui revient parce que deux emprunts ont eu lieu le même jour n'est pas une anomalie : la valeur se répète, mais elle décrit deux faits différents. L'anomalie, c'est de répéter la description d'un **même** objet.
- ▶ **S'arrêter au constat de redondance.** Dire « c'est redondant » ne vaut pas réponse : il faut exhiber l'opération qui tourne mal — mise à jour partielle, insertion impossible, suppression collatérale.
- ▶ **Oublier de vérifier les clés.** Une relation sans clé primaire et un attribut de référence sans contrainte sont deux anomalies distinctes de la redondance.
- ▶ **Proposer une correction qui déplace le problème.** Sortir le titre dans une relation Titre sans clé primaire ne corrige rien : la nouvelle relation hérite du même défaut.

Vérifier : pour chaque anomalie annoncée, écris la requête ou l'opération qui la manifeste — un `SELECT DISTINCT` qui renvoie deux orthographes du même auteur est une preuve, pas une impression. Puis vérifie que ta correction fait passer les trois opérations que l'ancien schéma ratait.

S'entraîner : exercices C3 du chapitre.

📌 MÉTHODE M2 Traduire une question en requête SQL

Quand l'utiliser ? Dès qu'un énoncé pose une question en français — « quels sont les titres des livres empruntés par Ada, du plus récent au plus ancien ? » — et attend une requête. Le cours donne la syntaxe de chaque clause ; ce qui se travaille ici, c'est l'ordre dans lequel on décide.

Pas à pas — décide dans cet ordre, écris dans l'ordre de SQL :

1. **Quelles colonnes veut-on VOIR ?** Souligne-les dans la question. Elles iront après `SELECT`. « Les titres » : une seule colonne, titre.
2. **Dans quelles relations vivent-elles ?** Si toutes les colonnes — celles affichées **et** celles qui servent aux conditions — sont dans une seule relation, tu n'as pas besoin de `JOIN`. Sinon, liste les relations nécessaires.
3. **Comment se raccordent-elles ?** Cherche le couple clé étrangère / clé primaire qui les relie, et écris-le tel quel : `JOIN Livre ON Emprunt.id_livre = Livre.id`. Une jointure sans condition produit **toutes** les combinaisons, ce qui n'est presque jamais ce qu'on veut.
4. **Quelle condition filtre les lignes ?** Traduis chaque contrainte de la question en comparaison, et relie-les par `AND` ou `OR`. Les chaînes de caractères prennent des apostrophes simples.
5. **Y a-t-il un ordre demandé ?** « du plus récent au plus ancien » donne `ORDER BY date DESC`. Sans consigne, ne trie pas.
6. **Écris la requête dans l'ordre de SQL** — `SELECT`, `FROM`, `JOIN`, `WHERE`, `ORDER BY` — puis relis-la à côté de la question, membre par membre.

Exemple : « les titres des livres empruntés par Ada, du plus récent au plus ancien ».

Colonnes vues : titre. *Relations :* titre est dans Livre, mais « empruntés par Ada » exige Emprunt et Usager : trois relations. *Raccords :* `Emprunt.id_livre = Livre.id` et `Emprunt.id_usager = Usager.id`. *Condition :* `Usager.nom = 'Ada'`. *Ordre :* `Emprunt.date DESC`.

```
SELECT Livre.titre
FROM Emprunt
JOIN Livre ON Emprunt.id_livre = Livre.id
JOIN Usager ON Emprunt.id_usager = Usager.id
WHERE Usager.nom = 'Ada'
ORDER BY Emprunt.date DESC;
```

Pièges :

- **Joindre sur la mauvaise égalité de clés.** `ON Emprunt.id = Livre.id` au lieu de `ON Emprunt.id_livre = Livre.id` donne un résultat **non vide** et faux : rien ne signale l'erreur, il faut relire la condition.
- **Oublier la condition de jointure.** `FROM Emprunt, Livre` sans `ON` produit le produit de toutes les lignes — ici 9 au lieu de 3.
- **Traduire « ou » par AND.** « Les livres de 1949 ou de 1953 » s'écrit avec `OR` ; avec `AND`, aucune ligne ne peut satisfaire les deux, et le résultat est vide.
- **Oublier les apostrophes autour d'un texte.** `WHERE nom = Ada` fait chercher à SQL une colonne appelée Ada, et la requête échoue.
- **Trier sans qu'on l'ait demandé, ou l'inverse.** `ORDER BY` se place après `WHERE`, et « du plus récent au plus ancien » impose `DESC`.

Vérifier : relis ta requête à côté de la question, morceau par morceau — chaque membre du `SELECT` correspond à un mot de la question, chaque `JOIN` à une relation traversée, chaque `WHERE` à une contrainte. Puis compte les lignes attendues sur les données de l'énoncé : un résultat dix fois trop grand est presque toujours une jointure sans condition.

S'entraîner : exercices C6 à C9 du chapitre.

Exercice 1 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 2 ♦

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 3 ♦♦

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 4 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

BASES DE DONNÉES RELATIONNELLES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Dans une relation, un attribut est défini sur :
 - A. un domaine, c'est-à-dire l'ensemble des valeurs qu'il peut prendre.
 - B. un tuple.
 - C. une clé étrangère, et rien d'autre.
 - D. une autre relation.

Capacité C2

2. [Q2] Le schéma d'une base de données décrit :
 - A. les valeurs actuellement enregistrées dans les tables.
 - B. la structure : les relations, leurs attributs, leurs domaines et leurs clés.
 - C. le nombre de lignes de chaque table.
 - D. l'historique des requêtes exécutées.

Capacité C3

3. [Q3] Répéter le titre et l'auteur d'un livre sur chaque ligne d'une table d'emprunts, au lieu de les placer dans une table dédiée, entraîne :
 - A. un gain de temps de calcul systématique.
 - B. une amélioration de la sécurité.
 - C. une duplication des données et un risque d'incohérence lors des mises à jour.
 - D. aucune conséquence particulière.

Capacité C4

4. [Q4] Le service de persistance d'un SGBD garantit que :
 - A. les mots de passe sont chiffrés.
 - B. les requêtes s'exécutent instantanément.
 - C. deux utilisateurs n'accèdent jamais à la base en même temps.
 - D. les données restent conservées durablement, même après l'arrêt du programme.

Capacité C5

5. [Q5] Dans la requête `SELECT nom FROM Client WHERE ville = 'Lyon';`, la clause `FROM` indique :
 - A. la relation dans laquelle les données sont puisées.
 - B. les colonnes à afficher.
 - C. la condition que les lignes doivent vérifier.
 - D. l'ordre d'affichage du résultat.

Capacité C6

6. [Q6] La requête `SELECT nom FROM Client WHERE ville = 'Lyon';` affiche :
 - A. toutes les colonnes des clients lyonnais.

- B. le seul nom des clients habitant Lyon.
- C. le nombre de clients lyonnais.
- D. toutes les villes des clients.

Capacité C7

7. [Q7] Écrire `WHERE ville = Lyon` sans apostrophes, au lieu de `WHERE ville = 'Lyon'` :
- A. ne change rien au résultat.
 - B. rend la requête plus rapide.
 - C. est une erreur : Lyon est alors compris comme un nom de colonne.
 - D. est obligatoire pour les textes courts.

Capacité C8

8. [Q8] La clause `JOIN B ON A.x = B.y` permet :
- A. de trier les résultats de la table A.
 - B. de renommer la colonne x en y.
 - C. de supprimer les doublons de A.
 - D. de rapprocher les tuples de A et de B dont les valeurs de x et de y coïncident.

Capacité C9

9. [Q9] Pour afficher les clients du plus jeune au plus âgé, on complète la requête par :
- A. `ORDER BY age ASC`
 - B. `WHERE age`
 - C. `GROUP BY age`
 - D. `ORDER BY age DESC`

Capacité C10

10. [Q10] Pour ajouter un client nommé Dupont, domicilié à Lyon, dans la table `Client(nom, ville)`, on écrit :
- A. `UPDATE Client SET nom = 'Dupont', ville = 'Lyon';`
 - B. `INSERT INTO Client (nom, ville) VALUES ('Dupont', 'Lyon');`
 - C. `SELECT 'Dupont', 'Lyon' FROM Client;`
 - D. `INSERT Client VALUES nom = 'Dupont';`

Capacité C11

11. [Q11] Dans une requête `UPDATE`, omettre la clause `WHERE` a pour effet :
- A. de faire échouer la requête.
 - B. de n'appliquer la modification qu'à la première ligne.
 - C. de modifier toutes les lignes de la table.
 - D. de créer une nouvelle table.

Capacité C12

12. [Q12] La requête `DELETE FROM Emprunt WHERE date_retour IS NOT NULL;` :
- A. supprime la table `Emprunt` tout entière.
 - B. vide la colonne `date_retour` des lignes concernées.
 - C. supprime toutes les lignes de la table, la condition étant ignorée.
 - D. supprime les lignes dont la date de retour est renseignée.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	D
Q5	C5	A
Q6	C6	B
Q7	C7	C
Q8	C8	D
Q9	C9	A
Q10	C10	B
Q11	C11	C
Q12	C12	D

Diagnostic des réponses erronées

► Q1

- **B** — L'élève intervertit les deux notions. Le tuple est une LIGNE ; l'attribut est une colonne, définie par le domaine de ses valeurs. *Renvoi : C1, relation, attribut, domaine.*
- **C** — Une clé étrangère est un rôle particulier joué par certains attributs, non la façon dont tout attribut est défini. *Renvoi : C1, relation, attribut, domaine.*
- **D** — Un attribut n'est pas défini par une relation : c'est la relation qui est définie par ses attributs. *Renvoi : C1, relation, attribut, domaine.*

► Q2

- **A** — L'élève décrit le CONTENU. Celui-ci change à chaque insertion, alors que le schéma demeure. *Renvoi : C2, structure et contenu.*
- **C** — L'effectif est une propriété du contenu, variable dans le temps, et non de la structure. *Renvoi : C2, structure et contenu.*
- **D** — Le journal des requêtes relève de l'exploitation du système ; il ne fait pas partie de la description de la base. *Renvoi : C2, structure et contenu.*

► Q3

- **A** — L'élève ne retient que l'économie d'une jointure. La redondance alourdit la base et rend les mises à jour bien plus coûteuses. *Renvoi : C3, anomalies de schéma.*
- **B** — Multiplier les copies d'une même donnée multiplie les endroits où elle peut être altérée : la sécurité en pâtit. *Renvoi : C3, anomalies de schéma.*
- **D** — Corriger un titre mal orthographié exigerait de modifier toutes les lignes concernées, et en oublier une suffit à rendre la base incohérente. *Renvoi : C3, anomalies de schéma.*

► Q4

- **A** — Le chiffrement relève du service de SÉCURISATION, distinct de la conservation durable des données. *Renvoi : C4, services d'un SGBD.*
- **B** — La rapidité relève du service d'EFFICACITÉ, et aucun système ne promet l'instantanéité. *Renvoi : C4, services d'un SGBD.*
- **C** — L'accès simultané est au contraire permis : c'est le service de gestion de la CONCURRENCE qui l'organise sans conflit. *Renvoi : C4, services d'un SGBD.*

► Q5

- **B** — Les colonnes affichées sont énumérées après SELECT. Chaque clause a un rôle distinct. *Renvoi : C5, composants d'une requête.*
- **C** — La condition est portée par WHERE ; FROM ne filtre rien. *Renvoi : C5, composants d'une requête.*
- **D** — L'ordre d'affichage relève de ORDER BY, absent de cette requête. *Renvoi : C5, composants d'une requête.*

► Q6

- **A** — Afficher toutes les colonnes exigerait `SELECT *`. Ici, une seule colonne est demandée. *Renvoi : C6, SELECT et FROM.*
- **C** — Compter demanderait `COUNT`. La requête énumère les noms, elle ne les dénombre pas. *Renvoi : C6, SELECT et FROM.*
- **D** — La ville figure dans la `CONDITION`, non dans la projection : elle n'apparaît donc pas dans le résultat. *Renvoi : C6, SELECT et FROM.*

► **Q7**

- **A** — La différence est essentielle : les apostrophes distinguent une valeur littérale d'un identifiant de colonne. *Renvoi : C7, littéraux et identifiants dans WHERE.*
- **B** — Aucun gain de vitesse n'est en jeu ; la requête est simplement rejetée ou renvoie un résultat faux. *Renvoi : C7, littéraux et identifiants dans WHERE.*
- **D** — La longueur du texte n'entre pas en ligne de compte : toute chaîne littérale est délimitée par des apostrophes. *Renvoi : C7, littéraux et identifiants dans WHERE.*

► **Q8**

- **A** — Le tri relève de `ORDER BY` ; une jointure ne garantit aucun ordre du résultat. *Renvoi : C8, jointure.*
- **B** — Le renommage s'écrit avec `AS`. La condition `ON` compare deux colonnes, elle n'en rebaptise aucune. *Renvoi : C8, jointure.*
- **C** — L'élimination des doublons s'obtient par `DISTINCT`. Une jointure peut au contraire en produire. *Renvoi : C8, jointure.*

► **Q9**

- **B** — `WHERE` filtre les lignes retenues, sans jamais les réordonner. La clause est de surcroît incomplète. *Renvoi : C9, ORDER BY.*
- **C** — `GROUP BY` regroupe les lignes pour les agréger ; il ne définit pas l'ordre d'affichage. *Renvoi : C9, ORDER BY.*
- **D** — `DESC` trie du plus grand au plus petit, donc du plus âgé au plus jeune : c'est l'ordre inverse de celui qui est demandé. *Renvoi : C9, ORDER BY.*

► **Q10**

- **A** — `UPDATE` modifie des lignes existantes. Sans `WHERE`, cette requête écraserait de surcroît TOUTES les lignes de la table. *Renvoi : C10, INSERT.*
- **C** — `SELECT` interroge la base sans jamais la modifier : aucune ligne ne serait ajoutée. *Renvoi : C10, INSERT.*
- **D** — La syntaxe mêle deux formes : `INSERT` attend `INTO` puis une liste de valeurs, jamais des affectations. *Renvoi : C10, INSERT.*

► **Q11**

- **A** — La requête est parfaitement valide : c'est bien ce qui la rend dangereuse, puisque rien ne signale l'oubli. *Renvoi : C11, UPDATE.*
- **B** — Aucune limitation implicite n'existe. Sans condition, la mise à jour porte sur l'intégralité de la relation. *Renvoi : C11, UPDATE.*
- **D** — `UPDATE` agit sur les données d'une table existante ; il ne touche jamais au schéma. *Renvoi : C11, UPDATE.*

► **Q12**

- **A** — Supprimer la table demanderait `DROP TABLE`. `DELETE` retire des lignes et laisse la structure en place. *Renvoi : C12, DELETE.*
- **B** — `DELETE` supprime des LIGNES entières ; vider une seule colonne relèverait d'un `UPDATE`. *Renvoi : C12, DELETE.*
- **C** — La condition est bien prise en compte : seules les lignes qui la vérifient sont supprimées. *Renvoi : C12, DELETE.*

Corrigé

Q1. Une **relation** est une table regroupant des tuples décrits par les mêmes attributs. Un **attribut** est une colonne de la relation, définie sur un domaine. Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon unique chaque tuple.

Q2. `Commande.id_client` est une clé étrangère vers `Client.id` : elle permet de savoir quel client a passé chaque commande, sans dupliquer les informations du client dans la table `Commande`.

Corrigé

Q1.

```
1 SELECT nom FROM Produit WHERE categorie = 'Papeterie';
```

Q2.

```
1 SELECT nom FROM Produit WHERE prix < 2.00;
```

Q3.

```
1 INSERT INTO Produit(nom, categorie, prix) VALUES ('Gomme', 'Papeterie', 0.60);
```

Évaluation A — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 8 pts (modèle relationnel); Ex. 2 : 12 pts (SELECT, WHERE, INSERT)

Exercice 1 — Modélisation

(8 points)

- (4 pts) Définir, en une phrase chacun, les termes *relation*, *attribut* et *clé primaire*.
- (4 pts) Donner un exemple de clé étrangère dans le schéma `Commande(id, id_client, date) / Client(id, nom)`, et expliquer à quoi elle sert.

Exercice 2 — SELECT, WHERE, INSERT

(12 points)

On donne la table :

```
1 CREATE TABLE Produit(id INTEGER PRIMARY KEY, nom TEXT, prix REAL, categorie TEXT);
2 INSERT INTO Produit VALUES
3   (1, 'Cahier', 1.50, 'Papeterie'),
4   (2, 'Stylo', 0.80, 'Papeterie'),
5   (3, 'Casque audio', 25.00, 'Electronique');
```

- (4 pts) Écrire la requête affichant le nom des produits de catégorie 'Papeterie'.
- (4 pts) Écrire la requête affichant le nom des produits dont le prix est strictement inférieur à 2.00.
- (4 pts) Écrire la requête ajoutant un produit 'Gomme', catégorie 'Papeterie', prix 0.60.

Corrigé

Q1. Persistance, concurrence, efficacité, sécurisation.

Q2. Un fichier CSV n'offre aucune gestion de la **concurrence** : si deux guichets modifient le fichier en même temps, l'une des deux réservations peut être écrasée ou perdue. Un SGBD sérialise les accès concurrents pour garantir qu'aucune réservation n'est perdue.

Corrigé

Q1.


```

1 SELECT Employe.nom, Service.nom
2 FROM Employe
3 JOIN Service ON Employe.id_service = Service.id
4 ORDER BY Employe.nom ASC;

```

Q2.

```

1 UPDATE Employe SET id_service = 1 WHERE nom = 'Ben Ali';

```

Q3.

```

1 DELETE FROM Employe WHERE id_service = 1;

```

Après la question précédente, 'Ben Ali' et 'Chraibi' sont tous deux dans le service 1 : il ne reste que 'Haddad'. Sans clause WHERE, DELETE FROM Employe; supprimerait **tous** les employés, quel que soit leur service.

Évaluation B — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 6 pts (SGBD); Ex. 2 : 14 pts (JOIN, ORDER BY, UPDATE, DELETE)

Exercice 1 — Services d'un SGBD

(6 points)

- (3 pts) Citer les quatre services rendus par un SGBD relationnel.
- (3 pts) Expliquer pourquoi un simple fichier CSV ne peut pas remplacer un SGBD pour gérer les réservations d'une salle de sport ouverte à plusieurs guichets simultanément.

Exercice 2 — JOIN, ORDER BY, UPDATE, DELETE

(14 points)

On donne :

```

1 CREATE TABLE Employe(id INTEGER PRIMARY KEY, nom TEXT, id_service INTEGER);
2 CREATE TABLE Service(id INTEGER PRIMARY KEY, nom TEXT);
3 INSERT INTO Service VALUES (1, 'Comptabilite'), (2, 'Informatique');
4 INSERT INTO Employe VALUES (1, 'Ben Ali', 2), (2, 'Chraibi', 1), (3, 'Haddad', 2);

```

- (5 pts) Écrire la requête affichant le nom de chaque employé avec le nom de son service, triée par nom d'employé croissant.
- (4 pts) Écrire la requête qui change le service de l'employé 'Ben Ali' vers le service 1.
- (5 pts) Écrire la requête qui supprime tous les employés du service 1 (après la modification de la question précédente). Que se passerait-il si l'on omettait la clause WHERE ?

Sujet S1 — Exercice 3 : journal d'événements et requêtes d'agrégat

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Un établissement journalise les événements de son parc informatique.

```

1 CREATE TABLE Machine(
2     id INTEGER PRIMARY KEY,
3     nom TEXT NOT NULL,
4     salle TEXT NOT NULL);
5
6 CREATE TABLE Evenement(
7     id INTEGER PRIMARY KEY,
8     id_machine INTEGER NOT NULL,
9     gravite TEXT NOT NULL,      -- 'info', 'alerte' ou 'critique'
10    jour TEXT NOT NULL,
11    FOREIGN KEY (id_machine) REFERENCES Machine(id));

```

id	nom	salle
1	poste-01	A
2	poste-02	A
3	serveur	B

id	machine	gravité	jour
1	1	info	03-02
2	1	alerte	03-02
3	1	alerte	03-03
4	2	info	03-02
5	3	critique	03-03
6	3	alerte	03-03
7	3	critique	03-04

1. Écrire la requête qui affiche le nom de la machine et le jour, pour tous les événements de gravité 'alerte' survenus en salle A, classés par jour puis par nom. Donner le résultat.
2. Écrire la requête qui affiche, pour chaque machine, son nom et son **nombre total** d'événements. Donner le résultat.
3. Écrire la requête qui affiche les machines ayant au moins deux événements de gravité 'alerte' ou 'critique', avec ce nombre. Donner le résultat. Expliquer pourquoi la condition « au moins deux » ne peut pas s'écrire dans la clause WHERE.
4. La requête de la question 3 ne mentionne jamais poste-02. Est-ce parce qu'il a zéro alerte, ou parce qu'il n'existe pas ? Écrire une requête qui affiche **toutes** les machines avec leur nombre d'alertes, y compris celles qui n'en ont aucune. Donner le résultat.
5. Un script tente d'insérer un événement sur la machine d'identifiant 99, qui n'existe pas. Que se passe-t-il, et pourquoi ? Sous quelle condition cette insertion réussirait-elle malgré tout ? Que deviendrait alors le résultat de la question 2 ?

Corrigé

Question 1. (3 points)

```

1 SELECT M.nom, E.jour
2 FROM Evenement E JOIN Machine M ON M.id = E.id_machine
3 WHERE E.gravite = 'alerte' AND M.salle = 'A'
4 ORDER BY E.jour, M.nom;

```

Résultat : (poste-01, 03-02) et (poste-01, 03-03).

Question 2. (3 points)

```

1 SELECT M.nom, COUNT(*)
2 FROM Evenement E JOIN Machine M ON M.id = E.id_machine
3 GROUP BY M.id ORDER BY M.nom;

```

Résultat : poste-01 → 3, poste-02 → 1, serveur → 3.

Question 3. (5 points)

```

1 SELECT M.nom, COUNT(*) AS n
2 FROM Evenement E JOIN Machine M ON M.id = E.id_machine

```

```

3 WHERE E.gravite IN ('alerte', 'critique')
4 GROUP BY M.id HAVING n >= 2 ORDER BY M.nom;

```

Résultat : poste-01 → 2, serveur → 3.

WHERE filtre les **lignes** avant tout regroupement : à ce moment-là, aucun comptage n'existe encore. HAVING filtre les **groupes** après agrégation. Écrire WHERE COUNT(*) >= 2 provoque une erreur.

Question 4. (6 points)

poste-02 existe : il a zéro alerte. Une jointure interne écarte les machines sans ligne correspondante, ce qui rend « zéro » indiscernable de « absent ». Il faut une jointure externe, et déplacer la condition de gravité **dans** la jointure :

```

1 SELECT M.nom, COUNT(E.id)
2 FROM Machine M
3 LEFT JOIN Evenement E ON M.id = E.id_machine AND E.gravite = 'alerte'
4 GROUP BY M.id ORDER BY M.nom;

```

Résultat : poste-01 → 2, poste-02 → 0, serveur → 1. On compte E.id et non * : sur une ligne sans correspondance, E.id vaut NULL et n'est pas compté, alors que COUNT(*) compterait la ligne et renverrait 1.

Question 5. (3 points)

La clé étrangère id_machine référence Machine(id) : l'insertion est **refusée**, la contrainte d'intégrité référentielle n'étant pas satisfaite. Elle réussirait si la vérification des clés étrangères était désactivée. La ligne orpheline n'apparaîtrait alors dans **aucun** résultat de la question 2 : la jointure interne ne trouve aucune machine d'identifiant 99, et l'événement disparaît des comptages sans le moindre message.

Critique

- **WHERE contre HAVING.** C'est la confusion la plus courante, et la question 3 la rend explicite plutôt que de la sanctionner en silence.
- **COUNT(*) dans une jointure externe.** L'erreur est invisible : elle donne 1 au lieu de 0, un nombre plausible. Seul COUNT d'une colonne de la table jointe donne le bon compte.
- **Déplacer la condition de gravité.** La mettre en WHERE après un LEFT JOIN annule l'effet de la jointure externe : les lignes NULL sont éliminées, et poste-02 disparaît de nouveau. C'est le vrai piège de la question 4.
- **« zéro » et « absent ».** La question 4 est posée en français avant d'être posée en SQL, parce que c'est la lecture du résultat, pas la syntaxe, qui pose problème.

Portée pédagogique

L'exercice construit une progression courte et stricte : filtrer, regrouper, filtrer un regroupement, puis découvrir qu'un regroupement peut effacer ce qu'on cherchait à compter. Les trois premières questions sont accessibles à tout candidat ayant pratiqué les requêtes de base ; la quatrième sépare ceux qui lisent un résultat de ceux qui l'acceptent.

La question 5 relie la base à ce qui la protège : une contrainte d'intégrité n'est pas une formalité de création de table, c'est ce qui empêche un comptage de mentir.

Sujet S3 — Exercice 2 : deux tables, une jointure, une contrainte

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Une revue scientifique gère ses articles et ses auteurs.

```

1 CREATE TABLE Auteur(
2     id INTEGER PRIMARY KEY,
3     nom TEXT NOT NULL UNIQUE);
4
5 CREATE TABLE Article(
6     id INTEGER PRIMARY KEY,
7     titre TEXT NOT NULL,
8     id_auteur INTEGER NOT NULL,
9     annee INTEGER NOT NULL,
10    FOREIGN KEY (id_auteur) REFERENCES Auteur(id));

```

id	nom
1	Bahri
2	Cherif
3	Dridi

id	titre	auteur	année
1	Tri par tas	1	2024
2	Graphes creux	1	2025
3	Index et jointures	2	2025

1. Nommer les trois contraintes déclarées sur la table Auteur et dire, pour chacune, ce qu'elle interdit. Que se passe-t-il si l'on tente d'insérer un second auteur nommé 'Bahri' ?
2. Écrire la requête affichant, pour chaque article, le nom de son auteur et son titre, classés par nom puis par titre. Donner le résultat.
3. La requête précédente ne mentionne pas Dridi. Écrire une requête affichant **tous** les auteurs avec leur nombre d'articles, y compris ceux qui n'en ont aucun. Donner le résultat, et préciser quelle colonne il faut compter et pourquoi.
4. On tente de supprimer l'auteur 'Bahri'. Que se passe-t-il, et pourquoi ? Et si l'on supprime 'Dridi' ? En déduire ce que la clé étrangère protège exactement.
5. Écrire la requête affichant les noms des auteurs ayant publié en 2025, chaque nom une seule fois. On ajoute ensuite un quatrième article de 'Bahri' en 2025. Montrer, en donnant les deux résultats, ce que devient la requête sans le mot-clé qui supprime les doublons.

Corrigé

Question 1. (4 points)

Contrainte	Ce qu'elle interdit
PRIMARY KEY	deux auteurs de même identifiant, et un identifiant vide
NOT NULL	un auteur sans nom
UNIQUE	deux auteurs de même nom

L'insertion d'un second 'Bahri' est **refusée** : la contrainte d'unicité n'est pas satisfaite.

Question 2. (3 points)

```

1 SELECT A.nom, R.titre
2 FROM Article R JOIN Auteur A ON A.id = R.id_auteur
3 ORDER BY A.nom, R.titre;

```

Résultat : (Bahri, Graphes creux), (Bahri, Tri par tas), (Cherif, Index et jointures).

Question 3. (5 points)

```

1 SELECT A.nom, COUNT(R.id)
2 FROM Auteur A LEFT JOIN Article R ON A.id = R.id_auteur
3 GROUP BY A.id ORDER BY A.nom;

```

Résultat : Bahri → 2, Cherif → 1, Dridi → 0.

Il faut compter R.id, colonne de la table **jointe**. Pour Dridi, la jointure externe produit une ligne où toutes les colonnes de Article valent NULL ; COUNT ignore les NULL et renvoie 0. COUNT(*) compterait la ligne elle-même et renverrait 1.

Question 4. (5 points)

Supprimer 'Bahri' est **refusé** : deux articles le référencent, et les supprimer laisserait des lignes pointant vers un auteur inexistant. Supprimer 'Dridi' **réussit** : aucun article ne le référence.

La clé étrangère ne protège pas les auteurs, elle protège les **références** : elle garantit que toute valeur de Article.id_auteur désigne une ligne réellement présente dans Auteur.

Question 5. (3 points)

```
1 SELECT DISTINCT A.nom
2 FROM Auteur A JOIN Article R ON A.id = R.id_auteur
3 WHERE R.annee = 2025 ORDER BY A.nom;
```

Avant l'ajout : Bahri, Cherif dans les deux versions. Après l'ajout du quatrième article, **sans** DISTINCT : Bahri, Bahri, Cherif. La jointure produit une ligne par article, pas par auteur.

Critique

- ▶ **COUNT(*) dans une jointure externe.** L'erreur donne 1 au lieu de 0 : un nombre parfaitement crédible, que rien ne signale. C'est le piège central de la question 3.
- ▶ « **La clé étrangère protège l'auteur** ». Formulation courante et inexacte. Ce qui est protégé, c'est l'article : il ne doit jamais désigner un auteur absent. La question 4 est posée dans les deux sens pour cela.
- ▶ **DISTINCT avant l'exemple.** Beaucoup de candidats l'écrivent par réflexe sans savoir ce qu'il évite. La question 5 attend précisément qu'ils montrent le doublon, pas qu'ils l'anticipent.
- ▶ **Confondre PRIMARY KEY et UNIQUE.** Les deux imposent l'unicité ; seule la première désigne l'identifiant de la ligne, et une table n'en a qu'une.

Portée pédagogique

L'exercice tient sur deux tables et cinq lignes de données, volontairement : il n'y a rien à déchiffrer, tout est dans ce que les requêtes révèlent ou cachent.

Les questions 3 et 5 posent la même leçon à deux endroits différents — une jointure change ce qu'on croit compter — une fois en effaçant une ligne attendue, une fois en en dupliquant une. La question 4 relie les deux au mécanisme qui, seul, garde la base cohérente pendant qu'on la modifie.

Sujet S5 — Exercice 3 : agrégats et regroupements sur un catalogue

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

```
1 CREATE TABLE Rayon(id INTEGER PRIMARY KEY, nom TEXT NOT NULL);
2
3 CREATE TABLE Produit(
4     id INTEGER PRIMARY KEY, nom TEXT NOT NULL,
5     id_rayon INTEGER NOT NULL, prix REAL NOT NULL, stock INTEGER,
6     FOREIGN KEY (id_rayon) REFERENCES Rayon(id));
```

id	nom	rayon	prix	stock
1	Cahier	1	2,50	120
2	Stylo	1	1,20	300
3	Ramette	1	4,90	NULL
4	Clé USB	2	9,90	45
5	Souris	2	14,50	12

Les rayons sont 1 Papeterie, 2 Informatique, 3 Mobilier.

1. Écrire la requête affichant, pour chaque rayon, son nom, son nombre de produits et le prix moyen arrondi au centime. Donner le résultat. Combien de rayons contient la table, et combien apparaissent dans le résultat ?
2. Corriger la requête pour que **tous** les rayons apparaissent, y compris ceux sans produit. Donner le résultat.
3. Le stock de la ramette est inconnu. Donner les résultats de COUNT(*), COUNT(stock), SUM(stock) et AVG(stock) sur la table Produit. Puis comparer WHERE stock = NULL, WHERE stock IS NULL, WHERE stock > 0 et WHERE NOT (stock > 0). Expliquer ce que ces quatre comptes révèlent.
4. Écrire la requête affichant les rayons dont le prix moyen dépasse 5 euros, avec ce prix moyen. Donner le résultat.
5. Écrire la requête affichant, pour chaque rayon, le nom et le prix de son produit **le plus cher**. Donner le résultat. Expliquer pourquoi SELECT R.nom, P.nom, MAX(P.prix) avec un simple GROUP BY n'est pas une réponse fiable à cette question.

Corrigé

Question 1. (3 points)

```
1 SELECT R.nom, COUNT(*), ROUND(AVG(P.prix), 2)
2 FROM Produit P JOIN Rayon R ON R.id = P.id_rayon
3 GROUP BY R.id ORDER BY R.nom;
```

Résultat : Informatique → (2; 12,20), Papeterie → (3; 2,87). La table compte **trois** rayons, le résultat n'en montre que **deux** : Mobilier n'a aucun produit.

Question 2. (3 points)

```
1 SELECT R.nom, COUNT(P.id)
2 FROM Rayon R LEFT JOIN Produit P ON R.id = P.id_rayon
3 GROUP BY R.id ORDER BY R.nom;
```

Informatique → 2, Mobilier → 0, Papeterie → 3.

Question 3. (7 points)

Expression	Valeur	Pourquoi
COUNT(*)	5	compte les lignes
COUNT(stock)	4	ignore les NULL
SUM(stock)	477	120 + 300 + 45 + 12
AVG(stock)	477/4	divise par 4, pas par 5

Filtre	Lignes
stock = NULL	0
stock IS NULL	1
stock > 0	4
NOT (stock > 0)	0

NULL n'est pas une valeur : c'est l'**absence** de valeur. Toute comparaison avec NULL ne vaut ni vrai ni faux, mais *inconnu* — et une clause WHERE ne retient que ce qui est vrai. D'où les deux derniers comptes : la ramette n'est ni dans stock > 0, ni dans sa négation, et $4 + 0 \neq 5$.

Question 4. (3 points)

```
1 SELECT R.nom, ROUND(AVG(P.prix), 2) AS moyen
2 FROM Produit P JOIN Rayon R ON R.id = P.id_rayon
3 GROUP BY R.id HAVING moyen > 5 ORDER BY R.nom;
```

Résultat : Informatique → 12,20.

Question 5. (4 points)

```
1 SELECT R.nom, P.nom, P.prix
2 FROM Produit P JOIN Rayon R ON R.id = P.id_rayon
3 WHERE P.prix = (SELECT MAX(prix) FROM Produit
4                 WHERE id_rayon = P.id_rayon)
5 ORDER BY R.nom;
```

Résultat : Informatique → Souris à 14,50 ; Papeterie → Ramette à 4,90.

Avec SELECT R.nom, P.nom, MAX(P.prix) ... GROUP BY R.id, la valeur de MAX(P.prix) est bien le maximum du groupe, mais P.nom est pris sur une ligne **arbitraire** du groupe : rien ne garantit que ce soit celle du produit le plus cher. Le résultat peut être juste par hasard, et faux sans prévenir.

Critique

- ▶ **= NULL.** Écrite par réflexe, elle ne renvoie jamais rien et ne lève aucune erreur. C'est le piège le plus silencieux de SQL, et la question 3 le met en évidence par le compte, pas par la règle.
- ▶ $4 + 0 \neq 5$. Un candidat qui attend que le filtre et sa négation partitionnent la table n'a pas encore admis qu'un troisième cas existe. Le tableau des quatre comptes est là pour l'y forcer.
- ▶ **AVG et les NULL.** Diviser par 5 ou par 4 change le résultat sans que rien ne le dise. La question demande les deux agrégats côte à côte pour rendre la différence lisible.
- ▶ **Le maximum par groupe.** L'erreur de la question 5 produit souvent la bonne réponse sur de petites tables, ce qui la rend particulièrement tenace.

Portée pédagogique

L'exercice porte sur une seule chose : ce que les agrégats font des lignes qu'ils ne voient pas. Un rayon sans produit disparaît d'un regroupement ; un stock inconnu disparaît d'une moyenne ; un nom de produit accompagne un maximum sans lui appartenir. Trois disparitions, trois mécanismes différents.

La question 3 vaut sept points parce qu'elle demande six mesures et une explication, et parce qu'elle est celle dont la leçon se transporte au-delà de SQL : une valeur absente n'est pas une valeur nulle.

Situation pratique P4 — trois requêtes sur une base fournie

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier stations.py construit la base **en mémoire** et appelle vos trois requêtes. Rien n'est à ouvrir sur le disque.

```

1 CREATE TABLE Commune(id INTEGER PRIMARY KEY, nom TEXT NOT NULL,
2                       departement TEXT NOT NULL, habitants INTEGER NOT NULL);
3
4 CREATE TABLE Station(id INTEGER PRIMARY KEY, nom TEXT NOT NULL,
5                       id_commune INTEGER NOT NULL, altitude INTEGER,
6                       FOREIGN KEY (id_commune) REFERENCES Commune(id));

```

id	commune	département	habitants
1	Ariana	Ariana	114 000
2	Sousse	Sousse	271 000
3	Tozeur	Tozeur	37 000
4	Kef	Kef	45 000

id	station	comm.	altitude
1	Ariana-Nord	1	25
2	Ariana-Sud	1	31
3	Sousse-Port	2	4
4	Tozeur-Oasis	3	NULL

Travail demandé

Écrire trois requêtes, chacune dans une variable du fichier fourni.

1. R1 : les communes de plus de 100 000 habitants, avec leur population, de la plus peuplée à la moins peuplée.
2. R2 : **chaque** commune avec son nombre de stations, y compris les communes qui n'en ont aucune.
3. R3 : l'altitude moyenne des stations, arrondie au centième.
4. Exécuter le script fourni : les trois résultats attendus y figurent, et la comparaison doit réussir.
5. Répondre par écrit, en commentaire : la station de Tozeur a une altitude inconnue. Combien de valeurs la moyenne de R3 prend-elle en compte, et pourquoi ? Que renverrait WHERE altitude = NULL ?

Ce qui est évalué

Critère	Points
R1 exacte, tri compris	4
R2 fait apparaître la commune sans station	6
R3 exacte	3
La réponse écrite explique le traitement du NULL	7

Réponse attendue à la question 5

La moyenne porte sur **trois** valeurs, pas quatre : les agrégats ignorent les NULL. Elle vaut $(25 + 31 + 4)/3 = 20,00$.

WHERE altitude = NULL ne renvoie **aucune** ligne : une comparaison avec NULL n'est ni vraie ni fausse, et WHERE ne retient que ce qui est vrai. Il faut écrire IS NULL.

Critique

- **Jointure interne pour R2.** Kef n'a aucune station et disparaît. Le résultat compte trois communes au lieu de quatre, et rien ne signale la perte.
- **COUNT(*) après une jointure externe.** Kef obtiendrait 1 au lieu de 0. Il faut compter une colonne de la table jointe.
- **Diviser par quatre.** Un candidat qui calcule la moyenne « à la main » sur les quatre stations obtient une valeur différente de celle de AVG, sans comprendre laquelle est juste.

Portée pédagogique

La situation évalue trois requêtes de difficulté croissante, dont une seule — R2 — demande une jointure externe. Le barème le reflète.

La question 5 vaut plus de points que R3 : sur une base réelle, les valeurs manquantes sont la première cause de résultats faux qui n'ont pas l'air faux. La reconnaître vaut mieux que d'écrire une requête de plus.

Exercice 5

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Version aménagée — Extrait Bases de données

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Repérer une redondance (C1, C3)

Un club range tous ses adhérents dans une seule table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_activite, tarif_activite)
```

Deux adhérents inscrits à la même activité y répètent donc son tarif.

Étape 1. Complète le tableau. La première ligne est faite.

Information	Répétée pour chaque inscription ?
<code>nom_adherent</code>	non
<code>nom_activite</code>	
<code>tarif_activite</code>	

Étape 2. Le tarif du judo change. Combien de lignes faut-il modifier ?

une seule

toutes celles où le judo apparaît

Étape 3. On sépare en deux tables. Complète.

```
1 Adherent(id, nom, prenom)
2 Activite(id, nom, _____)
```

Le champ manquant : `tarif` ou `prenom`

Étape 4. Après séparation, combien de lignes faut-il modifier pour changer le tarif du judo ?

.....

Exercice 2 — Deux requêtes SQL (C6, C7)

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT,
2                     realiseur TEXT, duree INTEGER);
3 INSERT INTO Film VALUES
4     (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
5     (2, 'Princesse Mononoke', 'Miyazaki', 134),
6     (3, 'Vice-versa', 'Docter', 95);
```

Étape 1. Une requête a trois parties. Complète le tableau.

Mot-clé	Ce qu'il indique
SELECT	les colonnes voulues
FROM	
WHERE	

Étape 2. Les titres des films de Miyazaki. Complète les trous.

```
1 SELECT _____ FROM Film WHERE realiseur = '_____';
```

Étape 3. Combien de lignes cette requête renvoie-t-elle ?

Étape 4. Les titres des films de plus de 100 minutes.

```
1 SELECT titre FROM Film WHERE duree _____ 100;
```

Le signe manquant :

Titres obtenus :

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realiseur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

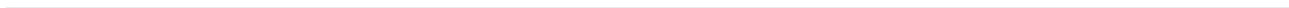
```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0`: seul 'Stylo' est supprimé.



4 Histoire de l'informatique

4.1 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.2 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.

- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Exercice 1 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 2 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de générique.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

HISTOIRE DE L'INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Alan Turing formalise le concept de machine universelle en :
 - A. 1936.
 - B. 1948.
 - C. 1969.
 - D. 1991.

2. [Q2] ARPANET, ancêtre d'Internet, est mis en service en :

- A. 1948.
- B. 1969.
- C. 1989.
- D. 2000.

Capacité C2

3. [Q3] Depuis les débuts de l'informatique, le rôle du matériel a évolué vers :

- A. une spécialisation croissante, chaque machine ne traitant qu'une seule tâche.
- B. une disparition progressive au profit du seul logiciel.
- C. davantage de généricité, la logique particulière étant portée par le logiciel.
- D. aucune évolution notable.

4. [Q4] Internet et le World Wide Web sont dans le rapport suivant :

- A. ce sont deux noms de la même chose.
- B. ce sont deux réseaux physiques distincts et indépendants.
- C. Internet est un service qui fonctionne au-dessus du Web.
- D. le Web est un service qui fonctionne au-dessus du réseau Internet.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C1	B
Q3	C2	C
Q4	C2	D

Diagnostic des réponses erronées

► Q1

- **B** — 1948 est l'époque des premières machines à programme enregistré. Le modèle théorique les précède de plus de dix ans. *Renvoi : C1, repères chronologiques.*
- **C** — 1969 est la mise en service d'ARPANET, un jalon des réseaux et non du modèle de calcul. *Renvoi : C1, repères chronologiques.*
- **D** — 1991 est la naissance de Python et l'ouverture du Web au public : bien après les fondements théoriques. *Renvoi : C1, repères chronologiques.*

► Q2

- **A** — 1948 précède l'existence même des ordinateurs interconnectés ; les premières machines à programme enregistré datent de cette période. *Renvoi : C1, repères chronologiques.*
- **C** — 1989 est l'année où Tim Berners-Lee propose le Web, soit vingt ans APRÈS le réseau qui le porte. *Renvoi : C1, Internet et le Web.*
- **D** — L'élève date le réseau de sa diffusion grand public, largement postérieure à sa création. *Renvoi : C1, repères chronologiques.*

► Q3

- **A** — C'est l'inverse du mouvement historique : les premières machines étaient câblées pour une tâche, et le programme enregistré les a rendues polyvalentes. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*
- **B** — Tout logiciel s'exécute sur du matériel. Sa part relative dans la valeur a crû, mais le support n'a pas disparu. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*
- **D** — Le déplacement de la logique du câblage vers le logiciel est l'une des évolutions majeures de la discipline. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*

► Q4

- **A** — Le courriel et bien d'autres services empruntent Internet sans passer par le Web : les deux ne se confondent donc pas. *Renvoi : C2, Internet et le Web.*
- **B** — Le Web n'est pas un réseau physique : c'est un ensemble de pages liées, échangées grâce au protocole HTTP. *Renvoi : C2, Internet et le Web.*
- **C** — L'élève intervertit les deux niveaux. L'infrastructure est Internet ; le Web est l'un des services qu'elle transporte. *Renvoi : C2, Internet et le Web.*

Corrigé

Q1. Machine universelle de Turing : 1936. Premiers ordinateurs à programme enregistré : 1948. ARPANET : 1969.

Q2. En 1936, les notions de machine, algorithme, langage et information sont pour la première fois pensées comme un tout cohérent (concept de machine universelle, capable d'exécuter tout algorithme calculable).

Q3. Exemple accepté : diversification en téléphones, tablettes, montres connectées, objets connectés, fermes de calcul (toute réponse cohérente avec le cours est valable).

Corrigé

Q1. Aux débuts de l'informatique, le matériel portait directement la logique de calcul (câblages spécifiques). Aujourd'hui, un même matériel générique (smartphone, ordinateur) exécute des logiques très variées portées par le logiciel : le rôle du matériel a évolué vers plus de générique.

Q2. Le système d'exploitation gère le partage des ressources matérielles (mémoire, processeur, périphériques) entre plusieurs programmes, ajoutant une couche logicielle entre l'utilisateur/les applications et le matériel, ce qui permet à ce dernier de rester générique.

Évaluation A — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques

(10 points)

- (4 pts) Associer chacun des événements suivants à son année : machine universelle de Turing ; premiers ordinateurs à programme enregistré ; mise en service d'ARPANET.
- (3 pts) Expliquer en une phrase pourquoi 1936 est considérée comme une date charnière dans l'histoire de l'informatique.
- (3 pts) Citer un exemple de diversification des formes d'ordinateurs depuis les années 1980.

Exercice 2 — Évolution logiciel/matériel

(10 points)

- (5 pts) Expliquer, avec un exemple, comment le rôle du matériel a évolué depuis les débuts de l'informatique.
- (5 pts) Expliquer le rôle d'un système d'exploitation dans cette évolution.

Corrigé

Q1. 1936, par Alan Turing.

Q2. 1948.

Q3. Exemple accepté : invention du World Wide Web par Tim Berners-Lee (1989-1991), qui popularise l'usage grand public d'Internet.

Corrigé

Q1. Dans les années 1950, le matériel portait directement la logique de calcul (cablages, code machine) : peu de flexibilité, chaque machine était quasi dédiée à une tâche. Aujourd'hui, le matériel est générique (processeurs polyvalents) et c'est le logiciel installé qui détermine entièrement le comportement de la machine, permettant une flexibilité quasi illimitée.

Q2. Exemple accepté : une console de jeux ou un ordinateur personnel peut exécuter des milliers de jeux ou d'applications très différents sans modification matérielle, uniquement par installation de logiciels différents.

Évaluation B — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques

(10 points)

- (4 pts) Quelle année marque la formalisation du concept de machine universelle, et par qui ?
- (3 pts) En quelle année les premiers ordinateurs à programme enregistré sont-ils construits ?

3. (3 pts) Citer un événement marquant du développement d'Internet après 1969 (par exemple le World Wide Web).

Exercice 2 — Évolution logiciel/matériel

(10 points)

- (5 pts) Expliquer la différence entre l'informatique des années 1950 (matériel dominant) et l'informatique actuelle (logiciel dominant).
- (5 pts) Donner un exemple concret (autre que celui du cours) illustrant cette évolution.

Sujet S6 — Exercice 3 : situer une évolution technique et ses conséquences

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

On considère quatre repères de l'histoire de l'informatique :

Année	Repère	Ce qu'il introduit
1936	machine universelle	un modèle théorique du calcul
1945	programme enregistré	les instructions rangées en mémoire
1971	microprocesseur	le traitement sur une seule puce
1989	Web	des documents reliés sur un réseau

- Classer ces quatre repères du plus ancien au plus récent, et calculer les trois écarts entre repères successifs. Vérifier que la somme de ces écarts vaut l'écart entre le premier et le dernier.
- Le principe du **programme enregistré** consiste à ranger les instructions dans la même mémoire que les données. On donne ci-dessous une machine minimale et son programme :

```
1 | memoire = {0: "CHARGER 5", 1: "AJOUTER 3", 2: "AFFICHER"}
```

Écrire `executer(memoire)` qui parcourt les adresses dans l'ordre et renvoie la liste des valeurs affichées. Donner le résultat.

- Écrire un second programme qui affiche 17, **sans modifier d'une ligne** la fonction `executer`. Que faut-il changer, et que ne faut-il pas changer ? Relier votre réponse au repère de 1945.
- Écrire un troisième programme qui affiche deux valeurs au lieu d'une. En quoi cela illustre-t-il l'idée de **machine universelle** de 1936 ? Une phrase suffit.
- Le microprocesseur (1971) est postérieur de 26 ans au programme enregistré. Expliquer en quelques lignes pourquoi l'ordre inverse aurait été impossible : que fallait-il avoir compris avant de pouvoir graver une unité de traitement complète sur une puce ?

Corrigé

Question 1. (4 points)

Ordre : 1936, 1945, 1971, 1989. Écarts : **9, 26, 18** ans. Leur somme vaut 53, soit 1989 – 1936 : les écarts successifs se télescopent.

Question 2. (5 points)

```
1 | def executer(memoire):
2 |     accumulateur, sortie = 0, []
3 |     for adresse in sorted(memoire):
```

```

4     instruction = memoire[adresse]
5     if instruction.startswith("CHARGER"):
6         accumulateur = int(instruction.split()[1])
7     elif instruction.startswith("AJOUTER"):
8         accumulateur += int(instruction.split()[1])
9     elif instruction == "AFFICHER":
10        sortie.append(accumulateur)
11    return sortie

```

Résultat : [8].

Question 3. (4 points)

```
1 autre = {0: "CHARGER 10", 1: "AJOUTER 7", 2: "AFFICHER"}
```

Il faut changer le **contenu de la mémoire** ; il ne faut rien changer à la fonction `executer`, qui joue ici le rôle du matériel. C'est exactement le principe de 1945 : le programme est une **donnée** rangée en mémoire, et non un câblage. Changer de tâche ne demande plus de changer de machine.

Question 4. (3 points)

```
1 troisieme = {0: "CHARGER 0", 1: "AFFICHER", 2: "AJOUTER 1", 3: "AFFICHER"}
```

Résultat : [0,1]. Une même machine, décrite une fois pour toutes, exécute des programmes de longueurs et de comportements différents : c'est l'idée de machine universelle — une machine capable de simuler n'importe quelle autre, à condition qu'on lui fournisse sa description.

Question 5. (4 points)

Un microprocesseur est la **réalisation matérielle** d'une unité de traitement dont le comportement ne dépend pas de la tâche : il lit des instructions et les exécute. Cette unité n'a de sens que si les instructions existent **en dehors** d'elle, dans une mémoire — c'est-à-dire si le principe du programme enregistré est acquis.

Avant 1945, une machine était construite pour une tâche : il n'y avait rien à graver de général. On peut miniaturiser une unité de traitement universelle ; on ne peut pas miniaturiser un câblage spécifique et en attendre l'universalité.

Critique

- ▶ **Retenir des dates sans ce qu'elles changent.** L'exercice ne demande jamais une date isolée : chaque question relie un repère à une conséquence observable dans le code.
- ▶ **Modifier `executer` à la question 3.** C'est l'erreur qui fait manquer tout le sens : si l'on touche à la machine, on n'a pas illustré le programme enregistré, on l'a contourné.
- ▶ **« L'ordre aurait pu être inversé ».** La question 5 attend un argument de dépendance conceptuelle, pas une opinion sur le progrès technique. Le microprocesseur suppose l'universalité ; l'inverse est faux.
- ▶ **Confondre 1936 et 1945.** Le premier est un modèle théorique, le second un principe d'architecture. La question 4 sépare les deux en les faisant illustrer par deux programmes différents.

Portée pédagogique

L'exercice traite l'histoire de l'informatique comme une **suite de dépendances**, pas comme une liste de dates : chaque repère rend le suivant possible, et la question 5 demande de justifier l'ordre plutôt que de le réciter.

Le petit interpréteur des questions 2 à 4 rend palpable ce que le cours énonce : le programme est une donnée. Un candidat qui ne saurait rien des dates mais qui écrit les trois programmes sans toucher à la machine a compris l'essentiel, et obtient douze points sur vingt.

Version aménagée — Extrait Histoire de l'informatique

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Remettre dans l'ordre (C1)

Étape 1. Relie chaque événement à son année. Écris la lettre.

Événement	Lettre
Machine de Turing (article fondateur)	B
Premier microprocesseur commercial	...
Naissance du Web	...

A. 1971 B. 1936 C. 1989

Étape 2. Écris les trois années dans l'ordre croissant.

..... < <

Étape 3. Lequel est le plus ancien ?

la machine de Turing

le microprocesseur

le Web

Exercice 2 — Matériel et logiciel (C2)

Un smartphone exécute des milliers d'applications sans que son matériel change.

Étape 1. Classe chaque élément.

Élément	Matériel ou logiciel ?
Le processeur	matériel
Une application de messagerie	
L'écran tactile	
Le système d'exploitation	

Étape 2. Pourquoi une même machine peut-elle exécuter des programmes différents ? Entoure.

le programme est une donnée, rangée en mémoire

le matériel se reconfigure

Étape 3. Complète la phrase :

C'est le principe de la machine . . . , énoncé dès 1936 : une seule machine peut simuler n'importe quelle autre, si on lui fournit sa description.

Mot proposé : universelle

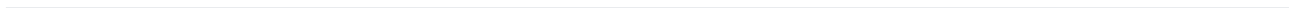
Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistre.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui détermine le comportement du smartphone, illustrant un matériel devenu generique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du système d'exploitation.



5 Langages, récursivité et paradigmes de programmation

5.1 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait le **contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.2 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```
1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n <= 1:
7         return n
```



```
8 return fib_naif(n - 1) + fib_naif(n - 2)
```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.3 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```
1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (`.py`) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
```

```

2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)

```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d’être mise à jour, tant que le comportement observable reste le même).

5.4 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d’une liste, dans trois styles différents.

Style impératif (une suite d’instructions qui modifient un état) :

```

1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total

```

Style fonctionnel (composition de fonctions, sans modifier d’état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.5 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```
1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres
```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```
1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)
```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 <= indice < len(liste)
```

```
3 return 0 <= indice < len(liste)
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 def somme_avec_bug(valeurs):
2     sum = 0           # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum        # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

🔗 MÉTHODE M1 Écrire une fonction récursive et dérouler son arbre d'appels

Quand l'utiliser ? Dans les deux sens : quand on te demande d'écrire une fonction récursive sur un problème neuf, et quand on te donne une fonction récursive et qu'on demande ce qu'elle renvoie, combien d'appels elle fait, ou si elle termine. Écrire et dérouler sont la même démarche prise par les deux bouts.

Pas à pas — écrire :

1. **Écris le cas de base en premier**, avant toute autre ligne. Quelle est la plus petite entrée pour laquelle tu connais la réponse sans réfléchir ? La liste vide, la chaîne vide, $n = 0$.
2. **Écris comment le cas général se RAMÈNE à plus petit**. Une seule règle compte : l'argument de l'appel récursif doit être **strictement** plus proche du cas de base. Note cette quantité qui décroît — c'est ta preuve de terminaison.
3. **Suppose l'appel récursif correct** et écris la ligne qui combine son résultat avec le travail du niveau courant. C'est la ligne la plus courte et la plus difficile.
4. **Relis la terminaison** : est-ce que tout chemin d'exécution atteint le cas de base ? Une branche qui rappelle la fonction avec le même argument suffit à tout casser.

Pas à pas — dérouler :

1. **Écris l'appel initial**, puis, sous lui et décalés vers la droite, les appels qu'il déclenche, dans l'ordre du code.
2. **Descends jusqu'aux cas de base** : ce sont les feuilles de l'arbre, les seules qui renvoient sans rappeler.
3. **Remonte en écrivant les valeurs de retour**, feuille par feuille, en appliquant la ligne de combinaison.
4. **Compte**. Le nombre d'appels est le nombre de nœuds de l'arbre. Repère si un même argument apparaît dans deux branches : c'est le signe qu'une mémorisation ferait gagner (voir la fiche du chapitre Algorithmique).

Exemple : `somme_chiffres(n)`, somme des chiffres d'un entier positif.

Cas de base : si $n < 10$, la somme est n lui-même.

Réduction : n vaut $n // 10$ avec un chiffre en moins ; la quantité qui décroît strictement est le nombre de chiffres.

Combinaison : $n \% 10 + \text{somme_chiffres}(n // 10)$.

Déroulé de `somme_chiffres(507)` : $7 + s(50)$, puis $0 + s(5)$, puis 5 — cas de base. On remonte : 5 , puis $0 + 5 = 5$, puis $7 + 5 = 12$. Trois appels, un seul par niveau : l'arbre est une chaîne, aucune mémorisation à espérer.

Pièges :

- ▶ **Écrire le cas général avant le cas de base.** C'est l'ordre qui fait oublier le cas de base, et sans lui rien ne s'arrête.
- ▶ **Un appel récursif qui ne réduit rien.** $f(n)$ qui rappelle $f(n)$ ne termine jamais, même si le cas de base existe. Nomme toujours la quantité qui décroît.
- ▶ **Confondre « beaucoup d'appels » et « ne termine pas ».** $\text{fib}(30)$ fait plus de deux millions d'appels et termine parfaitement. Le problème est le coût, pas la terminaison, et la réponse n'est pas la même.
- ▶ **Dérouler dans le désordre.** L'arbre d'appels se lit dans l'ordre du code : l'appel écrit à gauche se déclenche en premier, et sa valeur revient en premier.

Vérifier : teste ta fonction sur le cas de base, sur le cas juste au-dessus, et sur un cas moyen comparé à une méthode directe. Pour un déroulé, compte les feuilles : elles doivent toutes être des cas de base, et aucun nœud interne ne doit avoir la même valeur de retour qu'un de ses enfants sans qu'une raison l'explique.

S'entraîner : exercices C4 et C5 du chapitre.

Exercice 1 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 2 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 3 ◆◆

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 4 ◆

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```

1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

LANGAGES, RÉCURSIVITÉ ET PARADIGMES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Affirmer qu'un programme est aussi une donnée signifie que :
 - A. un programme peut être lu, transformé ou exécuté par un autre programme.
 - B. un programme n'a aucun effet lorsqu'on l'exécute.
 - C. les programmes et les données occupent la même quantité de mémoire.
 - D. un programme ne peut être écrit que par une machine.

Capacité C2

2. [Q2] Dire que la calculabilité ne dépend pas du langage employé signifie que :
 - A. tous les langages s'exécutent à la même vitesse.
 - B. ce qui est calculable dans un langage l'est dans tout autre langage universel.
 - C. tous les langages ont la même syntaxe.
 - D. un algorithme ne peut s'écrire que dans un seul langage.

Capacité C3

3. [Q3] Dire que le problème de l'arrêt est indécidable signifie que :
 - A. aucun programme ne s'arrête jamais.
 - B. on ne peut jamais prouver qu'un programme donné s'arrête.
 - C. aucun programme unique ne peut décider, pour tout programme et toute entrée, si l'exécution s'arrête.
 - D. cette difficulté ne concerne que Python.

Capacité C4

4. [Q4] Dans une fonction récursive, le cas de base sert à :
 - A. accélérer le calcul.
 - B. déclarer les variables locales.
 - C. vérifier le type des arguments reçus.
 - D. arrêter la récursion en fournissant une valeur sans nouvel appel.

Capacité C5

5. [Q5] Un grand nombre d'appels dans l'arbre d'appels d'une fonction récursive indique que :
 - A. la fonction termine, mais peut coûter cher en temps.
 - B. le code comporte nécessairement une erreur.
 - C. la fonction ne termine jamais.
 - D. le cas de base est mal écrit.

Capacité C6

6. [Q6] L'API d'une bibliothèque désigne :
- A. son code source complet.
 - B. l'ensemble des fonctions et des paramètres qu'elle met à disposition.
 - C. le nom de son auteur.
 - D. sa vitesse d'exécution.

Capacité C7

7. [Q7] La documentation d'une fonction indique qu'elle « renvoie None si la clé est absente ». Le programme appelant doit :
- A. modifier la bibliothèque pour qu'elle lève une exception.
 - B. ignorer cette mention, qui ne concerne que les développeurs de la bibliothèque.
 - C. traiter explicitement ce cas avant d'utiliser la valeur renvoyée.
 - D. supposer que la clé est toujours présente.

Capacité C8

8. [Q8] Dans un module Python destiné à être réutilisé, le bloc `if __name__ == "__main__":` sert à :
- A. déclarer les fonctions exportées par le module.
 - B. documenter le module.
 - C. importer automatiquement toutes les dépendances.
 - D. n'exécuter le code de démonstration ou de test que si le fichier est lancé directement.

Capacité C9

9. [Q9] Un même programme Python peut mobiliser :
- A. plusieurs paradigmes à la fois : impératif, fonctionnel et objet.
 - B. le seul paradigme objet.
 - C. un seul paradigme, imposé par le langage.
 - D. aucun paradigme identifiable.

Capacité C10

10. [Q10] Pour appliquer une même transformation à chaque élément d'une liste, sans modifier la liste de départ, le style le mieux adapté est :
- A. impératif, en modifiant la liste sur place case par case.
 - B. fonctionnel, par une compréhension ou une application de fonction produisant une nouvelle liste.
 - C. objet, en créant une classe par élément.
 - D. aucun : cette transformation est impossible.

Capacité C11

11. [Q11] Comparer deux nombres flottants avec l'opérateur `==` est une source de bugs parce que :
- A. Python interdit cette comparaison.
 - B. l'opérateur ne fonctionne qu'entre entiers.
 - C. les flottants ne représentent pas exactement toutes les valeurs décimales.
 - D. tous les flottants sont considérés comme égaux entre eux.
12. [Q12] Écrire `b = a`, où `a` est une liste :
- A. vide la liste `a`.
 - B. crée une copie indépendante de `a`.
 - C. provoque une erreur.
 - D. fait désigner à `b` le même objet que `a`.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	D
Q5	C5	A
Q6	C6	B
Q7	C7	C
Q8	C8	D
Q9	C9	A
Q10	C10	B
Q11	C11	C
Q12	C11	D

Diagnostic des réponses erronées

► Q1

- **B** — Être une donnée ne l'empêche pas d'être exécutable : c'est justement de porter les deux statuts qui fait sa force. *Renvoi : C1, programme et donnée.*
- **C** — L'élève rapproche deux notions sans rapport. Il s'agit de la NATURE du programme, non de son encombrement. *Renvoi : C1, programme et donnée.*
- **D** — Que des programmes puissent en produire d'autres, comme le fait un compilateur, ne retire rien à l'écriture humaine. *Renvoi : C1, programme et donnée.*

► Q2

- **A** — L'élève confond calculabilité et performance. Le résultat porte sur ce qui est possible, jamais sur le temps mis à l'obtenir. *Renvoi : C2, calculabilité et langages.*
- **C** — Les syntaxes diffèrent nettement ; c'est le POUVOIR d'expression qui coïncide, non l'écriture. *Renvoi : C2, calculabilité et langages.*
- **D** — C'est exactement l'inverse : un même algorithme s'écrit dans n'importe quel langage universel. *Renvoi : C2, calculabilité et langages.*

► Q3

- **A** — Beaucoup de programmes s'arrêtent, et on le constate. Ce qui est impossible est de trancher pour TOUS les cas par un procédé unique. *Renvoi : C3, indécidabilité du problème de l'arrêt.*
- **B** — L'élève passe du général au particulier. Pour un programme donné, une preuve est souvent accessible ; c'est la méthode universelle qui n'existe pas. *Renvoi : C3, indécidabilité du problème de l'arrêt.*
- **D** — Le résultat est indépendant du langage : il vaut pour tout modèle de calcul universel. *Renvoi : C3, indécidabilité du problème de l'arrêt.*

► Q4

- **A** — Le cas de base ne vise pas la rapidité mais la TERMINAISON : sans lui, les appels se poursuivent jusqu'au débordement de la pile. *Renvoi : C4, écrire un programme récursif.*
- **B** — La déclaration de variables n'a aucun rapport avec la condition d'arrêt des appels. *Renvoi : C4, écrire un programme récursif.*
- **C** — La vérification des arguments est une précaution indépendante, qui n'interrompt pas la récursion. *Renvoi : C4, écrire un programme récursif.*

► Q5

- **B** — La récursion naïve du calcul de Fibonacci est correcte, tout en engendrant un nombre exponentiel d'appels. *Renvoi : C5, analyse d'un programme récursif.*
- **C** — Un arbre d'appels FINI, même vaste, décrit précisément une exécution qui se termine. *Renvoi : C5, analyse d'un programme récursif.*
- **D** — Un cas de base fautif produirait une récursion sans fin, donc une erreur d'exécution, et non simplement un grand arbre. *Renvoi : C5, analyse d'un programme récursif.*

► Q6

- A — L'élève confond l'interface et sa réalisation. L'API est ce que la bibliothèque promet ; le code source est la façon dont elle tient sa promesse. *Renvoi : C6, utiliser une API.*
- C — La paternité relève des métadonnées du paquet, sans rapport avec les points d'entrée disponibles. *Renvoi : C6, utiliser une API.*
- D — La performance est une propriété mesurée, et non une description de ce qui est offert. *Renvoi : C6, utiliser une API.*

► Q7

- A — Modifier une dépendance pour éviter d'en lire le contrat est disproportionné, et rend toute mise à jour impossible. *Renvoi : C7, exploiter une documentation.*
- B — Cette mention s'adresse au contraire à l'appelant : elle décrit ce qu'il recevra, et donc ce qu'il doit prévoir. *Renvoi : C7, exploiter une documentation.*
- D — La documentation avertit précisément du contraire. L'ignorer conduira à manipuler None comme s'il s'agissait d'une valeur. *Renvoi : C7, exploiter une documentation.*

► Q8

- A — Les fonctions sont exportées du seul fait d'être définies ; ce bloc n'en déclare aucune. *Renvoi : C8, créer et documenter un module.*
- B — La documentation est portée par la docstring placée en tête du module, non par une condition d'exécution. *Renvoi : C8, créer et documenter un module.*
- C — Les imports s'écrivent explicitement en tête de fichier ; aucun mécanisme automatique n'existe. *Renvoi : C8, créer et documenter un module.*

► Q9

- B — Une simple boucle for qui modifie une variable relève de l'impératif, sans le moindre objet défini par le programmeur. *Renvoi : C9, paradigmes de programmation.*
- C — Python est un langage multiparadigme : il n'en impose aucun et les laisse coexister. *Renvoi : C9, paradigmes de programmation.*
- D — Tout programme relève d'au moins un style d'organisation ; l'absence de paradigme n'a pas de sens. *Renvoi : C9, paradigmes de programmation.*

► Q10

- A — Modifier sur place contredit l'exigence énoncée : la liste initiale doit rester intacte. *Renvoi : C10, choisir un paradigme adapté.*
- C — Introduire une classe par élément alourdit considérablement le programme sans rien apporter à une simple transformation. *Renvoi : C10, choisir un paradigme adapté.*
- D — L'opération est au contraire courante ; c'est même l'exemple type du style fonctionnel. *Renvoi : C10, choisir un paradigme adapté.*

► Q11

- A — La comparaison est autorisée et ne lève aucune erreur : c'est bien ce qui la rend trompeuse. *Renvoi : C11, causes typiques de bugs.*
- B — == s'applique à tous les types ; le problème vient de la représentation des flottants, non de l'opérateur. *Renvoi : C11, causes typiques de bugs.*
- D — Deux flottants distincts sont bien reconnus comme différents. La difficulté est que deux calculs mathématiquement égaux peuvent donner deux valeurs distinctes. *Renvoi : C11, causes typiques de bugs.*

► Q12

- A — Le contenu de a n'est en rien affecté : seul un nouveau nom est introduit. *Renvoi : C11, alias et objets mutables.*
- B — L'affectation ne copie jamais un objet mutable : elle ne fait que lui donner un second nom, et toute modification par l'un se voit par l'autre. *Renvoi : C11, alias et objets mutables.*
- C — L'opération est parfaitement licite ; elle est même très courante, ce qui explique que le piège passe inaperçu. *Renvoi : C11, alias et objets mutables.*

Corrigé

Q1. Le code source d'un programme est une suite de caractères, stockable et manipulable comme n'importe quelle donnée : par exemple, un interpréteur lit le code d'un autre programme comme une donnée en entrée, pour l'exécuter instruction par instruction.

Q2. Si un programme `s_arrete(P, e)` pouvait toujours décider correctement si `P` s'arrête sur `e`, on pourrait construire un programme paradoxe qui fait exactement le contraire de ce que `s_arrete(paradoxe, paradoxe)` prédit pour lui-même : `s_arrete` se tromperait alors nécessairement sur ce cas précis, ce qui contredit l'hypothèse qu'il décide correctement dans tous les cas. Un tel programme ne peut donc pas exister.

Corrigé

Q1.

```
1 def puissance(base, exposant):
2     if exposant == 0:
3         return 1
4     return base * puissance(base, exposant - 1)
```

Q2. `puissance(2, 4)` appelle `puissance(2, 3)`, qui appelle `puissance(2, 2)`, qui appelle `puissance(2, 1)`, qui appelle `puissance(2, 0)` : ce dernier atteint le cas de base (`exposant == 0`) et renvoie 1 directement. Les résultats remontent ensuite en cascade : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$.

Évaluation A — Calculabilité et récursivité

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (programme comme donnée, problème de l'arrêt); Ex. 2 : 12 pts (récursivité)

Exercice 1 — Programme comme donnée et calculabilité

(8 points)

- (4 pts) Expliquer, avec un exemple, en quoi un programme est aussi une donnée.
- (4 pts) Résumer en quelques phrases l'argument montrant que le problème de l'arrêt est indécidable (sans formalisme, l'idée du raisonnement par l'absurde suffit).

Exercice 2 — Récursivité

(12 points)

- (6 pts) Écrire une fonction récursive `puissance(base, exposant)` qui calcule $\text{base}^{\text{exposant}}$ pour un exposant entier positif ou nul (sans utiliser l'opérateur `**`).
- (6 pts) Décrire l'arbre d'appels de `puissance(2, 4)` : quels appels successifs sont déclenchés, et quel est le cas de base atteint ?

Corrigé

```
1 def est_premier(n):
2     """Indique si l'entier n (n > 1) est premier.
3
4     Parametre : n, entier strictement superieur a 1.
5     Renvoie : True si n est premier, False sinon.
6     """
7     for d in range(2, n):
8         if n % d == 0:
9             return False
10    return True
```

```

11
12 print(est_premier(7)) # True
13 print(est_premier(8)) # False

```

Corrigé

Q1.

```

1 def carres_positifs_fonctionnel(valeurs):
2     return [v ** 2 for v in valeurs if v > 0]

```

Q2. La fonction native masquée est `sum`. Le code fonctionne malgré tout **ici** car la variable locale `sum` n'est utilisée que comme accumulateur, sans jamais appeler la fonction native `sum(...)` dans le reste de la fonction. Le risque apparaîtrait si l'on avait besoin d'appeler `sum(...)` plus loin dans le même bloc : l'appel échouerait car `sum` désignerait alors un entier, plus une fonction.

Q3. `resultat == 0.1` renvoie `False` : à cause des arrondis internes de la représentation binaire des flottants, $0.3 - 0.2$ ne vaut pas exactement 0.1 (l'écart est de l'ordre de 10^{-17}). Comparer deux flottants avec `==` est risqué pour cette raison ; il faut comparer leur écart à une tolérance près : `abs(resultat - 0.1) < 1e-9`.

Évaluation B — Modules, paradigmes et débogage

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (module documenté) ; Ex. 2 : 12 pts (paradigmes, débogage)

Exercice 1 — Créer un module documenté

(8 points)

- (5 pts) Écrire une fonction `est_premier(n)`, destinée à un module `arithmetique.py`, qui renvoie `True` si l'entier `n` (supérieur à 1) est premier, `False` sinon. Ajouter une docstring décrivant son rôle, son paramètre et sa valeur de retour.
- (3 pts) Tester la fonction sur $n = 7$ (premier) et $n = 8$ (non premier).

Exercice 2 — Paradigmes et débogage

(12 points)

- (4 pts) Réécrire en style fonctionnel (une liste en compréhension) la fonction impérative suivante :

```

1 def carres_positifs_imperatif(valeurs):
2     resultat = []
3     for v in valeurs:
4         if v > 0:
5             resultat.append(v ** 2)
6     return resultat

```

- (4 pts) Le code suivant contient un bug de nommage :

```

1 def calculer_moyenne(notes):
2     sum = 0
3     for n in notes:
4         sum += n
5     return sum / len(notes)

```

Quelle fonction native est masquée ? Ce code fonctionne-t-il malgré tout pour calculer une moyenne simple ? Justifier.

- (4 pts) Le code suivant compare deux résultats de calculs flottants avec `==`. Que renvoie-t-il, pourquoi est-ce risqué de comparer ainsi, et comment corriger ?

```

1 resultat = 0.3 - 0.2
2 print(resultat == 0.1)

```

Sujet S2 — Exercice 2 : récursivité et coût d'un calcul répété

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Un robot part du coin inférieur gauche d'une grille et doit rejoindre le coin supérieur droit. Il ne peut avancer que d'une case vers la droite ou d'une case vers le haut. On note $C(\ell, h)$ le nombre de chemins possibles sur une grille de ℓ pas horizontaux et h pas verticaux.

- Justifier que $C(\ell, 0) = C(0, h) = 1$, puis que pour $\ell > 0$ et $h > 0$ on a $C(\ell, h) = C(\ell - 1, h) + C(\ell, h - 1)$. Écrire la fonction récursive `chemins(largeur, hauteur)` correspondante, et donner $C(2, 2)$ et $C(3, 3)$.
- On instrumente la fonction pour compter ses appels. On obtient 39 appels pour $C(3, 3)$ et 503 pour $C(5, 5)$. Expliquer d'où vient cette croissance, en montrant sur $C(2, 2)$ qu'un même couple est calculé plusieurs fois.
- Écrire `chemins_memo` qui mémorise les résultats déjà calculés dans un dictionnaire. Vérifier qu'elle donne les mêmes valeurs que la question 1. Combien d'entrées ce dictionnaire contient-il après le calcul de $C(5, 5)$? Justifier ce nombre.
- Un élève écrit la signature `def chemins_memo(largeur, hauteur, memo={})`. Le calcul reste juste, mais quelque chose change entre deux appels successifs de la fonction. Quoi, et pourquoi ? Dire si c'est souhaitable ici, et ce que la signature `memo=None` change.
- Écrire une version **itérative** `chemins_iteratif` qui n'utilise qu'une seule liste de taille $h + 1$. Vérifier qu'elle coïncide avec les précédentes, puis donner $C(10, 10)$. Comparer les trois versions sur ce que chacune consomme.

Corrigé

Question 1. (4 points)

Si $h = 0$, le robot ne peut qu'avancer tout droit : un seul chemin. Même raisonnement si $\ell = 0$. Sinon, le **dernier** pas est soit horizontal, soit vertical, et ces deux cas sont disjoints et couvrent tout : d'où la somme.

```

1 def chemins(largeur, hauteur):
2     if largeur == 0 or hauteur == 0:
3         return 1
4     return chemins(largeur - 1, hauteur) + chemins(largeur, hauteur - 1)

```

$C(2, 2) = 6$ et $C(3, 3) = 20$.

Question 2. (4 points)

$C(2, 2)$ appelle $C(1, 2)$ et $C(2, 1)$; chacun appelle $C(1, 1)$. Le couple $(1, 1)$ est donc calculé **deux fois**, entièrement, sans que rien ne retienne le premier résultat. Le phénomène se répète à chaque niveau : le nombre d'appels croît comme le nombre de *chemins*, pas comme le nombre de *couples*.

Question 3. (5 points)

```

1 def chemins_memo(largeur, hauteur, memo=None):
2     if memo is None:
3         memo = {}
4     if largeur == 0 or hauteur == 0:

```

```

5     return 1
6     if (largeur, hauteur) in memo:
7         return memo[(largeur, hauteur)]
8     resultat = (chemins_memo(largeur - 1, hauteur, memo)
9                 + chemins_memo(largeur, hauteur - 1, memo))
10    memo[(largeur, hauteur)] = resultat
11    return resultat

```

Après $C(5, 5)$, le dictionnaire contient **25** entrées : les couples (ℓ, h) avec $1 \leq \ell \leq 5$ et $1 \leq h \leq 5$. Les cas de base ne sont pas mémorisés, puisqu'ils sont renvoyés avant.

Question 4. (4 points)

Un argument par défaut est évalué **une seule fois**, à la définition de la fonction. Le dictionnaire est donc **partagé** par tous les appels : après `chemins_memo(2, 2)`, il contient déjà des entrées, et le prochain appel les réutilise.

Ici, le résultat reste juste — C ne dépend que de ses arguments — donc le partage accélère. Mais c'est une chance, pas une propriété : dès que la fonction dépendrait d'autre chose, le cache resservirait des valeurs périmées. La forme `memo=None` crée un dictionnaire **neuf** à chaque appel externe, et le partage devient un choix explicite de l'appelant.

Question 5. (3 points)

```

1 def chemins_iteratif(largeur, hauteur):
2     ligne = [1] * (hauteur + 1)
3     for _ in range(largeur):
4         for j in range(1, hauteur + 1):
5             ligne[j] = ligne[j] + ligne[j - 1]
6     return ligne[hauteur]

```

$C(10, 10) = 184\,756$.

Version	Appels ou tours	Mémoire
naïve	croît comme le résultat	pile d'appels
mémorisée	un par couple	un couple par case
itérative	un par case	une seule ligne

Critique

- « **La récursivité est lente** ». C'est faux tel quel : la version mémorisée est récursive et rapide. Ce qui coûte, c'est le recalcul, pas la récursion. La question 3 sert précisément à séparer les deux.
- **Mémoriser les cas de base**. Un candidat qui les mémorise obtient 35 entrées au lieu de 25 à la question 3, et une justification qui ne tient plus. Le compte demandé oblige à décrire exactement ce qui est stocké.
- **L'argument par défaut mutable**. Le piège est classique et l'exercice le pose là où il ne casse rien : le candidat doit expliquer un danger qu'il ne voit pas se réaliser. C'est plus exigeant que de constater un bug.
- **La version itérative « de droite à gauche »**. Parcourir j en sens inverse casserait la récurrence : `ligne[j-1]` doit être la valeur **déjà mise à jour** du tour courant.

Portée pédagogique

L'exercice traite le coût d'un calcul comme une propriété observable, pas comme une formule à réciter : on compte des appels, on compte des entrées de dictionnaire, on compare des mémoires. Aucune notation asymptotique n'est demandée, et c'est délibéré — la question 5 fait constater les trois régimes plutôt que de les nommer.

La question 4 est la seule à porter sur le langage lui-même plutôt que sur l'algorithme. Elle vaut quatre points parce qu'elle demande d'expliquer un mécanisme dont l'effet, ici, est invisible.

Sujet S3 — Exercice 3 : un effet de bord qui fausse un résultat

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Une station relève des températures. On veut « normaliser » une série en ramenant à 0 toute valeur négative, **sans** perdre la série d'origine. Un élève écrit :

```
1 def normaliser_bugue(relevés, minimum=0):
2     for i in range(len(relevés)):
3         if relevés[i] < minimum:
4             relevés[i] = minimum
5     return relevés
```

1. On exécute `mesures = [3, -2, 7, -5]` puis `resultat = normaliser_bugue(mesures)`. Donner la valeur de `resultat` et celle de `mesures`. Que renvoie `resultat` is `mesures` ? Interpréter.
2. Réécrire la fonction pour qu'elle renvoie une **nouvelle** liste sans modifier son argument. Vérifier sur le même exemple que `mesures` est intact.
3. Un programme applique la normalisation à chaque ligne d'un historique `[[1, -1], [2, -2]]`, puis compare l'historique à une copie prise avant. Dire ce que donne la comparaison avec chacune des deux fonctions, et expliquer pourquoi le défaut est resté invisible tant qu'on ne relisait pas l'original.
4. L'élève propose de se protéger en écrivant `copie = list(original)` avant d'appeler sa fonction sur `copie[0]`. Montrer que cela ne suffit pas. Quelle copie faut-il ? Expliquer la différence en une phrase.
5. Écrire une fonction de test qui, appliquée à l'une ou l'autre version, renvoie `True` pour la bonne et `False` pour la fautive. Écrire ensuite un test qui, lui, ne fait pas la différence, et dire ce qui lui manque.

Corrigé

Question 1. (4 points)

`resultat` vaut `[3, 0, 7, 0]`, et `mesures` vaut **aussi** `[3, 0, 7, 0]` : la série d'origine a été écrasée. `resultat` is `mesures` renvoie `True` — ce ne sont pas deux listes égales, c'est **une seule** liste, désignée par deux noms.

Question 2. (3 points)

```
1 def normaliser(relevés, minimum=0):
2     return [minimum if valeur < minimum else valeur for valeur in relevés]
```

`mesures` reste `[3, -2, 7, -5]`, et `resultat` is `mesures` renvoie `False`.

Question 3. (4 points)

Avec `normaliser`, l'historique est inchangé : la comparaison réussit. Avec `normaliser_bugue`, il devient `[[1, 0], [2, 0]]` : la comparaison échoue.

Le défaut est resté invisible parce que la **valeur renvoyée** est correcte dans les deux cas. Tant qu'on ne regarde que la sortie, les deux fonctions se comportent identiquement ; la différence n'apparaît qu'en relisant l'entrée.

Question 4. (5 points)

`list(original)` construit une nouvelle liste **extérieure**, mais ses cases pointent vers les **mêmes** sous-listes. Modifier `copie[0]` modifie donc `original[0]`.

Il faut une copie **profonde**, `copy.deepcopy(original)`, qui duplique aussi les sous-listes. En une phrase : une copie superficielle copie les références, une copie profonde copie ce qu'elles désignent.

Question 5. (4 points)

```
1 def test_normalisation(fonction):
2     entree = [3, -2]
3     temoin = list(entree)
4     sortie = fonction(entree)
5     return sortie == [3, 0] and entree == temoin
```

True pour normaliser, False pour normaliser_bugue.

Le test qui ne fait pas la différence :

```
1 def test_faible(fonction):
2     return fonction([3, -2]) == [3, 0]
```

Il renvoie True pour les deux. Il lui manque le **témoin** : sans conserver l'entrée avant l'appel, on ne peut pas constater qu'elle a changé.

Critique

- ▶ **Confondre == et is.** La question 1 les oppose délibérément. Deux listes peuvent être égales sans être la même ; ici elles sont la même, et c'est tout le problème.
- ▶ **Croire qu'un test qui passe suffit.** La question 5 construit l'exemple minimal d'un test vert sur une fonction fautive. C'est la leçon la plus transférable de l'exercice.
- ▶ **La copie superficielle.** Elle protège l'apparence — la longueur, l'ordre — mais pas le contenu imbriqué. L'erreur est fréquente et silencieuse.
- ▶ **Un effet de bord n'est pas toujours un défaut.** L'énoncé exige de *ne pas* perdre la série d'origine : c'est cette exigence, pas la mutation en soi, qui rend la première version fausse.

Portée pédagogique

L'exercice traite l'effet de bord comme un problème de **spécification**, pas de style : la fonction est fausse parce qu'elle viole une exigence écrite, et la question 5 montre qu'un jeu de tests mal conçu ne le verra jamais.

Il ne demande aucune structure de données avancée : tout tient en listes de listes. Ce qu'il évalue, c'est la capacité à distinguer ce qu'une fonction *renvoie* de ce qu'elle *fait*, et à concevoir un test qui observe la seconde.

Sujet S6 — Exercice 2 : spécifier puis tester une fonction de fusion

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

On veut une fonction `fusionner(a, b)` qui reçoit deux listes triées par ordre croissant et renvoie une liste triée contenant tous leurs éléments.

1. Écrire la **spécification** de cette fonction : son prototype, sa précondition, et trois postconditions distinctes. Les rédiger en français, puis les traduire en assertions exécutables quand c'est possible.
2. Écrire `fusionner`, puis un jeu de **huit** tests couvrant : le cas nominal, chacune des deux listes vide, les deux vides, des doublons, deux listes disjointes, deux listes entrelacées, et l'absence d'effet de bord.

- On appelle *mutant* une version volontairement modifiée de la fonction. Pour chacun des trois mutants suivants, dire s'il est tué par votre jeu de tests, et par lequel :
 - on oublie de concaténer le reste de la liste non épuisée ;
 - on consomme les listes avec `pop(0)` au lieu d'un indice ;
 - on remplace `<=` par `<`.
- L'un des trois mutants n'est tué par aucun test. Expliquer pourquoi, et construire un jeu de données — non entier — sur lequel la différence devient observable. Quel nom porte la propriété perdue ?
- Que doit faire fusionner si on l'appelle sur `[3, 1]` et `[2]` ? Montrer, en donnant la sortie d'une version sans contrôle, pourquoi laisser passer cet appel est pire que l'interdire.

Corrigé

Question 1. (4 points)

```

1 def fusionner(a, b):
2     """Fusionne deux listes triées croissantes.
3
4     Precondition : a et b sont triées par ordre croissant.
5     Postcondition : le resultat est trie ;
6                     il contient exactement les elements de a et de b,
7                     avec leurs multiplicites ;
8                     a et b ne sont pas modifiées.
9     """
10    assert a == sorted(a) and b == sorted(b), "entree non trie"

```

Les trois postconditions sont toutes vérifiables : `r == sorted(r)`, `sorted(r) == sorted(a + b)`, et la comparaison des arguments à un témoin pris avant l'appel.

Question 2. (5 points)

```

1 def fusionner(a, b):
2     assert a == sorted(a) and b == sorted(b), "entree non trie"
3     resultat, i, j = [], 0, 0
4     while i < len(a) and j < len(b):
5         if a[i] <= b[j]:
6             resultat.append(a[i]); i += 1
7         else:
8             resultat.append(b[j]); j += 1
9     return resultat + a[i:] + b[j:]

```

Les huit tests :

Cas	Appel
nominal	<code>([1, 4, 9], [2, 3])</code>
<i>a</i> vide	<code>([], [2, 3])</code>
<i>b</i> vide	<code>([1, 4], [])</code>
les deux vides	<code>([], [])</code>
doublons	<code>([2, 2], [2])</code>
disjointes	<code>([1, 2], [8, 9])</code>
entrelacées	<code>([1, 3, 5], [2, 4, 6])</code>
sans effet de bord	comparaison à un témoin après appel

Question 3. (5 points)

Mutant	Tué ?	Par
reste oublié	oui	six tests sur huit
<code>pop(0)</code>	oui	le test d'effet de bord, lui seul
<code><</code> au lieu de <code><=</code>	non	aucun

Le mutant `pop(0)` produit le **bon résultat** : seul le test d'effet de bord le démasque, en constatant que *a* et *b* ont été vidées.

Question 4. (4 points)

Sur des entiers, deux valeurs égales sont **indiscernables** : peu importe laquelle est placée en premier, la liste obtenue est la même. Le mutant reste donc correct au sens de la spécification écrite.

La différence apparaît sur des données portant une information supplémentaire, par exemple des couples (étiquette, clé) :

Comparaison	Fusion de $[(a, 1)]$ et $[(b, 1)]$
<code><=</code>	$[(a, 1), (b, 1)]$
<code><</code>	$[(b, 1), (a, 1)]$

La propriété perdue est la **stabilité** : à clé égale, l'élément venu de la liste de gauche doit rester devant.

Question 5. (2 points)

L'appel viole la précondition : il doit être **refusé**. Sans contrôle, la fonction renvoie `[2, 3, 1]` — une liste qui n'est même pas triée, alors que la postcondition l'exige. Elle rend donc un résultat d'apparence normale, que l'appelant utilisera comme s'il était trié, et l'erreur se manifestera bien plus loin, en un point qui n'a plus rien à voir avec sa cause.

Critique

- **Un mutant qui survit n'est pas un échec du jeu de tests.** Il signale que la spécification ne mentionne pas la propriété concernée. La question 4 demande d'enrichir les données, pas seulement d'ajouter un test.
- **Le test d'effet de bord.** Sept tests sur huit passent sur le mutant `pop(0)`. Sans ce huitième, une fonction qui détruit ses arguments serait déclarée correcte.
- **Écrire la précondition sans la vérifier.** La question 5 montre le coût exact de cette négligence : une sortie non triée, silencieuse, et une erreur diagnostiquée ailleurs.
- **Confondre « trié » et « contient les mêmes éléments ».** Les deux postconditions sont nécessaires : une fonction qui renvoie `sorted(a)` seule satisfait la première et pas la seconde.

Portée pédagogique

L'exercice inverse l'ordre habituel : on écrit d'abord ce que la fonction doit garantir, ensuite seulement comment elle le fait, et l'on juge le jeu de tests à sa capacité à tuer des versions fausses.

La question 4 est la plus formatrice : elle montre qu'un test qui ne détecte rien peut signaler un manque dans la spécification plutôt qu'une faiblesse du test. C'est ce déplacement — du code vers ce qu'on a promis — que l'exercice cherche à installer.

Situation pratique P5 — corriger une fonction récursive et la tester

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier `chiffres.py` contient une fonction **fautive** et le jeu de tests qui servira à l'évaluation.

```
1 def somme_chiffres(n):
2     if n == 0:
3         return 0
```

```
4 | return n % 10 + somme_chiffres(n // 10)
```

Travail demandé

1. Exécuter `somme_chiffres(7)`. Décrire précisément ce qui se passe, et pourquoi.
2. Corriger la fonction. Une seule opération est en cause : dire laquelle, et par quoi la remplacer.
3. Écrire la spécification en docstring : une précondition et trois postconditions, dont l'une relie la somme des chiffres à n modulo 9. Rendre la précondition exécutable.
4. Exécuter le jeu de tests fourni. Il doit passer entièrement, y compris la vérification de la postcondition modulo 9 sur cinq cents entiers.
5. Vérifier que les appels `somme_chiffres(-1)`, `somme_chiffres(3.5)` et `somme_chiffres("12")` sont tous les trois refusés.

Ce qui est évalué

Critère	Points
Le diagnostic de la question 1 est exact	4
La correction est minimale et juste	4
Les trois postconditions sont écrites, dont celle modulo 9	5
Le jeu de tests fourni passe entièrement	4
Les trois appels fautifs sont refusés	3

Réponses attendues

Question 1. L'appel provoque une `RecursionError`. Pour $n = 7$, l'expression `n % 10` vaut encore 7 : la fonction s'appelle avec le **même** argument, indéfiniment. Le cas d'arrêt `n == 0` n'est jamais atteint.

Question 2. L'appel récursif doit porter sur `n // 10`, qui retire le dernier chiffre, et non sur `n % 10`, qui l'isole. Le cas d'arrêt devient `n < 10`.

Question 3. Précondition : n est un entier positif ou nul. Postconditions : le résultat est un entier positif ou nul ; il est nul si et seulement si n l'est ; il est congru à n modulo 9.

Critique

- ▶ **Corriger le cas d'arrêt au lieu de l'appel.** Écrire `if n < 10` sans changer `n % 10` en `n // 10` fait renvoyer $n \bmod 10$ à tout coup : la fonction ne plante plus, et elle est fausse. C'est pire qu'avant.
- ▶ **La postcondition modulo 9.** Elle n'est pas décorative : c'est la seule qui teste le résultat sur cinq cents valeurs sans avoir à les calculer une par une.
- ▶ **Accepter les flottants.** `3.5 % 10` vaut 3,5 et ne lève aucune erreur : sans contrôle de type, la fonction renvoie un flottant absurde.

Portée pédagogique

La situation part d'un code qui *plante* pour aller vers un code *spécifié*. Le diagnostic de la question 1 vaut autant que la correction : comprendre pourquoi une récursion ne termine pas est plus formateur que d'écrire la version juste du premier coup.

La postcondition modulo 9 introduit l'idée d'une propriété vérifiable en masse — un invariant mathématique qui sert de test — sans exiger sa démonstration.

Exercice 5 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 | def indice_valide_bug(indice, n):
2 |     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Version aménagée — Extrait Langages et programmation

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Une fonction récursive (C4, C5)

On veut la somme des chiffres d'un entier positif. Exemple : $327 \rightarrow 12$.

Étape 1. Décompose 327. Une ligne à la fois.

Rappel : $n \% 10$ donne le dernier chiffre, $n // 10$ enlève ce dernier chiffre.

n	Dernier chiffre	Reste
327	7	32
32		..
3		..

Étape 2. Quand la décomposition s'arrête-t-elle ?

Quand n vaut

Étape 3. Complète la fonction. Le cas d'arrêt d'abord.

```
1 def somme_chiffres(n):
2     if n == ____:
3         return 0
4     return n % 10 + somme_chiffres(n ____ 10)
```

Le second trou : ou

Étape 4. Sans cas d'arrêt, que se passerait-il ?

Exercice 2 — Un effet de bord (C11)

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 mien = [12, 15]
6 tien = ajouter_note(mien, 18)
```

Étape 1. Complète le tableau après ces quatre lignes.

Variable	Valeur
tien	[12, 15, 18]
mien	

Étape 2. La liste de départ a-t-elle été modifiée ?

C'est ce qu'on appelle un de bord.

Étape 3. Complète la version sans effet de bord.

```
1 def ajouter_note(bulletin, note):
2     copie = _____(bulletin)
3     copie.append(note)
4     return copie
```

Le mot manquant : ou

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.



6 Structures de données

6.1 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```
1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînee:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None
```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.2 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier self dans la définition d'une méthode. Toute méthode d'instance doit avoir self comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel objet.methode()) : Python le fournit automatiquement.

6.3 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retire. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retire. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (collections.deque est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.4 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un noeud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de noeuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.5 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **arêtes**. Un graphe est **orienté** si ses arêtes ont un sens (representees par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arête de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j ou $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une memoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une memoire proportionnelle au nombre d'arêtes reellement presentes, plus adaptees aux graphes creux, mais tester l'existence d'une arête précise peut necessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaitre. Dans la représentation par listes de successeurs, un sommet isole ou puits (sans arête sortante) doit être present dans

le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

🔗 MÉTHODE M1 Spécifier une structure de données par son interface

Quand l'utiliser ? Quand l'énoncé décrit un besoin en français — « on veut gérer une file d'attente de patients », « on veut un annuaire consultable » — et demande de **spécifier** la structure avant de l'implémenter. Spécifier, c'est dire ce que la structure fait, jamais comment elle le fait.

Pas à pas :

1. **Liste les opérations** que le texte demande, une par verbe : ajouter, retirer, consulter, compter, tester la vacuité. Si un verbe n'apparaît pas dans l'énoncé, ne l'invente pas : une interface minimale se défend mieux qu'une interface bavarde.
2. **Donne à chacune sa signature** : son nom, ses paramètres, et ce qu'elle renvoie. Distingue nettement celles qui **renvoient** une valeur de celles qui **modifient** la structure.
3. **Écris ce que chaque opération exige** — sa précondition. Dépiler exige une pile non vide ; consulter le premier exige au moins un élément.
4. **Écris ce que chaque opération garantit** — sa postcondition, en reliant l'état d'après à l'état d'avant. Après empiler(v), la structure contient un élément de plus et son sommet vaut v.
5. **Décide des cas limites** explicitement : structure vide, un seul élément, élément absent. Une spécification qui ne dit rien du cas vide est incomplète.
6. **Relis en interdisant tout mot d'implémentation**. Si ta spécification parle de liste, d'indice, de maillon ou de append, elle a déjà choisi l'implémentation : réécris-la.

Exemple : « on veut gérer une file d'attente de patients : on inscrit un patient, on appelle le suivant, et on veut savoir s'il reste quelqu'un ».

Opérations : inscrire(patient), appeler(), est_vide().

Signatures : inscrire prend un patient et ne renvoie rien ; appeler ne prend rien et renvoie un patient ; est_vide ne prend rien et renvoie un booléen.

Préconditions : appeler exige une file non vide. Les deux autres n'exigent rien.

Postconditions : après inscrire(p), la file contient un patient de plus et est_vide() est faux. appeler renvoie le patient inscrit **le plus anciennement** et le retire.

C'est cette dernière phrase — le plus anciennement — qui fait de la structure une file et non une pile. Elle appartient à la spécification, pas à l'implémentation.

Pièges :

- ▶ **Décrire l'implémentation en croyant spécifier.** « On stocke dans une liste et on retire avec pop(0) » n'est pas une interface : c'est déjà un choix, et il ferme la porte à toute autre implémentation.
- ▶ **Oublier la précondition du cas vide.** C'est le seul cas où la structure ne peut pas répondre, et c'est celui que l'énoncé teste.
- ▶ **Confondre renvoyer et modifier.** Une opération qui consulte sans retirer et une opération qui retire sont deux opérations différentes, avec deux postconditions.
- ▶ **Une spécification qui ne distingue pas file et pile.** Si ton contrat est satisfait aussi bien par une pile que par une file, c'est qu'il ne dit pas dans quel ordre les éléments sortent — la phrase la plus importante manque.

Vérifier : écris ton contrat sous forme de jeu de tests, puis fais-le passer par **deux** implémentations aux données internes différentes. S'il passe les deux, il ne parle bien que de l'interface. S'il échoue sur l'une, c'est qu'il a déjà choisi.

S'entraîner : exercices C1 du chapitre.

④ MÉTHODE M2 Modéliser une situation par un graphe

Quand l'utiliser ? Quand l'énoncé décrit un réseau, des relations, des dépendances ou des déplacements, et demande de « modéliser la situation par un graphe ». La difficulté n'est pas de dessiner : elle est de décider ce que sont les sommets, et surtout de s'y tenir.

Pas à pas :

1. **Choisis les sommets** : ce sont les **objets** entre lesquels une relation existe. Écris la phrase « un sommet est un... ». Si tu n'arrives pas à finir la phrase avec un nom unique, ton choix est mauvais.
2. **Choisis les arêtes** : ce sont les **relations**. Écris la phrase « il y a une arête entre u et v lorsque... ». Cette phrase doit être vérifiable sur l'énoncé, sans interprétation.
3. **Décide de l'orientation**. La relation est-elle symétrique ? « être ami avec » l'est, « suivre sur un réseau social » ne l'est pas, « précéder dans un enchaînement de tâches » non plus. Un graphe orienté à tort double le nombre d'arêtes ; un graphe non orienté à tort efface l'information du sens.
4. **Décide de la pondération**. Si l'énoncé donne des durées, des distances ou des coûts, ils vont **sur les arêtes**. S'il n'en donne pas, ne pondère pas.
5. **Écris la représentation** : dictionnaire de listes de successeurs, ou matrice d'adjacence. Pour un graphe non orienté, chaque arête doit apparaître **deux fois**, une dans chaque sens.
6. **Relis la question posée**. Elle se traduit presque toujours en un terme du cours : « peut-on aller de A à B » est une recherche de chemin, « peut-on tout visiter » est une question de connexité, « y a-t-il une boucle de dépendances » est une détection de cycle.

Exemple : « Quatre stations de métro. La ligne 1 relie A et B , et B et C . La ligne 2 relie C et D . Peut-on aller de A à D ? »

Sommets : un sommet est une **station**. Quatre sommets : A, B, C, D .

Arêtes : il y a une arête entre deux stations lorsqu'une ligne les relie directement.

Orientation : le métro se parcourt dans les deux sens, la relation est symétrique : graphe **non orienté**.

Pondération : l'énoncé ne donne aucune durée : aucune pondération.

Représentation : $\{A : [B], B : [A, C], C : [B, D], D : [C]\}$ — chaque arête figure bien deux fois.

Traduction de la question : « peut-on aller de A à D » est une recherche de chemin. Un parcours depuis A atteint D : oui.

Pièges :

- **Prendre pour sommets ce que la question ne demande pas**. Ici, modéliser les *lignes* comme sommets donne un graphe correct mais qui ne peut pas répondre à une question portant sur des stations.
- **Orienter une relation symétrique**. Le modèle devient asymétrique : on peut aller de A à D mais plus de D à A , ce qu'aucun métro ne fait.
- **Symétriser une relation qui ne l'est pas**. « Alan suit Ada » ne veut pas dire « Ada suit Alan » : symétriser invente des trajets qui n'existent pas.
- **N'écrire l'arête que dans un sens** dans un dictionnaire censé être non orienté. Le parcours devient faux, et rien ne le signale.

Vérifier : pour un graphe non orienté, contrôle que chaque voisin te renvoie l'ascenseur — si v figure chez u , u doit figurer chez v . Puis relis les deux phrases de l'étape 1 et 2 à côté de l'énoncé : elles doivent se vérifier ligne à ligne.

S'entraîner : exercices C11 du chapitre.

④ MÉTHODE M3 Passer d'une représentation de graphe à l'autre

Quand l'utiliser ? Quand l'énoncé donne une matrice d'adjacence et demande les listes de successeurs, ou l'inverse, ou quand un algorithme fourni attend l'autre représentation que celle de l'énoncé. La conversion est mécanique, mais deux détails la font rater : l'ordre des sommets, et les sommets sans voisin.

Pas à pas — de la matrice vers les listes :

1. **Numérote les sommets** et écris la correspondance numéro $\leftrightarrow$ nom. La ligne i et la colonne i désignent le **même** sommet : si tu changes l'ordre en route, tout est faux.
2. **Parcours la ligne i** , de gauche à droite. Chaque 1 en colonne j donne un successeur j du sommet i .
3. **Écris la liste de la ligne i** , même vide. Un sommet sans successeur doit apparaître avec une liste **vide**, pas disparaître.
4. **Recommence pour chaque ligne**. Tu dois obtenir autant d'entrées que la matrice a de lignes.

Pas à pas — des listes vers la matrice :

1. **Compte les sommets** : ce sont les clés, plus tout sommet qui n'apparaît que comme voisin. Oublier ces derniers donne une matrice trop petite.
2. **Pose une matrice $n \times n$ remplie de 0**, sommets rangés dans l'ordre choisi à l'étape précédente.
3. **Pour chaque successeur v de u , mets un 1 en ligne u , colonne v** — dans cet ordre, jamais l'inverse.
4. **Si le graphe est non orienté**, mets aussi le 1 symétrique. La matrice d'un graphe non orienté est **symétrique** : c'est ton contrôle.

Exemple : graphe orienté à trois sommets, $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 2$.

Matrice, lignes et colonnes dans l'ordre 0, 1, 2 :

ligne 0 : 0 1 1 ; ligne 1 : 0 0 1 ; ligne 2 : 0 0 0.

Listes : $\{0 : [1, 2], 1 : [2], 2 : []\}$. Le sommet 2 est un **puits** : sa liste est vide, et il figure quand même.

Retour aux listes vers la matrice : la ligne 2 ne contient que des 0, ce qui est cohérent. La matrice n'est **pas** symétrique, et c'est normal : le graphe est orienté.

Pièges :

- **Échanger ligne et colonne**. Mettre le 1 en ligne v , colonne u construit la **transposée** : toutes les arêtes sont inversées. Sur un graphe non orienté, l'erreur est invisible — la matrice est symétrique — et elle apparaîtra plus tard, sur un graphe orienté, sans que rien ne l'annonce.
- **Faire disparaître un sommet sans successeur**. Un puits doit rester dans le dictionnaire avec une liste vide, sinon un parcours l'ignore.
- **Oublier un sommet qui n'apparaît que comme voisin**. Il n'est clé de rien, mais il compte dans la taille de la matrice.
- **Changer l'ordre des sommets entre lignes et colonnes**. La correspondance numéro $\leftrightarrow$ nom doit être écrite une fois, et servir aux deux dimensions.

Vérifier : refais la conversion **dans l'autre sens** et compare au point de départ — les deux conversions sont réciproques. Et pour un graphe non orienté, contrôle que la matrice est symétrique : c'est immédiat à l'œil.

S'entraîner : exercices C14 du chapitre.

Exercice 1 ♦♦

On veut vérifier si une expression comportant des parenthèses $()$, crochets $[]$ et accolades $\{\}$ est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 2 ◆

Écrire une classe `Compte` représentant un compte bancaire simplifiée, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 3 ◆◆

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 4 ◆◆

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

STRUCTURES DE DONNÉES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] L'interface d'une structure de données décrit :
 - A. les opérations disponibles, sans préciser comment elles sont réalisées.
 - B. la façon dont les données sont rangées en mémoire.
 - C. le nombre d'octets occupés.
 - D. le langage de programmation employé.

Capacité C2

2. [Q2] Passer d'une implémentation d'une pile par tableau à une implémentation par liste chaînée, sans changer son interface :
 - A. oblige à réécrire tous les programmes qui utilisent la pile.
 - B. laisse inchangés les programmes qui n'utilisent que `empiler` et `depiler`.
 - C. change le principe LIFO en principe FIFO.
 - D. est impossible : l'interface dépend de l'implémentation.

Capacité C3

3. [Q3] Une file peut être implémentée par une liste Python où l'on retire en tête avec `pop(0)`, ou par deux piles. Ces deux implémentations :
- A. respectent le même contrat, mais n'ont pas le même coût.
 - B. sont interdites : une file n'admet qu'une implémentation.
 - C. produisent des ordres de sortie différents.
 - D. ont exactement les mêmes performances.

Capacité C4

4. [Q4] Dans la définition d'une classe Python, la méthode chargée d'initialiser un nouvel objet se nomme :
- A. `init`
 - B. `__init__`
 - C. `create`
 - D. `new`

Capacité C5

5. [Q5] Dans une méthode de classe Python, le paramètre `self` désigne :
- A. le premier attribut déclaré.
 - B. la classe elle-même.
 - C. l'objet sur lequel la méthode est appelée.
 - D. rien de particulier : il est facultatif.

Capacité C6

6. [Q6] On empile successivement 1, puis 2, puis 3 dans une pile vide, puis on dépile une fois. La valeur obtenue est :
- A. 1
 - B. la plus petite des trois.
 - C. 2
 - D. 3

Capacité C7

7. [Q7] Pour mémoriser les pages visitées afin de revenir en arrière dans un navigateur, la structure adaptée est :
- A. une pile.
 - B. un arbre binaire de recherche.
 - C. une file.
 - D. une matrice d'adjacence.

Capacité C8

8. [Q8] Rechercher une valeur associée à une clé est en moyenne plus rapide dans un dictionnaire que dans une liste, parce que :
- A. un dictionnaire est toujours plus petit qu'une liste.
 - B. l'accès passe par une table de hachage, sans dépendre du nombre d'éléments.
 - C. l'affirmation est fausse : les deux coûtent autant.
 - D. les dictionnaires sont triés automatiquement.

Capacité C9

9. [Q9] Parmi ces situations, laquelle appelle le plus naturellement une structure arborescente ?
- A. La liste des élèves d'une classe, par ordre alphabétique.
 - B. La file d'attente des travaux d'impression.
 - C. L'arborescence des dossiers et fichiers d'un disque.
 - D. Le relevé des températures d'une journée, heure par heure.

Capacité C10

10. [Q10] La taille d'un arbre binaire est :
- A. la longueur du plus long chemin de la racine à une feuille.
 - B. le nombre de ses niveaux moins un.
 - C. son nombre de feuilles.

D. son nombre de nœuds.

Capacité C11

11. [Q11] Pour modéliser un réseau social où l'amitié est réciproque, on emploie :
- A. un graphe non orienté, les personnes étant les sommets et les amitiés les arêtes.
 - B. une pile de profils.
 - C. un arbre binaire de recherche.
 - D. un graphe orienté, car l'amitié a toujours un sens.

Capacité C12

12. [Q12] Dans la matrice d'adjacence d'un graphe à n sommets, la case (i, j) vaut 1 lorsque :
- A. les sommets i et j portent le même numéro.
 - B. il existe une arête reliant i à j .
 - C. le sommet i est une feuille.
 - D. la somme $i + j$ vaut n .

Capacité C13

13. [Q13] Dans une implémentation par listes de successeurs, l'espace occupé par un graphe à n sommets et m arêtes est de l'ordre de :
- A. n^2 , quel que soit m .
 - B. m seulement, sans dépendre de n .
 - C. $n + m$.
 - D. $n \times m$.

Capacité C14

14. [Q14] Pour convertir une matrice d'adjacence en listes de successeurs, il faut :
- A. conserver seulement la diagonale de la matrice.
 - B. trier les sommets par nombre de voisins.
 - C. recopier chaque ligne de la matrice telle quelle, zéros compris.
 - D. pour chaque sommet, retenir les indices des colonnes dont la case vaut 1.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	A
Q4	C4	B
Q5	C5	C
Q6	C6	D
Q7	C7	A
Q8	C8	B
Q9	C9	C
Q10	C10	D
Q11	C11	A
Q12	C12	B
Q13	C13	C
Q14	C14	D

Diagnostic des réponses erronées

► Q1

- **B** — L'élève décrit l'IMPLEMENTATION. L'interface énonce le contrat ; le rangement en mémoire est un choix de réalisation qu'elle laisse libre. *Renvoi : C1, spécification par l'interface.*
- **C** — L'encombrement dépend de l'implémentation retenue, et varie donc d'une réalisation à l'autre pour une même interface. *Renvoi : C1, spécification par l'interface.*
- **D** — Une même interface, par exemple celle d'une pile, se retrouve dans tous les langages : elle en est indépendante. *Renvoi : C1, spécification par l'interface.*

► Q2

- **A** — C'est précisément ce que la séparation évite. Tant que les opérations gardent leur nom et leur comportement, le code appelant ne voit rien. *Renvoi : C2, interface et implémentation.*
- **C** — L'ordre de service fait partie du contrat, donc de l'interface. Changer d'implémentation ne peut pas le modifier. *Renvoi : C2, interface et implémentation.*
- **D** — L'élève inverse la dépendance : c'est l'implémentation qui doit respecter l'interface, et non l'inverse. *Renvoi : C2, interface et implémentation.*

► Q3

- **B** — C'est l'inverse : pouvoir réaliser une même structure de plusieurs façons est justement l'intérêt de la séparation entre interface et implémentation. *Renvoi : C3, implémentations multiples.*
- **C** — Si l'ordre différait, l'une des deux ne serait pas une file. Toutes deux servent les éléments dans leur ordre d'arrivée. *Renvoi : C3, implémentations multiples.*
- **D** — `pop(0)` décale tous les éléments restants, à un coût linéaire, alors que la version à deux piles amortit ce coût. *Renvoi : C3, coût d'une implémentation.*

► Q4

- **A** — Sans les doubles soulignements, la méthode est ordinaire : Python ne l'appelle pas automatiquement à la construction. *Renvoi : C4, définition d'une classe.*
- **C** — Aucune convention Python ne reconnaît ce nom ; la méthode ne serait jamais appelée d'elle-même. *Renvoi : C4, définition d'une classe.*
- **D** — L'élève transpose le mot-clé d'autres langages. En Python, l'instanciation s'écrit sans `new`. *Renvoi : C4, définition d'une classe.*

► Q5

- **A** — `self` donne accès à TOUS les attributs, par exemple `self.taille` ; il n'en désigne aucun en particulier. *Renvoi : C5, attributs et méthodes.*
- **B** — La classe est le moule ; `self` désigne l'exemplaire. Deux objets d'une même classe reçoivent des `self` différents. *Renvoi : C5, attributs et méthodes.*
- **D** — Il est au contraire indispensable : l'omettre prive la méthode de tout accès à l'objet et provoque une erreur à l'appel. *Renvoi : C5, attributs et méthodes.*

► Q6

- A — L'élève applique le principe FIFO, celui de la FILE. Une pile sert en dernier entré, premier sorti : c'est 3 qui ressort. *Renvoi : C6, LIFO et FIFO.*
- B — Une pile ne compare jamais les valeurs ; elle ne connaît que l'ordre d'arrivée. *Renvoi : C6, LIFO et FIFO.*
- C — Aucun élément intermédiaire n'est accessible : seul le sommet de la pile peut être retiré. *Renvoi : C6, LIFO et FIFO.*

► Q7

- B — L'arbre de recherche organise les données par valeur. Ici, c'est l'ordre de visite qui compte, non un ordre sur les adresses. *Renvoi : C7, choix d'une structure.*
- C — Une file rendrait d'abord la page la PLUS ANCIENNE, alors que le retour arrière doit rendre la plus récente. *Renvoi : C7, choix d'une structure.*
- D — La matrice d'adjacence décrit des liens entre sommets ; elle ne mémorise aucune chronologie. *Renvoi : C7, choix d'une structure.*

► Q8

- A — La taille n'entre pas en jeu : l'écart tient à la méthode d'accès, et il grandit justement avec le nombre d'éléments. *Renvoi : C8, recherche par clé et par parcours.*
- C — La recherche dans une liste parcourt les éléments un à un, à un coût linéaire, tandis que l'accès par clé est en temps quasi constant. *Renvoi : C8, recherche par clé et par parcours.*
- D — Aucun tri n'a lieu, et il ne servirait pas ici : le hachage calcule directement l'emplacement de la clé. *Renvoi : C8, recherche par clé et par parcours.*

► Q9

- A — Une liste ordonnée est linéaire. Un arbre ne s'impose que si l'on veut de surcroît accélérer les recherches. *Renvoi : C9, situations arborescentes.*
- B — Une file d'attente est purement linéaire : chaque travail a un seul suivant, sans ramification. *Renvoi : C9, situations arborescentes.*
- D — Une suite de mesures indexée par le temps est un tableau ; aucune hiérarchie n'y apparaît. *Renvoi : C9, situations arborescentes.*

► Q10

- A — L'élève définit la HAUTEUR. Les deux mesures diffèrent : un arbre dégénéré à n nœuds a pour taille n et pour hauteur $n - 1$. *Renvoi : C10, taille, hauteur et feuilles.*
- B — C'est encore la hauteur, exprimée par le nombre de niveaux. La taille dénombre les nœuds, pas les niveaux. *Renvoi : C10, taille, hauteur et feuilles.*
- C — Les feuilles ne forment qu'une partie des nœuds ; les nœuds internes comptent aussi dans la taille. *Renvoi : C10, taille, hauteur et feuilles.*

► Q11

- B — Une pile ordonne dans le temps ; elle ne représente aucune relation entre deux éléments. *Renvoi : C11, modélisation par un graphe.*
- C — Un arbre interdit les cycles et impose une racine. Or trois personnes amies deux à deux forment un cycle, situation banale ici. *Renvoi : C11, modélisation par un graphe.*
- D — L'orientation ne s'impose que si la relation est à sens unique, comme l'abonnement. Une amitié réciproque se représente par une arête sans direction. *Renvoi : C11, graphes orientés et non orientés.*

► Q12

- A — La diagonale ne vaut 1 que si le graphe comporte une boucle sur ce sommet, ce qui est un cas particulier et non la règle. *Renvoi : C12, matrice d'adjacence.*
- C — La notion de feuille appartient au vocabulaire des arbres ; la matrice code des arêtes, non des rôles de sommets. *Renvoi : C12, matrice d'adjacence.*
- D — L'élève invente une relation arithmétique entre indices. Le contenu de la matrice décrit le graphe, non la numérotation. *Renvoi : C12, matrice d'adjacence.*

► Q13

- **A** — C'est le coût de la MATRICE d'adjacence, qui réserve une case par couple de sommets, même absent. Les listes de successeurs ne stockent que les arêtes présentes. *Renvoi : C13, listes de successeurs.*
- **B** — Il faut de toute façon une entrée par sommet, y compris pour un sommet isolé sans aucune arête. *Renvoi : C13, listes de successeurs.*
- **D** — Aucune structure ne duplique les arêtes pour chaque sommet ; chaque arête n'est enregistrée qu'une ou deux fois. *Renvoi : C13, listes de successeurs.*

► **Q14**

- **A** — La diagonale ne renseigne que sur d'éventuelles boucles ; toutes les autres arêtes seraient perdues. *Renvoi : C14, conversion entre représentations.*
- **B** — Le tri modifierait l'ordre sans rien apporter : la conversion doit préserver exactement les mêmes arêtes. *Renvoi : C14, conversion entre représentations.*
- **C** — Conserver les zéros reviendrait à garder la matrice sous un autre nom, et à perdre le gain de place recherché. *Renvoi : C14, conversion entre représentations.*

Corrigé

```

1 class Rectangle:
2     def __init__(self, largeur, hauteur):
3         self.largeur = largeur
4         self.hauteur = hauteur
5
6     def aire(self):
7         return self.largeur * self.hauteur
8
9     def perimetre(self):
10        return 2 * (self.largeur + self.hauteur)

```

Corrigé

Q1. Interface : empiler(v), depiler(), est_vide(). Principe LIFO : le dernier élément empilé est le premier dépiler.

Q2. Après avoir empilé 5, 3, 8, 1, la pile est [5, 3, 8, 1] (du bas vers le sommet). Dépiler trois fois retire 1, puis 8, puis 3 (LIFO) : il reste [5].

Évaluation A — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (programmer une classe) ; Ex. 2 : 10 pts (pile, LIFO)

Exercice 1 — Classe Rectangle

(10 points)

1. (4 pts) Écrire une classe Rectangle avec un constructeur prenant largeur et hauteur.
2. (3 pts) Ajouter une méthode aire() qui renvoie l'aire du rectangle.
3. (3 pts) Ajouter une méthode perimetre() qui renvoie le perimetre du rectangle.

Exercice 2 — Pile

(10 points)

1. (4 pts) Rappeler l'interface d'une pile (trois méthodes) et le principe LIFO.
2. (6 pts) On empile successivement 5, 3, 8, 1, puis on dépile trois fois. Donner, en le justifiant, le contenu final de la pile.

Corrigé

Q1. Un graphe modélisé naturellement des éléments (stations) en relation (lignes directes) : les stations sont les sommets, les lignes directes les arêtes.

Q2. Si les lignes fonctionnent dans les deux sens, le graphe doit être **non orienté** (une arête entre deux stations représente une liaison utilisable dans les deux sens, sans distinction de direction).

Corrigé

Q1. $0 \rightarrow 1, 0 \rightarrow 2, 2 \rightarrow 0$.

Q2. La liste associée au sommet 1 est vide : 1 n'a aucun successeur, c'est un puits du graphe (aucune arête sortante).

Q3.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 0],
4     [1, 0, 0],
5 ]
```

Évaluation B — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (modélisation) ; Ex. 2 : 12 pts (graphes, conversion)

Exercice 1 — Modélisation

(8 points)

Un réseau de transport relie des stations par des lignes directes.

- (4 pts) Justifier qu'un graphe est une structure adaptée pour modéliser ce réseau.
- (4 pts) Ce graphe doit-il être orienté ou non orienté si les lignes fonctionnent dans les deux sens ? Justifier.

Exercice 2 — Graphes, conversion

(12 points)

On donne un graphe orienté à 3 sommets (0, 1, 2) par ses listes de successeurs :

```
1 successeurs = {0: [1, 2], 1: [], 2: [0]}
```

- (4 pts) Lister toutes les arêtes de ce graphe.
- (4 pts) Le sommet 1 a-t-il des successeurs ? Qu'est-ce que cela signifie ?
- (4 pts) Donner la matrice d'adjacence correspondant à ce graphe.

Sujet S1 — Exercice 1 : un arbre binaire de recherche de mesures

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Une station météorologique enregistre des relevés de température entiers. Pour retrouver rapidement une valeur, on les range dans un **arbre binaire de recherche** : à chaque nœud, toutes les valeurs

du sous-arbre gauche sont strictement inférieures à celle du nœud, toutes celles du sous-arbre droit strictement supérieures.

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Les relevés arrivent dans cet ordre : 18, 7, 25, 3, 12, 21, 30.

1. Écrire la fonction récursive `insérer(arbre, mesure)` qui renvoie l'arbre après insertion. Une valeur déjà présente ne doit pas être insérée une seconde fois. Dessiner ensuite l'arbre obtenu après les sept insertions, puis donner sa **taille** et sa **hauteur** (la hauteur d'un arbre réduit à sa racine vaut 0).
2. Écrire la fonction récursive `infixe(arbre)` qui renvoie la liste des valeurs par parcours infixe. Donner le résultat pour l'arbre de la question 1, et expliquer en une phrase ce que ce parcours garantit sur un arbre binaire de recherche.
3. Les mêmes sept relevés arrivent maintenant dans l'ordre croissant. Dessiner l'arbre obtenu et donner sa hauteur. Le parcours infixe est-il modifié ? Justifier.
4. Écrire `comparaisons(arbre, cle)`, qui renvoie le nombre de nœuds examinés pour retrouver `cle`. Donner sa valeur pour la clé 30 dans chacun des deux arbres des questions 1 et 3. Que peut-on en conclure sur l'intérêt d'un arbre équilibré ?
5. Un capteur défaillant renvoie deux fois la valeur 12. Quelle est la taille de l'arbre après cette seconde insertion ? Justifier à partir de votre code de la question 1.

Corrigé

Question 1. (4 points)

```
1 def insérer(arbre, mesure):
2     if arbre is None:
3         return Noeud(mesure)
4     if mesure < arbre.valeur:
5         arbre.gauche = insérer(arbre.gauche, mesure)
6     elif mesure > arbre.valeur:
7         arbre.droite = insérer(arbre.droite, mesure)
8     return arbre      # mesure déjà présente : rien à faire
```

L'arbre obtenu a 18 pour racine, 7 et 25 pour fils, puis 3 et 12 sous le 7, 21 et 30 sous le 25. **Taille 7, hauteur 2.**

Barème : 2 points pour la récursion correcte, 1 pour le traitement du doublon, 1 pour l'arbre et ses deux mesures.

Question 2. (3 points)

```
1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)
```

Résultat : [3, 7, 12, 18, 21, 25, 30]. Le parcours infixe d'un arbre binaire de recherche rend toujours les valeurs **dans l'ordre croissant**.

Question 3. (4 points)

Chaque nouvelle valeur étant supérieure à toutes les précédentes, elle descend systématiquement à droite : l'arbre dégénère en **peigne** de hauteur **6**. Le parcours infixe est **inchangé** : [3, 7, 12, 18, 21, 25, 30]. Deux arbres de formes très différentes peuvent porter le même ensemble de valeurs.

Question 4. (6 points)

```

1 def comparaisons(arbre, cle):
2     n = 0
3     while arbre is not None:
4         n += 1
5         if cle == arbre.valeur:
6             return n
7         arbre = arbre.gauche if cle < arbre.valeur else arbre.droite
8     return n

```

Arbre équilibré : 3 comparaisons. Arbre en peigne : 7. Le coût d'une recherche suit la **hauteur**, pas la taille. Sur un arbre équilibré de n valeurs, la hauteur est de l'ordre de $\log_2 n$; sur un peigne, elle vaut $n - 1$, et la recherche ne vaut pas mieux qu'un parcours de liste.

Question 5. (3 points)

La taille reste 7. Dans `insérer`, le cas `mesure == arbre.valeur` ne tombe ni dans le `if` ni dans le `elif` : l'arbre est renvoyé inchangé.

Critique

- ▶ **Le piège de la question 3.** Beaucoup de candidats répondent que le parcours infixe change parce que l'arbre change. C'est l'inverse : la forme dépend de l'ordre d'insertion, l'ordre infixe n'en dépend pas.
- ▶ **La hauteur d'un arbre réduit à sa racine.** L'énoncé la fixe à 0 parce que les deux conventions circulent. Sans cette précision, la question 1 aurait deux réponses également défendables.
- ▶ **Le doublon oublié.** Un `else` à la place du `elif` insérerait 12 une seconde fois à droite, et casserait l'invariant de l'arbre sans que rien ne le signale. C'est exactement ce que la question 5 vient tester.
- ▶ **Répondre « $\log n$ » sans condition.** Le coût logarithmique suppose l'équilibre. La question 4 est construite pour que le candidat constate le cas dégénéré avant de conclure.

Portée pédagogique

L'exercice fait travailler ensemble deux choses qu'on enseigne souvent séparément : l'écriture d'une insertion récursive, et le lien entre la forme d'une structure et le coût des opérations qu'on y fait. La progression est volontairement lente sur les trois premières questions — construire, parcourir, recommencer autrement — pour que la question 4 se réponde par observation plutôt que par récitation.

Il s'adresse à un candidat qui maîtrise la récursivité simple. La question 5 est courte et rapporte trois points : elle permet à un candidat en difficulté sur le coût de ne pas terminer l'exercice à vide.

Sujet S2 — Exercice 1 : une pile pour évaluer une expression postfixée

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

En notation **postfixée**, l'opérateur suit ses deux opérands : $3\ 5\ +$ vaut 8, et $3\ 5\ +\ 2\ *$ vaut 16. Cette notation n'a besoin d'aucune parenthèse.

1. On veut une classe `Pile` dont l'**interface** est `est_vide`, `empiler`, `depiler`, `sommet` et `taille`. Écrire cette classe en utilisant une liste Python comme implémentation. Expliquer en une phrase pourquoi l'attribut qui porte la liste est nommé `_contenu` et n'apparaît pas dans l'interface.

2. Écrire `evaluer(expression)`, qui reçoit une chaîne de jetons séparés par des espaces et renvoie la valeur de l'expression postfixée. On se limite aux opérateurs `+`, `-` et `*` sur des entiers. Dérouler le contenu de la pile, jeton par jeton, pour `3 5 + 2 *`.
3. Évaluer `9 4 -` puis `4 9 -`. Un élève dépile d'abord l'opérande de gauche puis celle de droite. Quel résultat obtient-il pour `9 4 -` ? Pourquoi son erreur reste-t-elle invisible sur `9 4 +` ?
4. Que doit faire votre fonction sur les expressions `3 +`, `+` et `3 4` ? Pour chacune, dire ce qui manque ou ce qui est en trop, et où votre code s'en aperçoit.
5. Écrire `hauteur_max(expression)`, qui renvoie le nombre maximal de valeurs présentes simultanément dans la pile. Donner sa valeur pour `3 5 + 2 *` et pour `1 2 3 4 + + +`. Que mesure cette quantité ?

Corrigé

Question 1. (4 points)

```

1 class Pile:
2     def __init__(self):
3         self._contenu = []
4     def est_vide(self):
5         return self._contenu == []
6     def empiler(self, valeur):
7         self._contenu.append(valeur)
8     def depiler(self):
9         assert not self.est_vide(), "depiler une pile vide"
10        return self._contenu.pop()
11    def sommet(self):
12        assert not self.est_vide(), "sommet d'une pile vide"
13        return self._contenu[-1]
14    def taille(self):
15        return len(self._contenu)

```

Le préfixe `_` signale que l'attribut relève de l'**implémentation**, pas de l'interface : on doit pouvoir remplacer la liste par autre chose sans qu'aucun utilisateur de la pile ait à changer une ligne.

Question 2. (6 points)

```

1 def evaluer(expression):
2     pile = Pile()
3     for jeton in expression.split():
4         if jeton in "+-*":
5             droite = pile.depiller()
6             gauche = pile.depiller()
7             if jeton == "+":
8                 pile.empiler(gauche + droite)
9             elif jeton == "-":
10                pile.empiler(gauche - droite)
11            else:
12                pile.empiler(gauche * droite)
13        else:
14            pile.empiler(int(jeton))
15    resultat = pile.depiller()
16    assert pile.est_vide(), "jetons en trop"
17    return resultat

```

Déroulé de `3 5 + 2 *` :

Jeton	Action	Pile après
3	empiler	[3]
5	empiler	[3, 5]
+	dépiler 5 puis 3, empiler 8	[8]
2	empiler	[8, 2]
*	dépiler 2 puis 8, empiler 16	[16]

Question 3. (4 points)

$9\ 4\ -\ =\ 5$ et $4\ 9\ -\ =\ -5$. L'élève qui dépile dans l'autre ordre calcule $4\ -\ 9$ et obtient -5 pour $9\ 4\ -$. Son erreur reste invisible sur $9\ 4\ +$ parce que l'addition est **commutative** : l'ordre des opérandes n'y change rien. Seul un opérateur non commutatif la révèle.

Question 4. (4 points)

Expression	Défaut	Où on le voit
3 +	une opérande manque	second depiler, pile vide
+	les deux manquent	premier depiler, pile vide
3 4	un opérateur manque	assert final, pile non vide

Les deux premiers cas sont détectés **pendant** le parcours, le troisième seulement **à la fin** : c'est pourquoi l'assertion finale est nécessaire. Sans elle, 3 4 renverrait 4, un résultat plausible et faux.

Question 5. (2 points)

```

1 def hauteur_max(expression):
2     pile, maxi = Pile(), 0
3     for jeton in expression.split():
4         if jeton in "+-*":
5             pile.depiler()
6             pile.depiler()
7             pile.empiler(0)
8         else:
9             pile.empiler(0)
10        maxi = max(maxi, pile.taille())
11    return maxi

```

$3\ 5\ +\ 2\ *\$ donne 2 ; $1\ 2\ 3\ 4\ +\ +\ +$ donne 4. Cette quantité mesure la **mémoire** nécessaire à l'évaluation : deux expressions de même longueur peuvent exiger des piles très différentes.

Critique

- ▶ **L'ordre de dépilement.** C'est le cœur de l'exercice, et la question 3 est construite pour que le candidat découvre que son test sur une addition ne prouvait rien.
- ▶ **L'assertion finale.** La plupart des copies l'omettent. Le cas 3 4 est justement celui qui ne lève aucune erreur naturelle : la fonction renvoie un nombre, et le nombre est faux.
- ▶ **Exposer_contenu.** Un candidat qui écrit `pile._contenu[-1]` dans `evaluer` contourne son interface. Le code fonctionne, mais la classe ne sert plus à rien.
- ▶ **La question 5 n'est pas décorative.** Elle demande de raisonner sur la pile sans évaluer l'expression : les valeurs empilées y sont arbitraires, seule la hauteur compte.

Portée pédagogique

L'exercice fait la démonstration, en un seul objet, de ce que sépare interface et implémentation : la classe de la question 1 est utilisée quatre fois ensuite sans que sa liste interne réapparaisse.

La progression va du contenant à ce qu'il permet, puis à ce qu'il détecte. Les questions 3 et 4 portent la même leçon sous deux formes : un programme qui donne un résultat plausible n'est pas un programme correct. C'est la raison d'être des deux assertions.

Sujet S4 — Exercice 1 : une file d'attente à deux piles

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

On dispose d'une bibliothèque qui ne fournit que des **piles** (dernier entré, premier sorti). On veut en construire une **file** (premier entré, premier sorti), dont l'interface est `est_vide`, `enfiler`, `defiler` et `taille`.

L'idée : deux piles. On empile les arrivées dans une pile d'**entrée** ; pour servir, on bascule toute l'entrée dans une pile de **sortie** — ce qui inverse l'ordre — et on sert depuis celle-ci.

1. Écrire la classe `FileDeuxPiles`. Vérifier sur la séquence « enfiler A, B, C puis défiler trois fois » que l'ordre obtenu est bien A, B, C.
2. Dérouler, en donnant le contenu des deux piles à chaque étape, la séquence : enfiler A, enfiler B, défiler, enfiler C, défiler, défiler. Quelles valeurs sont servies, et dans quel ordre ?
3. Un élève déclenche le transfert à chaque appel de `defiler`, sans vérifier que la pile de sortie est vide. Sur la séquence de la question 2, quelle valeur est servie au deuxième `defiler` ? Expliquer précisément pourquoi c'est faux.
4. On compte le nombre total de valeurs transférées d'une pile à l'autre. Donner ce nombre pour cent enfilements suivis de cent défilements, puis pour cinquante fois la séquence « enfiler, défiler ». Justifier par une phrase portant sur le nombre de transferts que subit **une** valeur donnée.
5. Que doit faire `defiler` sur une file vide ? Justifier ce choix par comparaison avec l'alternative « renvoyer `None` ».

Corrigé

Question 1. (5 points)

```

1 class FileDeuxPiles:
2     def __init__(self):
3         self._entree = []
4         self._sortie = []
5     def est_vide(self):
6         return not self._entree and not self._sortie
7     def enfiler(self, valeur):
8         self._entree.append(valeur)
9     def _transférer(self):
10        if not self._sortie:
11            while self._entree:
12                self._sortie.append(self._entree.pop())
13    def defiler(self):
14        self._transférer()
15        assert self._sortie, "defiler une file vide"
16        return self._sortie.pop()
17    def taille(self):
18        return len(self._entree) + len(self._sortie)

```

Les trois défilements donnent A, B, C.

Question 2. (5 points)

Opération	Entrée	Sortie	Servi
enfiler A	[A]	[]	
enfiler B	[A, B]	[]	
défiler	[]	[B]	A
enfiler C	[C]	[B]	
défiler	[C]	[]	B
défiler	[]	[]	C

Ordre servi : A, B, C — l'ordre d'arrivée.

Question 3. (5 points)

Au deuxième défiler, la version fautive rebascule C **par dessus** B dans la pile de sortie. Elle sert donc C, alors que B était arrivé avant.

Le transfert inverse l'ordre : l'appliquer à une pile de sortie non vide mélange deux générations d'arrivées, et l'inversion ne joue plus que sur la plus récente. La condition « seulement si la sortie est vide » est ce qui garantit le FIFO.

Question 4. (3 points)

Cent enfilements puis cent défilements : **100** transferts. Cinquante fois « enfiler, défiler » : **50** transferts.

Dans les deux cas, chaque valeur est transférée **exactement une fois** : elle passe de l'entrée à la sortie, et n'y revient jamais. Le coût total est donc proportionnel au nombre de valeurs, quelle que soit la façon dont enfilements et défilements s'entrelacent.

Question 5. (2 points)

défiler doit **échouer** — ici par une assertion. Renvoyer None rendrait indiscernables « la file est vide » et « la file contenait None », et repousserait l'erreur loin de son origine : l'appelant manipulerait un None sans savoir d'où il vient.

Critique

- **Le transfert inconditionnel.** C'est l'erreur centrale, et elle ne se voit pas sur la séquence de la question 1 : tant qu'on n'entrelace pas enfilements et défilements, les deux versions coïncident. La question 2 construit exactement l'entrelacement qui les sépare.
- **Compter deux transferts par valeur.** Un candidat qui répond 200 à la question 4 n'a pas vu qu'une valeur ne retourne jamais dans la pile d'entrée.
- **Vider l'entrée dans est_vide.** La file est vide si les **deux** piles le sont ; ne tester que la sortie ferait déclarer vide une file qui vient de recevoir une valeur.
- **« Renvoyer None est plus souple ».** La question 5 demande une justification, pas une préférence : l'argument attendu est l'ambiguïté du résultat, pas le confort d'écriture.

Portée pédagogique

L'exercice montre qu'une structure peut être *construite* à partir d'une autre, et que la correction de la construction tient à une condition d'une ligne. C'est un cas rare où l'erreur est à la fois minuscule et totale : le 'if' retiré, la file n'est plus une file.

La question 4 introduit l'idée de coût amorti sans en prononcer le nom : on ne demande pas une complexité, on demande combien de fois une valeur donnée est déplacée. La réponse — une seule — suffit à conclure.

Sujet S5 — Exercice 1 : une classe pour un stock, interface et implémentation

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Un atelier gère un stock de pièces. On spécifie une structure Stock par son **interface** seule :

Opération	Effet
ajouter(ref, n)	ajoute n pièces de la référence
retirer(ref, n)	retire n pièces de la référence
quantite(ref)	quantité disponible, 0 si absente
references()	liste triée des références en stock
est_vide()	vrai si aucune pièce n'est en stock

Une référence dont la quantité tombe à zéro **disparaît** du stock : elle n'apparaît plus dans `references()`.

1. Écrire une classe Stock implémentée par un dictionnaire. Dérouler l'état après : ajouter 4 clés, ajouter 10 vis, ajouter 3 clés, retirer 10 vis. Donner à chaque étape `references()`.
2. Écrire les préconditions de ajouter et de retirer, et les rendre exécutables. Donner cinq appels qui doivent être refusés, un par précondition violée. Vérifier qu'un appel refusé ne modifie pas le stock.
3. Écrire une **seconde** implémentation StockListe qui n'utilise aucun dictionnaire, mais une liste de couples [référence, nombre]. L'interface doit rester identique, au caractère près.
4. Écrire une fonction scenario(fabrique) qui joue la séquence de la question 1 sur une classe passée en paramètre et renvoie la trace des réponses. Vérifier qu'elle donne le même résultat pour les deux classes. Que prouve exactement cette égalité, et que ne prouve-t-elle pas ?
5. On compte les lignes examinées par quantite dans StockListe, sur un stock de cent références. Donner ce nombre pour la dernière référence ajoutée, puis pour la première. Comparer avec l'implémentation par dictionnaire. Conclure sur ce que l'interface ne dit pas.

Corrigé

Question 1. (4 points)

```

1 class Stock:
2     def __init__(self):
3         self._quantites = {}
4     def ajouter(self, reference, nombre):
5         assert nombre > 0, "ajout non strictement positif"
6         self._quantites[reference] = self._quantites.get(reference, 0) + nombre
7     def retirer(self, reference, nombre):
8         assert nombre > 0, "retrait non strictement positif"
9         disponible = self._quantites.get(reference, 0)
10        assert nombre <= disponible, "stock insuffisant"
11        if nombre == disponible:
12            del self._quantites[reference]
13        else:
14            self._quantites[reference] = disponible - nombre
15    def quantite(self, reference):
16        return self._quantites.get(reference, 0)
17    def references(self):
18        return sorted(self._quantites)
19    def est_vide(self):
20        return self._quantites == {}

```

Après	references()
départ	[]
+4 clés	[cle]
+10 vis	[cle, vis]
+3 clés	[cle, vis], quantité clé = 7
-10 vis	[cle]

Question 2. (5 points)

Préconditions : le nombre ajouté ou retiré est **strictement positif** ; un retrait ne peut excéder la quantité disponible, et une référence absente a une quantité de 0.

Cinq appels refusés : ajouter("cle", 0), ajouter("cle", -1), retirer("cle", 0), retirer("cle", 99), retirer("absent", 1).

Chaque assertion est placée **avant** toute modification : l'état du stock est donc inchangé après un refus.

Question 3. (5 points)

```

1 class StockListe:
2     def __init__(self):
3         self._lignes = []
4     def _index(self, reference):
5         for i, ligne in enumerate(self._lignes):
6             if ligne[0] == reference:
7                 return i
8         return -1
9     def ajouter(self, reference, nombre):
10        assert nombre > 0
11        i = self._index(reference)
12        if i == -1:
13            self._lignes.append([reference, nombre])
14        else:
15            self._lignes[i][1] += nombre
16    def retirer(self, reference, nombre):
17        assert nombre > 0
18        i = self._index(reference)
19        assert i != -1 and nombre <= self._lignes[i][1]
20        self._lignes[i][1] -= nombre
21        if self._lignes[i][1] == 0:
22            del self._lignes[i]
23    def quantite(self, reference):
24        i = self._index(reference)
25        return 0 if i == -1 else self._lignes[i][1]
26    def references(self):
27        return sorted(ligne[0] for ligne in self._lignes)
28    def est_vide(self):
29        return self._lignes == []

```

Question 4. (4 points)

scenario(Stock) == scenario(StockListe).

L'égalité prouve que les deux implémentations sont **indiscernables sur ce scénario** : un utilisateur qui ne passe que par l'interface ne peut pas dire laquelle il manipule. Elle ne prouve **pas** qu'elles se comportent identiquement sur tous les scénarios, ni qu'elles ont le même coût.

Question 5. (2 points)

Dernière référence ajoutée : 100 lignes examinées. Première : 1. Avec le dictionnaire, l'accès ne dépend pas de la position ni du nombre de références.

L'interface décrit ce que les opérations **font**, jamais ce qu'elles **coûtent**. Deux implémentations conformes peuvent être également correctes et très inégalement utilisables.

Critique

- ▶ **Oublier la disparition à zéro.** C'est une exigence de l'énoncé, pas un détail : une implémentation qui laisse une ligne à 0 échoue au scénario de la question 4 sur `references()`, et l'échec ne dit pas laquelle des deux classes est fautive.
- ▶ **Assertion après modification.** Placer le contrôle de stock après la soustraction laisse une quantité négative en mémoire avant d'échouer. La question 2 exige explicitement de vérifier l'état après un refus.
- ▶ **« Le test passe, donc les deux classes sont équivalentes ».** La question 4 demande la limite de cette conclusion, et c'est là que se joue la différence entre un candidat qui teste et un candidat qui sait ce qu'un test établit.
- ▶ **Accéder à `_lignes` depuis `scenario`.** Le scénario ne doit utiliser que l'interface, sinon il cesse de valoir pour les deux classes.

Portée pédagogique

L'exercice fait écrire **deux** fois la même structure pour rendre visible ce qui, autrement, reste une phrase de cours : l'interface est ce qui survit au changement d'implémentation.

La question 4 en donne la vérification pratique — un même scénario, deux classes, une même trace — et la question 5 en donne la limite, sans laquelle l'idée serait fausse dans l'autre sens : rien, dans une interface, n'oblige deux implémentations à être également efficaces.

Sujet S6 — Exercice 1 : deux représentations d'un même graphe

Durée conseillée : 70 minutes. Cet exercice est indépendant des deux autres du sujet.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Énoncé

Un graphe orienté à cinq sommets, numérotés de 0 à 4, est donné par sa matrice d'adjacence. La case ligne i , colonne j vaut 1 s'il existe une flèche $i \rightarrow j$.

```
1 matrice = [
2     [0, 1, 0, 1, 0],
3     [0, 0, 1, 0, 0],
4     [0, 0, 0, 0, 1],
5     [0, 0, 1, 0, 0],
6     [0, 0, 0, 0, 0],
7 ]
```

1. Lister toutes les flèches sous la forme $i \rightarrow j$. Donner, pour chaque sommet, son degré sortant et son degré entrant. Vérifier une relation entre la somme des degrés sortants et le nombre de flèches.
2. Justifier, en citant deux cases précises, que ce graphe est **orienté** et non pas non orienté.
3. Écrire `matrice_vers_listes` et `listes_vers_matrice`. Donner les listes de successeurs obtenues, et vérifier que la conversion inverse redonne la matrice de départ.
4. Pour répondre à « le sommet i pointe-t-il vers j ? », on compte le nombre de cases examinées. Donner ce nombre, dans chaque représentation, pour (0,3) puis pour (4,0). Commenter le cas (4,0).
5. Comparer l'occupation mémoire des deux représentations sur ce graphe, puis sur un graphe complet à cinq sommets, puis sur un graphe sans aucune flèche. En déduire dans quel cas chaque représentation est préférable.

Corrigé

Question 1. (4 points)

Flèches : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 2 \rightarrow 4, 3 \rightarrow 2$. Il y en a 5.

Sommet	0	1	2	3	4
Degré sortant	2	1	1	1	0
Degré entrant	0	1	2	1	1

La somme des degrés sortants vaut 5, soit le nombre de flèches : chaque flèche sort d'exactly un sommet. La somme des degrés entrants vaut aussi 5, pour la même raison.

Question 2. (2 points)

`matrice[0][1]` vaut 1 et `matrice[1][0]` vaut 0 : la matrice n'est pas symétrique. Le sommet 0 pointe vers 1 sans que 1 pointe vers 0.

Question 3. (5 points)

```

1 def matrice_vers_listes(m):
2     return {i: [j for j, valeur in enumerate(ligne) if valeur == 1]
3             for i, ligne in enumerate(m)}
4
5 def listes_vers_matrice(listes):
6     n = len(listes)
7     m = [[0] * n for _ in range(n)]
8     for i, successeurs in listes.items():
9         for j in successeurs:
10             m[i][j] = 1
11     return m

```

Listes : $\{0: [1, 3], 1: [2], 2: [4], 3: [2], 4: []\}$. La conversion inverse redonne exactement la matrice de départ.

Question 4. (5 points)

Question	Matrice	Listes
(0,3)	1 case	2 cases
(4,0)	1 case	0 case

La matrice répond toujours en une seule lecture, quelle que soit la question. Les listes dépendent du degré sortant : pour (4,0), le sommet 4 n'a aucun successeur, la boucle ne s'exécute pas et la réponse est immédiate. Les listes sont donc parfois plus rapides, jamais garanties de l'être.

Question 5. (4 points)

Graphe	Matrice	Listes
celui de l'énoncé (5 flèches)	25	5
complet (20 flèches)	25	20
sans flèche	25	0

La matrice occupe n^2 cases **quel que soit** le nombre de flèches ; les listes en occupent autant qu'il y a de flèches. Pour un graphe **creux** — peu de flèches devant n^2 — les listes sont bien plus économiques. Pour un graphe **dense**, l'écart se referme et la matrice reprend l'avantage par la rapidité de son accès.

Critique

- **Lire la matrice en colonnes.** Confondre degré entrant et degré sortant est l'erreur la plus fréquente. La vérification par la somme, en question 1, permet au candidat de s'en apercevoir seul.

- **Croire les listes toujours meilleures.** La question 4 montre le contraire sur $(0, 3)$: deux lectures au lieu d'une. Le cas $(4, 0)$ est là pour empêcher la conclusion inverse.
- **Oublier le sommet isolé dans la conversion.** Le sommet 4 doit figurer dans les listes avec une liste vide. L'omettre casse la conversion inverse, et la vérification de la question 3 le révèle.
- **Compter la mémoire en « nombre de listes ».** Ce qui compte est le nombre total de successeurs stockés, pas le nombre d'entrées du dictionnaire — lequel vaut toujours n .

Portée pédagogique

L'exercice fait constater qu'une même information admet deux représentations exactes, et que le choix entre elles ne se juge pas dans l'absolu : il dépend de la densité du graphe et de la question qu'on lui pose.

La conversion aller-retour de la question 3 fournit au candidat sa propre vérification : si la matrice reconstruite diffère, l'une des deux fonctions est fautive, et il le voit sans corrigé. Les questions 4 et 5 séparent deux critères qu'on résume souvent en un seul mot « efficace » — le temps d'une requête et la place occupée — et montrent qu'ils ne désignent pas le même gagnant.

Situation pratique P6 — implémenter une file à partir de son interface

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier `file.py` contient la spécification de l'interface et le jeu de tests. Aucune implémentation n'est donnée.

Opération	Effet
<code>est_vide()</code>	vrai si la file ne contient rien
<code>enfiler(v)</code>	ajoute v en fin de file
<code>defiler()</code>	retire et renvoie la valeur de tête
<code>tete()</code>	renvoie la valeur de tête sans la retirer
<code>taille()</code>	nombre de valeurs présentes

`defiler` et `tete` ont pour précondition que la file ne soit pas vide.

Travail demandé

1. Écrire la classe `File`. L'attribut qui porte les données doit être préfixé d'un `_` et ne doit apparaître nulle part ailleurs que dans les méthodes de la classe.
2. Rendre les deux préconditions exécutables, et vérifier que `defiler` et `tete` sur une file vide sont refusés.
3. Exécuter le jeu de tests fourni. Il doit passer entièrement, y compris le test d'entrelacement où l'on enfiler pendant qu'on défile.
4. Vérifier sur cent valeurs que l'ordre de sortie est exactement l'ordre d'entrée.
5. Répondre par écrit, en commentaire : votre implémentation utilise-t-elle `pop(0)` ? Si oui, dire ce que coûte cette opération sur une liste Python, et nommer une structure de la bibliothèque standard qui n'a pas ce défaut. **Ne pas** modifier votre code : la question porte sur l'analyse, pas sur l'optimisation.

Ce qui est évalué

Critère	Points
Le jeu de tests fourni passe entièrement	8
Les deux préconditions sont vérifiées	4
L'attribut interne n'est utilisé que dans la classe	3
La réponse écrite est juste et nomme une alternative	5

Réponse attendue à la question 5

`pop(0)` retire le premier élément d'une liste, ce qui oblige à décaler tous les suivants : le coût est proportionnel à la taille de la file, à chaque défilement. `collections.deque` offre un retrait en tête à coût constant.

Critique

- **Confondre tête et defiler.** La première ne modifie rien. Le jeu de tests le vérifie explicitement en comparant la taille avant et après.
- **Enfiler en tête.** `insert(0, v)` avec un `pop()` en fin donne une file correcte, mais déplace le coût au lieu de le supprimer. La question 5 reste valide dans ce cas, et la réponse doit le refléter.
- **Modifier son code après la question 5.** La consigne l'interdit : ce qui est évalué est l'analyse d'un coût, pas la capacité à l'éviter.
- **Exposer `_contenu`.** Le jeu de tests ne l'utilise jamais ; un candidat qui s'en sert dans ses propres essais s'écarte de l'interface qu'il a écrite.

Portée pédagogique

La situation dissocie deux jugements souvent confondus : une implémentation est **correcte** si elle satisfait l'interface, et **efficace** si son coût convient à l'usage. La question 5 demande de reconnaître un défaut d'efficacité dans un code par ailleurs juste, sans le corriger — ce qui est exactement la posture d'une relecture.

Situation pratique P8 — passer d'une matrice d'adjacence aux listes de successeurs

Durée : 1 heure. Travail sur machine, à partir du document fourni.

Contenu original Nexus — ce n'est pas un sujet officiel d'examen.

Document fourni

Le fichier `graphes.py` contient les deux spécifications et le jeu de tests.

```

1 def matrice_vers_listes(matrice):
2     """Précondition : matrice carree de 0 et de 1.
3     Postcondition : dict sommet -> liste croissante de successeurs."""
4     ...
5
6 def listes_vers_matrice(listes):
7     """Précondition : les successeurs sont des sommets declares."""
8     ...

```

La matrice de travail :

	0	1	2	3
0	0	1	1	0
1	0	0	0	1
2	0	0	0	1
3	0	0	0	0

Travail demandé

1. Compléter les deux fonctions. Un sommet sans successeur doit apparaître dans les listes, avec une liste **vide**.
2. Rendre les préconditions exécutables : matrice non carrée, valeur autre que 0 ou 1, successeur hors des sommets déclarés. Vérifier que les trois sont refusées.
3. Vérifier l'**aller-retour** sur quatre graphes : celui de l'énoncé, un graphe sans aucune flèche, un graphe complet à quatre sommets, et un graphe à deux sommets portant chacun une boucle sur lui-même.
4. Écrire deux fonctions de comptage des flèches, une par représentation, et vérifier qu'elles coïncident sur les quatre graphes.
5. Répondre par écrit, en commentaire : sur le graphe complet à quatre sommets, combien de cases la matrice occupe-t-elle, et combien de valeurs les listes stockent-elles ? Même question sur le graphe sans flèche. Conclure en une phrase.

Ce qui est évalué

Critère	Points
Le jeu de tests fourni passe entièrement	7
Les trois préconditions sont refusées	4
L'aller-retour est vérifié sur les quatre graphes	4
Les deux comptages coïncident	2
La réponse écrite donne les quatre nombres et conclut	3

Réponse attendue à la question 5

Graphe complet à quatre sommets : la matrice occupe 16 cases, les listes stockent 12 valeurs. Graphe sans flèche : la matrice occupe toujours 16 cases, les listes n'en stockent aucune.

La matrice occupe n^2 cases quel que soit le nombre de flèches ; les listes en occupent autant qu'il y a de flèches. L'écart est d'autant plus grand que le graphe est creux.

Critique

- **Omettre les sommets isolés.** Une compréhension qui ne garde que les sommets ayant au moins un successeur casse l'aller-retour : la matrice reconstruite est trop petite. Le graphe sans flèche est le test qui le révèle.
- **Les boucles sur soi-même.** La diagonale peut valoir 1 ; rien ne l'interdit, et le quatrième graphe est là pour qu'un candidat qui la filtre s'en aperçoive.
- **Vérifier l'aller-retour dans un seul sens.** Reconstruire la matrice à partir des listes ne suffit pas : il faut comparer à la matrice de départ, pas à une matrice quelconque.
- **Compter les entrées du dictionnaire.** Elles valent toujours n . Ce qui mesure la mémoire est le nombre total de successeurs stockés.

Portée pédagogique

La situation tient en deux fonctions courtes, et tout l'intérêt est dans les cas de test : le graphe vide, le graphe complet, les boucles. C'est en les traversant qu'un candidat découvre les hypothèses implicites de son code.

L'aller-retour fournit une vérification que le candidat peut mener seul, sans corrigé : si la matrice reconstruite diffère, l'une des deux fonctions est fausse. La question 5 relie enfin le choix de représentation à une quantité mesurée, pas à une préférence.

Exercice 5

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Version aménagée — Extrait Structures de données

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Pile ou file ? (C6, C7)

On vérifie si les parenthèses d'une expression sont bien fermées.

Expression : ([])

Étape 1. Complète la lecture, caractère par caractère.

Règle : une ouvrante s'empile ; une fermante doit correspondre au **sommet**.

Caractère lu	Action	Pile après
(	empiler	[(]
[	...	
]	...	
)	...	

Étape 2. À la fin, la pile est-elle vide ?

☐ Oui

☐ Non

L'expression est donc :

Étape 3. Quelle structure convient ici ? Entoure.

☐ une pile (LIFO)

☐ une file (FIFO)

Aide : la dernière ouvrante posée est la première à refermer.

Exercice 2 — Deux représentations d'un graphe (C12, C13, C14)

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

La case ligne i , colonne j vaut 1 s'il existe une flèche $i \rightarrow j$.

Étape 1. Lis la matrice ligne par ligne. La première est faite.

Ligne	Flèches partant de ce sommet
0	$0 \rightarrow 1$ et $0 \rightarrow 3$
1	
2	
3	

Étape 2. Le sommet 2 a-t-il des successeurs ?

☐ Oui

☐ Non

Que signifie cette ligne entièrement à zéro ?

Étape 3. Complète la représentation par listes de successeurs.

```
1 successeurs = {
2     0: [1, 3],
3     1: [_____],
4     2: [_____],
5     3: [_____],
6 }
```

Étape 4. Combien de flèches ce graphe compte-t-il au total ?

.....

Contrôle : ce nombre doit être égal au nombre de 1 dans la matrice.

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
```

```

8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15     def est_equilibre(expr):
16         p = Pile()
17         paires = {'(': '(', '[': '[', '}': '{'}
18         for c in expr:
19             if c in '([':
20                 p.empiler(c)
21             elif c in ')]}':
22                 if p.est_vide() or p.depiler() != paires[c]:
23                     return False
24         return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```



◆ Corrigés

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).



7 Conduire un projet informatique

7.1 Conduire un projet informatique en Terminale

THÉORÈME Contrat horaire officiel

Le programme réserve **au moins 25 %** de l'horaire total de la spécialité à la conception et à l'élaboration de projets conduits par les élèves. Sur une référence de 216 heures (6 heures hebdomadaires pendant 36 semaines), cela représente au moins 54 heures. Si l'horaire effectivement assuré diffère, le plan annuel est ajusté pour préserver la proportion minimale d'un quart.

Programme d'enseignement et Définition de l'épreuve

Cette unité met en œuvre une obligation du **Programme d'enseignement** (MENE1921247A). Elle ne constitue pas une nouvelle partie de l'examen et **ne remplace pas l'épreuve pratique**. La **Définition de l'épreuve** relève d'une autorité distincte : la note de service MENE2516123N définit, à compter de la session 2026, une partie écrite de 3 h 30 et une partie pratique d'une heure, chacune notée sur 20, avec des poids respectifs de 0,75 et 0,25. Le projet prépare des compétences utiles, mais son volume, ses livrables et sa grille ci-dessous relèvent de l'enseignement et non du format de l'épreuve.

Prérequis

Avant le cadrage, chaque équipe vérifie qu'elle sait :

- écrire, exécuter et tester un programme Python ;
- mobiliser au moins une notion du programme de Terminale NSI ;
- conserver un historique des versions et attribuer chaque contribution ;
- partager uniquement des données licites, documentées et dépourvues de données personnelles d'élèves ;
- expliquer une décision technique à l'oral et par écrit.

Question de départ et périmètre

Le projet répond à une question formulée par l'équipe et validée avec le professeur. Le thème peut provenir d'une autre discipline ou mobiliser, par exemple, une base de données, un graphe, une structure complexe, un traitement d'image ou de son, un site Web, un objet connecté ou un module d'intelligence artificielle. Le choix reste compatible avec les ressources disponibles et une réalisation complète en 54 heures de référence.

Le cahier des charges distingue :

- un **produit minimal viable**, indispensable et démontrable ;
- des fonctionnalités souhaitables, réalisées seulement après validation du socle ;
- les limites connues, hypothèses, dépendances, licences et données utilisées ;
- les critères observables qui permettront d'accepter ou de refuser chaque fonction.

ERREUR FRÉQUENTE

Accumuler des fonctionnalités sans produit minimal testable. Le programme demande une ambition raisonnable : une fonctionnalité non indispensable est reportée si elle met en danger l'aboutissement, les tests ou la documentation.

Jalons communs — plan de référence de 54 heures

Chaque jalon se termine par un point d'étape avec le professeur. L'équipe présente une preuve vérifiable, consigne la décision et met à jour les objectifs.

Jalon	Travail attendu	Durée	Preuve de passage
J0	Problème, équipe, responsabilités, contraintes et risques	3 h	Note de cadrage validée
J1	Cahier des charges, cas d'usage et critères d'acceptation	6 h	Spécification versionnée
J2	Architecture, données, interfaces et prototype exploratoire	6 h	Schéma et prototype exécutable
J3	Développement du produit minimal viable	9 h	Démonstration du socle
J4	Deuxième itération et intégration des contributions	9 h	Version intégrée et journal de décisions
J5	Tests normaux, cas limites, correction et robustesse	9 h	Rapport de tests reproductible
J6	Documentation utilisateur et technique, licences, limites	6 h	Documentation relue
J7	Stabilisation, démonstration, oral et bilan individuel	6 h	Version finale et rétrospective
Total		54 h	

À chaque point d'étape, l'équipe répond à quatre questions : qu'est-ce qui fonctionne ? Quelle preuve le montre ? Quel obstacle reste prioritaire ? Quel objectif faut-il conserver, réduire ou redéfinir ? Une modification de périmètre n'efface jamais l'historique de la décision.

Livrables obligatoires

1. une note de cadrage et un cahier des charges datés ;
2. un découpage en tâches, des responsabilités et un calendrier de jalons ;
3. les sources exécutables avec historique des versions et instructions de lancement ;
4. un jeu de tests couvrant les fonctions centrales et des cas limites ;
5. une documentation utilisateur et une documentation technique ;
6. un registre des données, bibliothèques, licences, hypothèses et limites ;
7. une démonstration finale reproductible ;
8. un bilan individuel identifiant contributions, apprentissages et améliorations.

Grille d'évaluation

La grille est communiquée dès J0. L'évaluation croise une production collective et des preuves individuelles ; elle ne se réduit ni au volume de code ni à la seule démonstration finale.

Critère	Poids	Preuve principale
Problème, périmètre et critères d'acceptation	15 %	Cadrage et cahier des charges
Architecture et choix techniques justifiés	15 %	Schémas et journal de décisions
Exactitude et fonctionnement du produit	25 %	Version exécutable
Tests, cas limites et traitement des erreurs	20 %	Rapport de tests
Documentation, reproductibilité et provenance	10 %	Guides et registre des sources
Organisation, jalons et contributions individuelles	10 %	Historique et bilans
Démonstration et argumentation orale	5 %	Présentation finale
Total	100 %	

Un produit non exécutable ne peut pas obtenir les points de fonctionnement sur la base d'une simple intention. Inversement, un projet modeste mais complet, testé, documenté et argumenté satisfait mieux le contrat qu'un prototype ambitieux impossible à reproduire.

Dossiers de projets authentiques

Quatre dossiers types sont proposés pour guider les équipes dans la définition d'un périmètre équilibré, reliant les thèmes du programme de Terminale à un problème concret.

Dossier 1 — Système de gestion et consultation de données (SGBD et Web)

- **Problématique** : Concevoir une application de gestion et d'interrogation pour une médiathèque ou une association, avec persistance des données et interface de consultation.
- **Notions mobilisées** : Modèle relationnel, schéma relationnel (clés primaires et étrangères), requêtes SQL (SELECT, JOIN, agrégation, mises à jour), interface Web ou locale, séparation modèle/-vue.
- **Produit minimal viable (MVP)** : Base de données SQLite avec au moins 3 tables reliées, script d'initialisation, interface permettant la recherche multicritère et l'insertion contrôlée avec validation de saisie.
- **Extensions conditionnelles** : Authentification avec mot de passe haché, export de statistiques en format ouvert (CSV/JSON), pagination des résultats.
- **Critères de recette** : Intégrité référentielle garantie, requêtes paramétrées sans concaténation brute, tests unitaires sur les opérations CRUD élémentaires.

Dossier 2 — Planificateur d'itinéraires et optimisation sur graphes

- **Problématique** : Développer un outil d'aide à la mobilité calculant les itinéraires optimaux sur un réseau de transport ou routier.
- **Notions mobilisées** : Graphes orientés et pondérés (listes et dictionnaires d'adjacence), parcours en largeur (BFS), algorithme de Dijkstra, structures de données linéaires et files de priorité.
- **Produit minimal viable (MVP)** : Chargement d'un réseau depuis un fichier structuré, calcul du plus court chemin entre deux sommets, affichage textuel de l'itinéraire détaillé et du coût cumulé.
- **Extensions conditionnelles** : Prise en compte de contraintes horaires (correspondances), détection des nœuds critiques de coupure, rendu graphique du graphe.
- **Critères de recette** : Exactitude du chemin comparé à un oracle vérifié, gestion propre des graphes non connexes (détection d'absence de chemin), temps d'exécution maîtrisés.

Dossier 3 — Moteur d'indexation textuelle et compression par arbres

- **Problématique** : Réaliser un moteur de recherche local capable d'indexer une collection de documents textuels et de compresser les archives textuelles.
- **Notions mobilisées** : Arbres binaires de recherche (ABR), arbres de Huffman, parcours d'arbres (infixe, préfixe), dictionnaires Python, analyse fréquentielle, complexité algorithmique.

- ▶ **Produit minimal viable (MVP)** : Construction d'un index inversé reliant chaque mot aux documents sources, recherche par mot-clé avec score de pertinence, module de compression et décompression de Huffman sans perte.
- ▶ **Extensions conditionnelles** : Recherche booléenne combinatoire (ET / OU / SAUF), élimination des mots vides (stop-words), mesure comparée des taux de compression.
- ▶ **Critères de recette** : Décompression bit à bit rigoureusement identique à l'original (somme de contrôle vérifiée), recherche en temps logarithmique moyen sur les index arborescents.

Dossier 4 — Simulateur de réseau et protocoles de communication

- ▶ **Problématique** : Construire un simulateur d'échanges de messages entre plusieurs entités communicantes reliées par des routeurs virtuels.
- ▶ **Notions mobilisées** : Protocoles de routage (RIP / OSPF), architecture client-serveur avec sockets ou simulation à événements discrets, tables de routage, encapsulation des paquets, détection de pertes.
- ▶ **Produit minimal viable (MVP)** : Simulation de 4 à 6 routeurs interconnectés, calcul des routes par vecteur de distance (algorithme de Bellman-Ford / RIP simplifié), acheminement pas à pas d'un paquet.
- ▶ **Extensions conditionnelles** : Détection automatique de panne d'un lien et convergence vers une route alternative, gestion du décompte de sauts (TTL), simulation d'un protocole d'acquittement.
- ▶ **Critères de recette** : Détection des boucles de routage, convergence démontrée après modification de la topologie, journalisation horodatée de chaque transfert de paquet.

Bilan final

Le bilan collectif compare le résultat au cahier des charges. Le bilan individuel explicite une contribution précise, un problème rencontré, la méthode utilisée pour le résoudre et une amélioration possible. Les limites connues sont documentées ; elles ne sont ni masquées ni présentées comme des fonctionnalités achevées.

Guide de pilotage et d'évaluation continue

La démarche de projet en Terminale requiert un encadrement méthodologique régulier afin d'éviter les dérives classiques (dispersion technique, surcharge, travail non partagé).

Indicateurs observables par jalon

- ▶ **J0-J1 (Cadrage et spécification)** : Le problème doit être formulé sans jargon excessif. Vérifier que le MVP est restreint et réalisable en 15 heures de codage effectif. Exiger un schéma conceptuel ou relationnel avant toute ligne de code.
- ▶ **J2-J3 (Architecture et MVP)** : Chaque membre doit disposer d'un rôle identifié (ex. modélisation données, logique algorithmique, interface/IHM, protocole de test). Le prototype doit compiler et exécuter le cas nominal le plus simple.
- ▶ **J4-J5 (Intégration et validation)** : Le professeur vérifie l'historique des commits : les contributions doivent être réparties dans le temps et entre les élèves. Le jeu de tests automatisé doit comporter au moins un cas d'erreur et un cas limite.
- ▶ **J6-J7 (Documentation et soutien)** : La documentation utilisateur doit permettre à un pair d'installer et d'exécuter le projet en moins de 5 minutes sans intervention des auteurs.

Détection des difficultés et remédiations types

- ▶ **Asymétrie de contribution** : Lorsqu'un élève prend l'ascendant ou qu'un autre décroche, redéfinir individuellement un sous-module ou le plan de test unitaire sous la responsabilité exclusive de l'élève en difficulté.
- ▶ **Blocage sur une bibliothèque externe** : Si une bibliothèque graphique ou un framework Web pose problème pendant plus de 2 heures, basculer sur une interface textuelle ou une interface

simple (module standard Python). Rappeler que l'évaluation porte sur les notions NSI, pas sur la maîtrise d'outils tiers hors programme.

- **Intégrité académique et IA générative** : Tout extrait de code issu d'une documentation officielle, d'un forum ou d'un assistant doit être explicité en commentaire avec sa source. Lors de la soutenance orale, chaque élève est interrogé individuellement sur la justification algorithmique et la complexité des fonctions qu'il a signées.

CONDUIRE UN PROJET INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Le problème auquel répond un projet informatique doit être formulé :
 - A. de façon explicite, avec des critères permettant de constater qu'il est résolu.
 - B. de la manière la plus large possible, pour ne rien s'interdire.
 - C. seulement une fois le programme écrit.
 - D. par le professeur seul, et jamais par l'équipe.
2. [Q2] Dans un projet mené en équipe, répartir le travail suppose avant tout :
 - A. que chacun écrive le même code de son côté, pour comparer ensuite.
 - B. de définir qui fait quoi et comment les parties s'assemblent.
 - C. qu'une seule personne programme et que les autres observent.
 - D. de tirer les tâches au sort.

Capacité C2

3. [Q3] Un point d'étape est utile à condition de porter sur :
 - A. le nombre de lignes écrites depuis la dernière séance.
 - B. l'impression générale des membres de l'équipe.
 - C. des critères observables : ce qui fonctionne, ce qui est testé, ce qui reste à faire.
 - D. la seule date de rendu finale.
4. [Q4] À mi-parcours, l'équipe constate qu'une fonctionnalité prévue ne sera pas terminée. La conduite adaptée est :
 - A. conserver le périmètre initial et rendre un projet inachevé.
 - B. abandonner le projet et en commencer un autre.
 - C. ajouter des membres à l'équipe pour rattraper le retard.
 - D. réduire le périmètre en accord avec le professeur, et livrer un ensemble cohérent et testé.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C1	B
Q3	C2	C
Q4	C2	D

Diagnostic des réponses erronées

► Q1

- **B** — Un énoncé trop large empêche de savoir quand le projet est terminé, et conduit à une ambition impossible à tenir dans le temps imparti. *Renvoi : C1, cahier des charges d'un projet.*
- **C** — L'élève inverse l'ordre : écrire avant de savoir ce que l'on cherche à résoudre expose à produire un programme sans destinataire. *Renvoi : C1, cahier des charges d'un projet.*
- **D** — La conception fait partie du travail évalué. Le professeur valide et recadre, mais l'équipe formule. *Renvoi : C1, conception en équipe.*

► Q2

- **A** — Dupliquer le travail gaspille le temps disponible et laisse entière la difficulté véritable : l'assemblage des parties. *Renvoi : C1, travail en équipe.*
- **C** — L'évaluation porte sur la contribution de chacun ; concentrer le développement prive les autres de tout travail à présenter. *Renvoi : C1, travail en équipe.*
- **D** — La répartition doit tenir compte des compétences et des dépendances entre tâches, ce que le hasard ignore. *Renvoi : C1, travail en équipe.*

► Q3

- **A** — Le volume de code ne mesure pas l'avancement : une fonction courte et testée vaut mieux que cent lignes qui ne s'exécutent pas. *Renvoi : C2, points d'étape et critères.*
- **B** — Une impression ne se vérifie pas et ne permet aucune décision. Un jalon doit reposer sur des faits constatables. *Renvoi : C2, points d'étape et critères.*
- **D** — Ne regarder que l'échéance finale prive de toute occasion de corriger la trajectoire pendant qu'il en est encore temps. *Renvoi : C2, points d'étape et critères.*

► Q4

- **A** — Un produit inachevé ne se démontre pas. Ajuster l'ambition fait explicitement partie de la démarche de projet. *Renvoi : C2, adapter les objectifs.*
- **B** — Repartir de zéro à mi-parcours perd tout le travail accompli et rend l'échéance encore moins atteignable. *Renvoi : C2, adapter les objectifs.*
- **C** — Renforcer une équipe en cours de route coûte du temps d'explication à ceux qui avancent, et ne rattrape pas le retard. *Renvoi : C2, adapter les objectifs.*

Version aménagée — Extrait Projet annuel

Consignes séquencées, mise en page aérée, énoncés allégés.

Cette fiche ne remplace pas le dossier de projet : elle en séquence la préparation. Les exigences évaluées restent celles de la grille critériée.

Partie A — Poser le besoin (C1)

Étape 1. Écris en **une seule phrase** ce que ton programme permettra de faire.

« Mon programme permet à de »

Contrôle : si ta phrase ne tient pas sur une ligne, le projet n'est pas encore assez précis.

Étape 2. Coche ce que tu remettras à la fin.

Livrable	Prévu ?
Le programme	oui
Un jeu de tests	...
Une documentation	...
Une présentation orale	...

Partie B — Découper en jalons (C2)

Étape 3. Complète le plan. Le premier jalon est donné en modèle.

Rappel : un jalon porte une **date** et un **état vérifiable**.

Jalon	Date	État vérifiable ce jour-là
J1	..	le programme démarre et affiche un menu
J2	..	
J3	..	
J4	..	

Étape 4. Ton premier jalon produit-il quelque chose qui **tourne** ?

☐ Oui

☐ Non

Aide : si la réponse est non, échange J1 avec un jalon plus simple.

Étape 5. Combien de jours séparent ton dernier jalon de la date de rendu ?

..... jours

☐ c'est une marge suffisante

☐ il faut avancer les jalons

Partie C — Se répartir le travail (C2)

Étape 6. Pour chaque jalon, écris **un seul** prénom responsable.

Jalon	Responsable
J1	
J2	
J3	
J4	

Contrôle : un jalon sans responsable est un jalon que personne ne fait.

Apprendre, s'entraîner, se tester, progresser : la collection Nexus

Réussite accompagne chaque élève jusqu'à la maîtrise.

- Un cours structuré en strates, avec la rubrique « À la machine » : chaque notion se reproduit au clavier.
- » Du code Python vérifié par exécution : aucune sortie de programme imprimée sans avoir tourné.
- ① Des méthodes pas à pas avec exemples résolus commentés et pièges à éviter.
- ♦♦ Trois parcours d'exercices gradués et chronométrés, avec coups de pouce.
- ↪ QCM diagnostiques, auto-évaluation par compétences et sujets aux formats officiels.
- ⊗ Des fiches de remédiation ciblées, reliées aux erreurs réellement diagnostiquées.